

PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG THÔNG TIN

PHAN ĐÌNH SINH

HỆ THỐNG THÔNG TIN – KHOA CÔNG NGHỆ THÔNG TIN



Nội dung

- Giới thiệu
- Ngôn ngữ mô hình hóa UML
- Phân tích hướng đối tượng
- Thiết kế hướng đối tượng

1. Giới thiệu

- Hệ thống thông tin
- Quy trình phát triển hệ thống thông tin
- Các cách tiếp cận phân tích và thiết kế hệ thống thông tin
- Công cụ hỗ trợ phát triển hệ thống thông tin
- Các khái niệm cơ bản về hướng đối tượng
- Các bước phân tích và thiết kế hướng đối tượng

1.1. Hệ thống thông tin

- Định nghĩa
- Vai trò
- Tầm quan trọng
- Các thành phần của hệ thống thông tin
- Phân loại hệ thống thông tin

1.1.1. Định nghĩa

- Hệ thống thông tin là một tập hợp các thành phần liên kết với nhau để thu thập, xử lý, lưu trữ và phân phối thông tin nhằm hỗ trợ việc ra quyết định, phối hợp và kiểm soát trong một tổ chức

1.1.2. Vai trò

- Hệ thống thông tin giúp cải thiện hiệu quả hoạt động, tăng cường khả năng cạnh tranh, hỗ trợ ra quyết định và quản lý thông tin trong doanh nghiệp và xã hội

1.1.3. Tầm quan trọng của hệ thống thông tin

- Hệ thống thông tin đóng vai trò quan trọng trong việc tối ưu hóa quy trình kinh doanh, nâng cao chất lượng dịch vụ và sản phẩm và thúc đẩy sự phát triển bền vững của tổ chức.

1.1.4. Các thành phần của hệ thống thông tin

- **Phần cứng (Hardware):** Các thiết bị vật lý như máy tính, máy chủ, thiết bị mạng và các thiết bị đầu cuối
- **Phần mềm (Software):** Các chương trình và ứng dụng được sử dụng để xử lý và quản lý thông tin

1.1.4. Các thành phần của hệ thống thông tin

- **Dữ liệu (Data):** Thông tin được thu thập, lưu trữ và xử lý trong hệ thống
- **Con người (People):** Người sử dụng và quản lý hệ thống thông tin
- **Quy trình (Processes):** Các quy trình và thủ tục được thiết lập để thu thập, xử lý và phân phối thông tin

1.1.5. Phân loại hệ thống thông tin

- **Hệ thống thông tin quản lý (MIS):** Hỗ trợ quản lý và điều hành các hoạt động kinh doanh của tổ chức.
- **Hệ thống hỗ trợ quyết định (DSS):** Hỗ trợ việc ra quyết định trong các tình huống phức tạp và không chắc chắn.

1.1.5. Phân loại hệ thống thông tin

- **Hệ thống thông tin điều hành (EIS):** Cung cấp thông tin tổng hợp và phân tích cho các nhà quản lý cấp cao để hỗ trợ việc ra quyết định chiến lược.
- **Các loại hệ thống thông tin khác:** Hệ thống thông tin văn phòng (OIS), hệ thống thông tin địa lý (GIS), hệ thống thông tin doanh nghiệp (ERP),...

1.2. Quy trình phát triển hệ thống thông tin

- Các giai đoạn phát triển hệ thống thông tin
- Mô hình phát triển hệ thống thông tin

1.2.1. Các giai đoạn phát triển hệ thống thông tin

- **Khảo sát hiện trạng:** Thu thập thông tin về hệ thống hiện tại, xác định các vấn đề và cơ hội cải tiến.
- **Phân tích yêu cầu:** Xác định và mô tả các yêu cầu của người dùng và hệ thống.

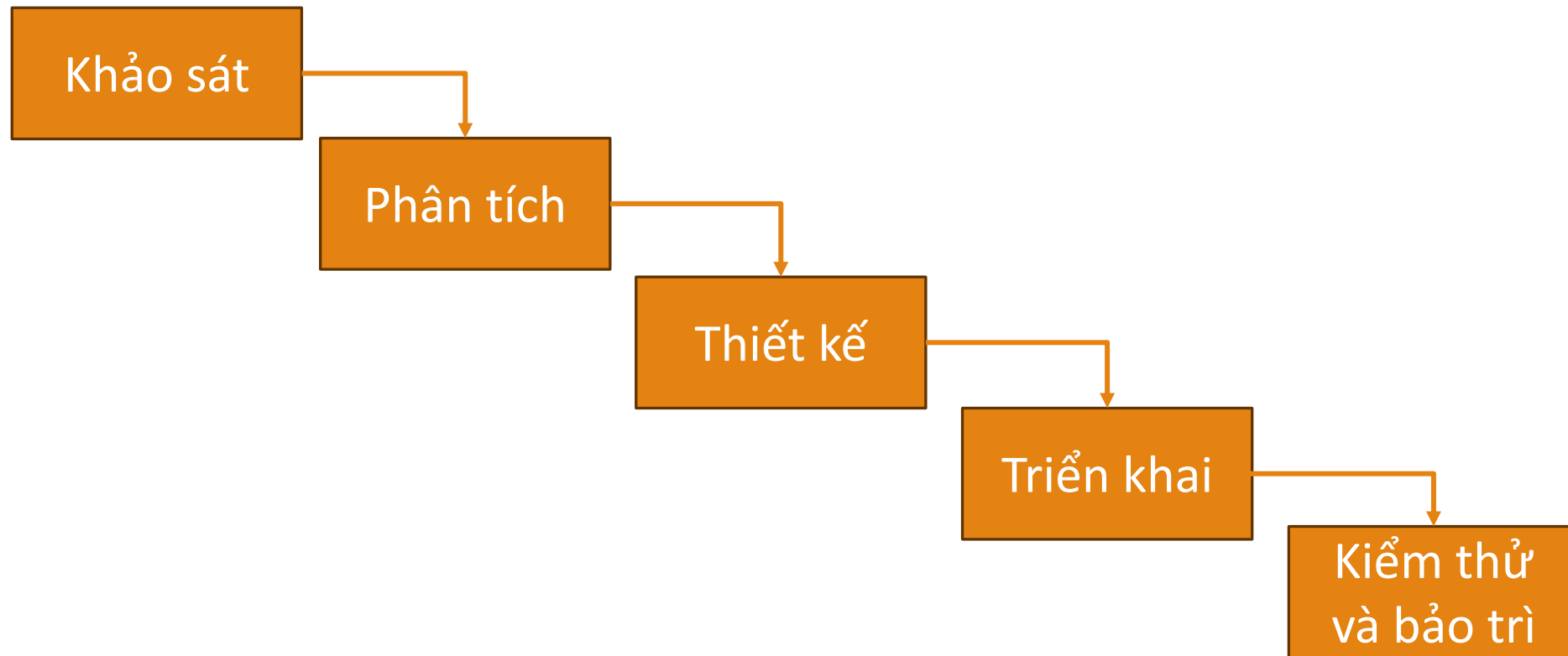
1.2.1. Các giai đoạn phát triển hệ thống thông tin

- **Thiết kế hệ thống:** Xây dựng các mô hình và kế hoạch chi tiết: thiết kế kiến trúc, thiết kế giao diện người dùng và thiết kế cơ sở dữ liệu.
- **Triển khai hệ thống:** Thực hiện cài đặt, cấu hình hệ thống.
- **Kiểm thử và bảo trì:** Đảm bảo hệ thống hoạt động đúng yêu cầu, phát hiện và sửa lỗi, cập nhật và nâng cấp hệ thống khi cần thiết.

1.2.2. Mô hình phát triển hệ thống thông tin

- Mô hình thác nước (Waterfall)
- Mô hình xoắn ốc (Spiral)
- Mô hình Agile
- Mô hình hợp nhất (Unified)

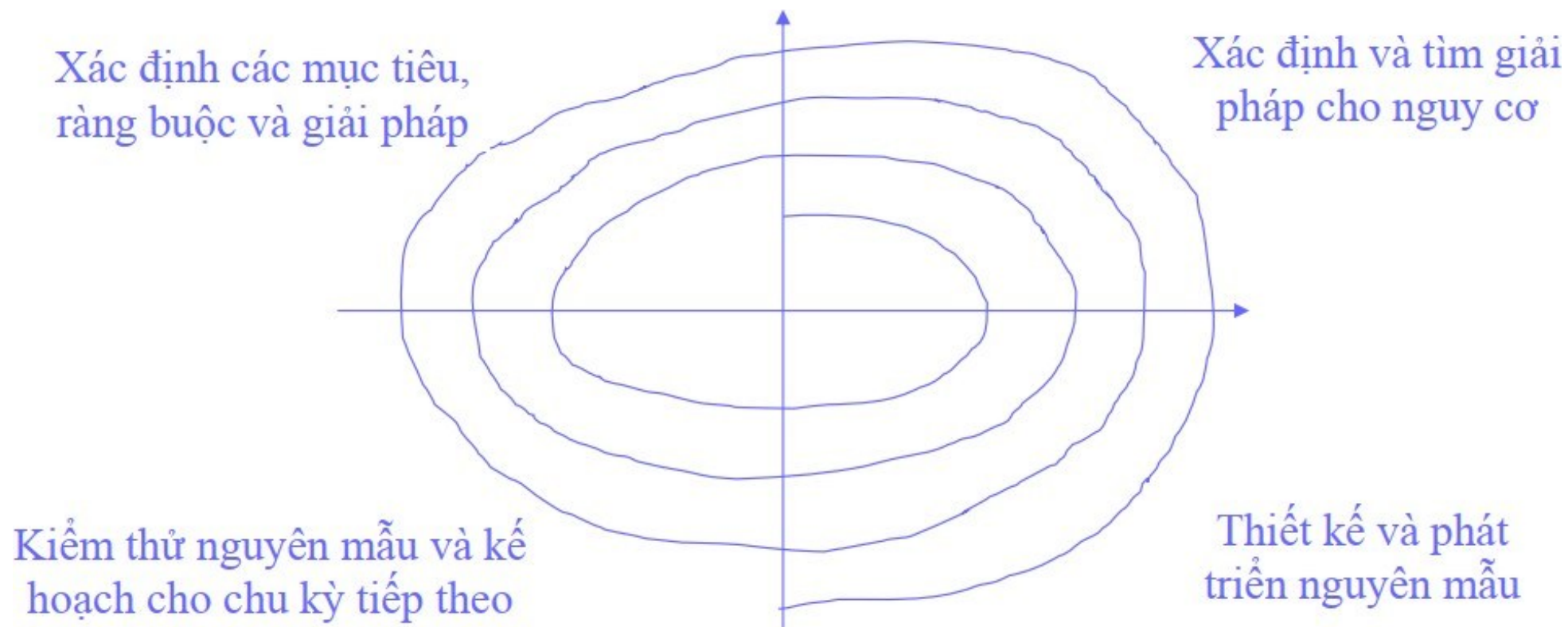
a. Mô hình thác nước



a. Mô hình thác nước

- **Đặc điểm:** Quy trình phát triển phần mềm tuần tự, bao gồm các giai đoạn: khảo sát, phân tích, thiết kế, triển khai, kiểm thử và bảo trì.
- **Ưu điểm:** Dễ quản lý, rõ ràng về tiến độ và kết quả.
- **Nhược điểm:** Thiếu linh hoạt, khó thay đổi khi yêu cầu thay đổi

b. Mô hình xoắn ốc



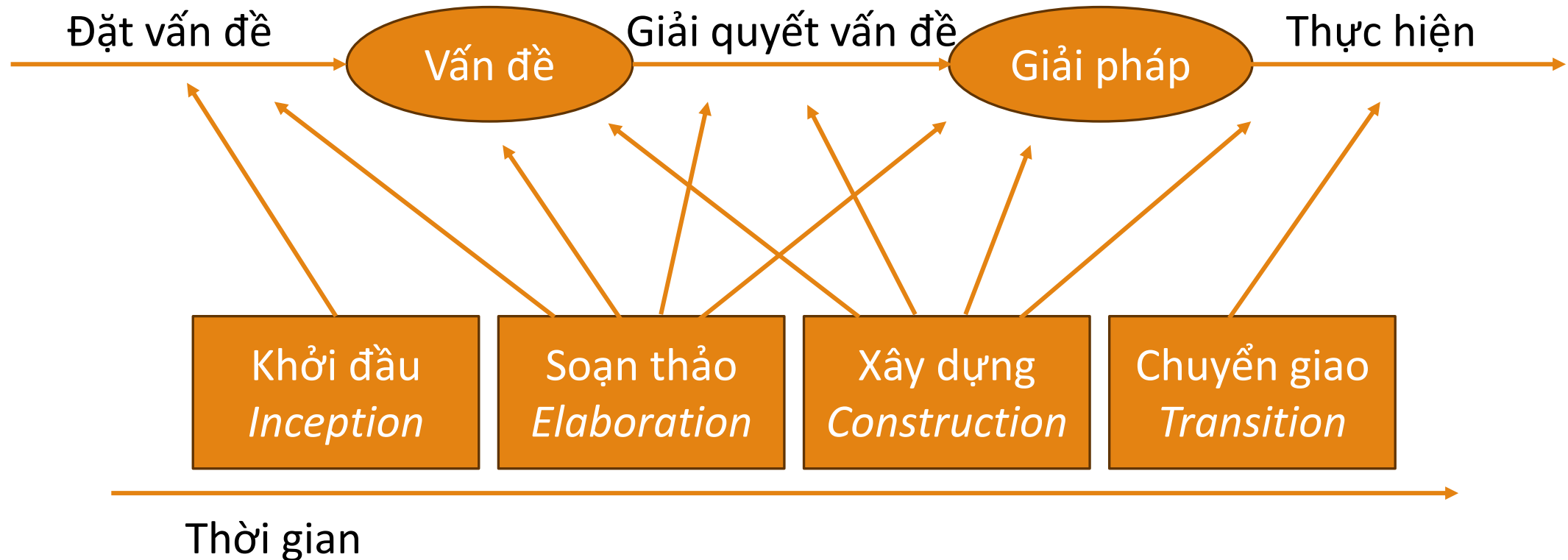
b. Mô hình xoắn ốc

- **Đặc điểm:** Quy trình phát triển phần mềm lặp đi lặp lại, tập trung vào việc đánh giá rủi ro và cải tiến liên tục.
- **Ưu điểm:** Linh hoạt, dễ thích ứng với thay đổi, giảm thiểu rủi ro.
- **Nhược điểm:** Khó quản lý tiến độ và kết quả, yêu cầu sự hợp tác chặt chẽ giữa các thành viên trong nhóm.

c. Mô hình Agile

- **Đặc điểm:** Quy trình phát triển phần mềm linh hoạt, tập trung vào việc phát triển nhanh chóng và phản hồi liên tục từ người dùng.
- **Ưu điểm:** Linh hoạt, dễ thích ứng với thay đổi, tăng cường sự tham gia của người dùng.
- **Nhược điểm:** Khó quản lý tiến độ và kết quả, yêu cầu sự hợp tác chặt chẽ giữa các thành viên trong nhóm.

d. Mô hình hợp nhất



d. Mô hình hợp nhất

- **Đặc điểm:** được mở rộng từ mô hình xoắn ốc, phát triển bởi công ty Rational, hỗ trợ phát triển hướng đối tượng
- **Ưu điểm:** Linh hoạt, dễ thích ứng với thay đổi, dễ bảo trì và mở rộng.
- **Nhược điểm:** Đòi hỏi kiến thức chuyên sâu về hướng đối tượng.

d. Mô hình hợp nhất

○ Khởi đầu

- Được thực hiện bởi các chuyên gia và nhà tin học
- Đưa ra quyết định có thực hiện hay không?

○ Soạn thảo

- Phân tích yêu cầu, xác định chức năng hệ thống, lựa chọn kiến trúc, xây dựng kế hoạch thực hiện

d. Mô hình hợp nhất

○ Xây dựng

- Phát triển phần mềm.
- Thực hiện các bước lặp: Xây dựng chi tiết; Phát triển một phần (nguyên mẫu); Kiểm thử một phần; Cài đặt một phần

○ Chuyển giao

- Đánh giá phần mềm. Chuyển giao phần mềm: Huấn luyện người dùng; Tiếp thị, phân phối và bán

1.3. Các cách tiếp cận phân tích và thiết kế

- Phương pháp hướng cấu trúc
- Phương pháp hướng đối tượng

1.3.1. Phương pháp hướng cấu trúc

- Chỉ tập trung hoặc vào dữ liệu, hoặc vào hành động (chức năng)
 - Hướng dữ liệu: phân rã phần mềm theo các chức năng cần đáp ứng và dữ liệu cho các chức năng đó
 - Hướng hành động: tập trung phân tích dựa trên hoạt động thực thi các chức năng của phần mềm đó

1.3.1. Phương pháp hướng cấu trúc

- Tiếp cận theo hướng từ trên xuống, phân rã bài toán thành các bài toán nhỏ hơn, tiếp tục phân rã các bài toán con thành các bài toán có thể cài đặt được
- Lập trình cấu trúc:

Chương trình = Thuật toán + Cấu trúc dữ liệu

1.3.1. Phương pháp hướng cấu trúc

○ Ưu điểm:

- Tư duy phân tích thiết kế rõ ràng
- Chương trình dễ hiểu

○ Nhược điểm:

- Khó tái sử dụng, không phù hợp phát triển các phần mềm lớn
- Tính mở của hệ thống thấp, chi phí sửa chữa lớn

1.3.2. Phương pháp hướng đối tượng

- Lấy đối tượng làm trung tâm
- Hệ thống = tập hợp các đối tượng + quan hệ giữa chúng
- Tập trung vào cả 2 khía cạnh của hệ thống: dữ liệu và hành động (phương thức)
 - Hệ thống được chia tương ứng thành các phần nhỏ → đối tượng
 - Mỗi đối tượng, bao gồm cả dữ liệu và hành động

1.3.2. Phương pháp hướng đối tượng

- Các đối tượng tương đối độc lập với nhau trong một hệ thống, nhưng chúng được kết hợp lại với nhau thông qua mối quan hệ và tương tác giữa chúng
- Lập trình hướng đối tượng:

Tập hợp đối tượng = chương trình

Đối tượng = thuật toán + cấu trúc dữ liệu

1.3.2. Phương pháp hướng đối tượng

○ Dựa trên các nguyên tắc cơ bản:

- **Trừu tượng hóa (abstraction):** Thực thể mô hình hóa → đối tượng; Dựa vào thuộc tính và phương thức, các đối tượng được trừu tượng hóa ở mức cao hơn → lớp; Các lớp được trừu tượng hóa ở mức cao hơn → sơ đồ các lớp (kế thừa lẫn nhau)
- **Modul hóa (modularity):** các bài toán được chia thành các vấn đề nhỏ, đơn giản và quản lý được

1.3.2. Phương pháp hướng đối tượng

○ Dựa trên các nguyên tắc cơ bản:

- **Ẩn giấu thông tin (hide):** đối tượng có thể có phương thức hoặc thuộc tính riêng (private) → cài đặt các đối tượng này độc lập với đối tượng khác, các lớp cũng độc lập với nhau
- **Phân cấp (hierarchy):** cấu trúc chung của hệ thống hướng đối tượng là dạng phân cấp theo mức độ trừu tượng từ cao đến thấp

1.3.2. Phương pháp hướng đối tượng

○ Ưu điểm:

- Phù hợp với hệ thống lớn, vì không phụ thuộc vào số biến dữ liệu hoặc số lượng thao tác mà chỉ quan tâm đến các đối tượng có trong hệ thống
- Đóng gói, che dấu thông tin làm cho hệ thống tin cậy cao; Sử dụng lại mã nguồn (kế thừa) giúp giảm chi phí, hệ thống có tính mở cao

1.4. Công cụ hỗ trợ phát triển HTTP

- **Các công cụ CASE (Computer-Aided Software Engineering):** Hỗ trợ các giai đoạn phát triển phần mềm, bao gồm phân tích, thiết kế, mã hóa và kiểm thử
- **Các ngôn ngữ lập trình:** Các ngôn ngữ lập trình phổ biến như Java, C#, Python,...
- **Các nền tảng phát triển ứng dụng:** Các nền tảng như .NET, Java EE, Spring,...

1.5. Các khái niệm về hướng đối tượng

- Đối tượng
- Lớp
- Các tính chất của hướng đối tượng

1.5.1. Đối tượng

- **Định nghĩa:** Đối tượng là một thực thể cụ thể trong thế giới thực
- **Tính chất:** Đối tượng = trạng thái + hành vi + định danh
 - Trạng thái, là các đặc tính (thuộc tính) của đối tượng
 - Hành vi, là thể hiện các hành vi (phương thức) của đối tượng
 - Định danh, là thể hiện sự tồn tại duy nhất của đối tượng

1.5.1. Đối tượng

- Trạng thái: tập hợp các thuộc tính
 - Mỗi thuộc tính mô tả một đặc tính
 - Tại một thời điểm cụ thể, các thuộc tính có giá trị xác định trong miền giá trị
- Ví dụ, một chiếc xe máy có các giá trị của thuộc tính màu đen, dung tích 125cm^3 , số km đi được 12.000 km,...

1.5.1. Đối tượng

- Hành vi: tập hợp các phương thức
 - Phương thức là một thao tác được thực hiện bởi chính nó, hoặc thực hiện khi có yêu cầu từ môi trường (thông điệp từ đối tượng khác)
 - Hành vi phụ thuộc vào trạng thái
- Ví dụ, một chiếc xe máy có các hành vi: khởi động, chạy,...

1.5.1. Đối tượng

- Giữa các đối tượng có mối liên kết với nhau



- Giữa các đối tượng có thể có giao tiếp với nhau



1.5.1. Đối tượng

○ Các loại thông điệp:

- Hàm dựng (constructor): có cùng tên với lớp, khi một đối tượng được tạo ra, hàm dựng sẽ được gọi tự động để thiết lập các giá trị ban đầu cho các thuộc tính của đối tượng đó
- Hàm hủy (destructor): được sử dụng để giải phóng tài nguyên mà đối tượng đã sử dụng, có cùng tên với lớp, được gọi tự động khi đối tượng bị hủy hoặc khi chương trình kết thúc

1.5.1. Đối tượng

○ Các loại thông điệp:

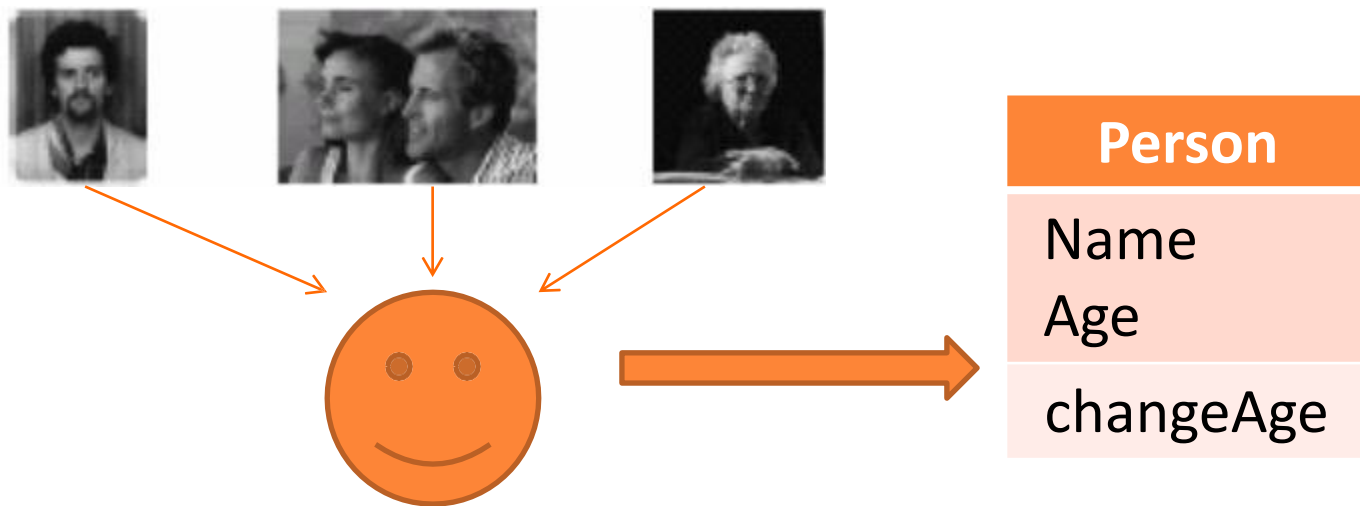
- Hàm chọn lựa (get): được sử dụng để truy cập và lấy giá trị của các thuộc tính (biến) của một đối tượng
- Hàm sửa đổi (set): được sử dụng để thay đổi giá trị của các thuộc tính (biến) của một đối tượng
- Một số hàm khác...

1.5.2. Lớp

- **Định nghĩa:** Lớp dùng để mô tả một tập hợp các đối tượng có cùng một cấu trúc, cùng hành vi và có cùng những mối quan hệ với các đối tượng khác.
- **Tính chất:** Lớp = các thuộc tính + các phương thức

1.5.2. Lớp

- Lớp là một bước trừu tượng hóa, có thể hiểu tìm kiếm các điểm giống nhau, bỏ qua các điểm khác nhau của đối tượng, làm giảm độ phức tạp



1.5.2. Lớp

- Giữa các lớp có mối quan hệ kết hợp
- Một quan hệ kết hợp là một tập hợp các mối liên kết giữa các đối tượng



1.5.3. Các tính chất của hướng đối tượng

○ Tính đóng gói (encapsulation)

- dữ liệu + xử lý dữ liệu = đối tượng
- thuộc tính + phương thức = lớp

○ Ưu điểm:

- Hạn chế ảnh hưởng khi có sự thay đổi
- Ngăn cản sự truy cập thông tin từ bên ngoài
- Che dấu thông tin

1.5.3. Các tính chất của hướng đối tượng

○ Tính kế thừa (inheritance)

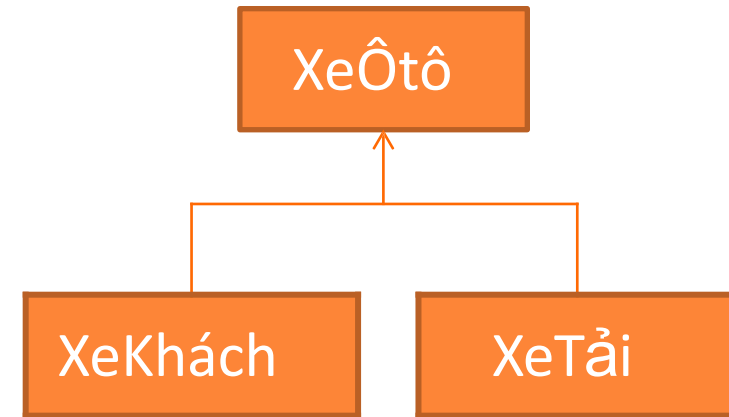
- Một lớp con kế thừa các thuộc tính và phương thức từ một hoặc nhiều lớp cha
- Tính chất tổng quát hóa: các lớp con có các tính chất chung của lớp cha
- Tính chất chuyên biệt hóa: một lớp con có các tính chất riêng của nó

1.5.3. Các tính chất của hướng đối tượng

○ **Đơn kế thừa:** một lớp con chỉ thừa kế từ một lớp cha duy nhất.

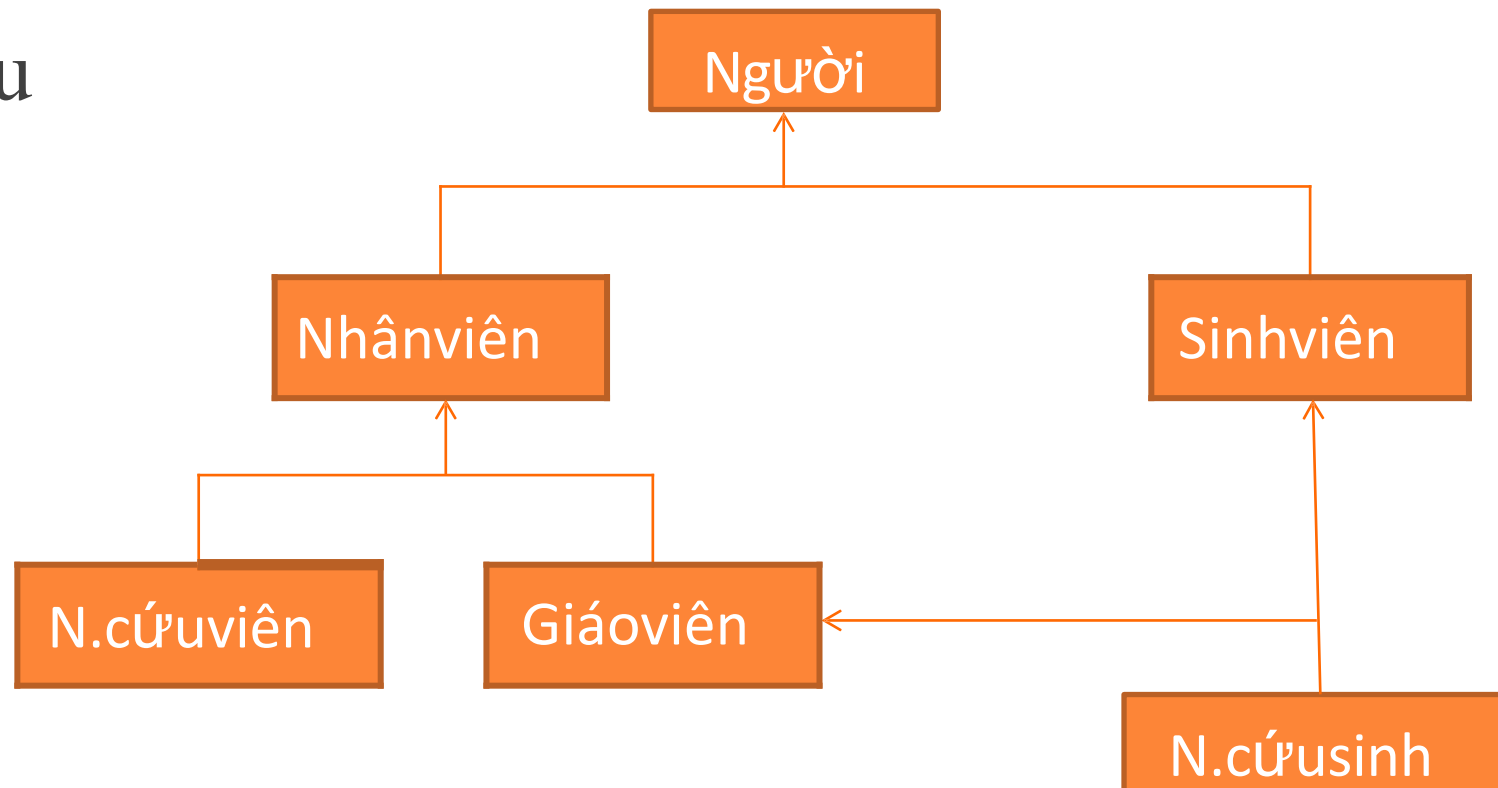
○ Ví dụ:

- XeÔtô: lớp trừu tượng (lớp cha)
- XeKhách, XeTải: lớp cụ thể (lớp con)
- Lớp cụ thể có thể thay thế cho lớp trừu tượng trong ứng dụng



1.5.3. Các tính chất của hướng đối tượng

- **Đa kế thừa:** một lớp con thừa kế từ nhiều lớp cha khác nhau



1.5.3. Các tính chất của hướng đối tượng

○ Ưu điểm

- Phân loại các lớp: các lớp được phân loại, sắp xếp theo thứ bậc để dễ quản lý
- Xây dựng các lớp: các lớp con được xây dựng từ các lớp cha
- Tiết kiệm thời gian xây dựng, tránh lặp lại thông tin

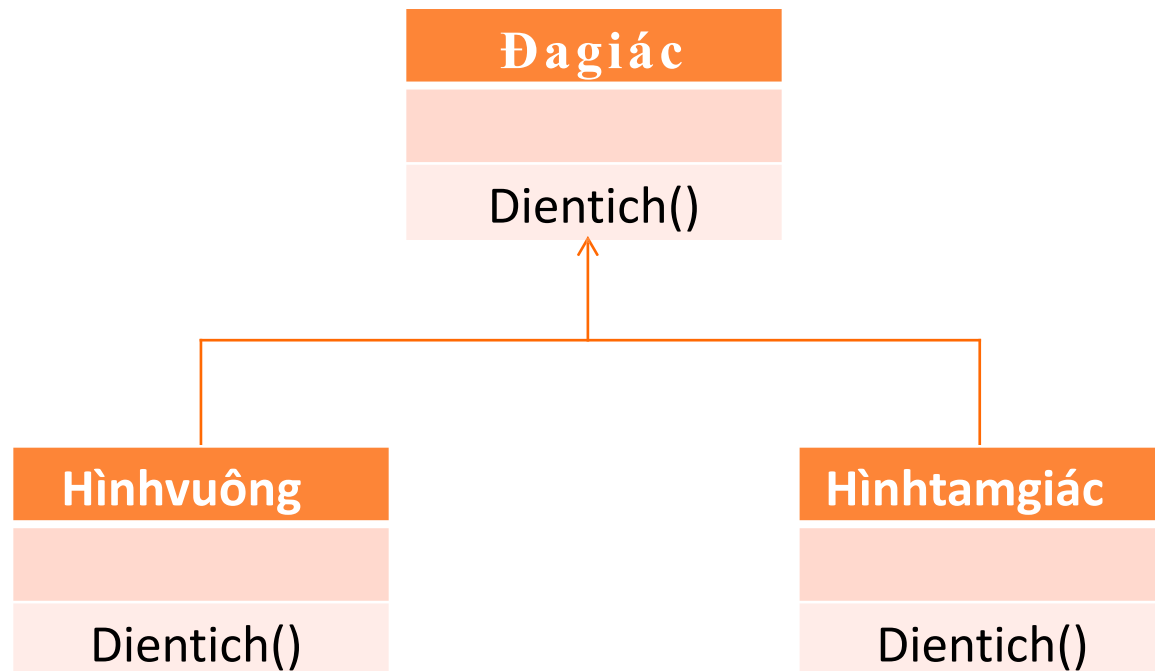
1.5.3. Các tính chất của hướng đối tượng

○ Tính đa hình (polymorphism)

- Là khả năng của các đối tượng thuộc các lớp khác nhau có thể được xử lý thông qua cùng một giao diện
- Lớp con thừa kế đặc tả các phương thức từ lớp cha và các phương thức này có thể sửa đổi trong lớp con để thực hiện các chức năng riêng trong lớp đó

1.5.3. Các tính chất của hướng đối tượng

- Một phương thức (cùng một tên phương thức) có nhiều dạng (định nghĩa) khác nhau trong các lớp khác nhau



1.6. Các bước phân tích và thiết kế hướng đối tượng

○ Pha phân tích hướng đối tượng

- Biểu đồ ca sử dụng; Mô hình khái niệm (Biểu đồ lớp)

○ Pha thiết kế hướng đối tượng

- Biểu đồ trạng thái; Biểu đồ hoạt động; Biểu đồ lớp chi tiết; Biểu đồ tương tác (Biểu đồ tuần tự, Biểu đồ cộng tác); Biểu đồ thành phần; Biểu đồ triển khai.

1.6.1. Phân tích hướng đối tượng

○ Biểu đồ ca sử dụng (use case)

- Xác định các tác nhân, các use case mô tả chức năng của hệ thống.
- Xác định mối quan hệ giữa các use case, giữa các tác nhân và mối quan hệ tương tác giữa các tác nhân và use case
- Xây dựng kịch bản mô tả hoạt động của mỗi use case cụ thể trong hệ thống, đây là phần quan trọng

1.6.1. Phân tích hướng đối tượng

○ Mô hình khái niệm_Biểu đồ lớp (class)

- Xác định tên các lớp: lớp cơ sở dữ liệu (thực thể), lớp giao diện và lớp điều khiển
- Xác định một số thuộc tính và phương thức (nếu có thể) của lớp
- Mối quan hệ cơ bản trong giữa các lớp

1.6.2. Thiết kế hướng đối tượng

○ Biểu đồ trạng thái (state)

- Mô tả các trạng thái và chuyển tiếp trạng thái trong hoạt động của một đối tượng thuộc một lớp nào đó

○ Biểu đồ hoạt động (activity)

- Mô tả hoạt động của các phương thức trong mỗi lớp hoặc các hoạt động của nhiều lớp, là cơ sở để cài đặt các phương thức

1.6.2. Thiết kế hướng đối tượng

○ Biểu đồ tương tác (interaction)

- Mô tả chi tiết hoạt động của các use case dựa trên các kịch bản (scenario) đã có và các lớp đã xác định trong pha phân tích
- Có 2 dạng biểu đồ
 - Biểu đồ tuần tự (sequence)
 - Biểu đồ cộng tác (collaboration)

1.6.2. Thiết kế hướng đối tượng

○ Biểu đồ lớp chi tiết

- Tiếp tục hoàn thiện biểu đồ lớp ở pha phân tích, bổ sung các lớp còn thiếu
- Dựa trên biểu đồ trạng thái, biểu đồ tương tác để bổ sung các thuộc tính, các phương thức và mối quan hệ giữa các lớp

1.6.2. Thiết kế hướng đối tượng

○ Biểu đồ thành phần (component)

- Xác định gói, các thành phần và tổ chức phần mềm theo các thành phần

○ Biểu đồ triển khai (deployment)

- Xác định các thành phần, các thiết bị cần thiết để triển khai hệ thống, các giao thức và dịch vụ hỗ trợ

2. Ngôn ngữ mô hình hóa UML

- Mô hình hóa
- Phương pháp mô hình hóa
- Ngôn ngữ mô hình hóa UML

2.1. Mô hình hóa

○ Khái niệm

- Mô hình (model) là sự đơn giản hóa của hệ thống thực, bằng một hình thức biểu diễn (văn bản, hình ảnh,...)
- Mô hình hóa (modeling) là quá trình dùng mô hình để biểu diễn hệ thống

2.1. Mô hình hóa

○ Ưu điểm của mô hình hóa

- Dễ dễ hiểu, dễ nhận thức vấn đề
- Là phương tiện trao đổi giữa những nhà phát triển hệ thống
- Dễ dàng nhận thấy sự phù hợp giữa mô hình và nhu cầu thực tế của hệ thống để cải tiến và hoàn thiện

2.1. Mô hình hóa

○ Các nguyên tắc mô hình hóa

- Chọn mô hình thích hợp, chẳng hạn: góc nhìn logic có biểu đồ lớp; góc nhìn người dùng có biểu đồ use case,...
- Mô hình phải có liên hệ với thế giới thực
- Một hệ thống phải được mô hình hóa bởi một tập hợp các mô hình

2.2. Phương pháp mô hình hóa

- OOD (Object Oriented Design)
- OOSE (Object Oriented Software Engineering)
- OMT (Object Modeling Technique)

2.2.1. OOD (Object Oriented Design)

- Được phát triển bởi Grady Booch
- Gồm 2 mô hình:
 - Mô hình tĩnh: Biểu đồ lớp; Biểu đồ đối tượng
 - Mô hình động: Biểu đồ trạng thái; Biểu đồ thời gian

2.2.2. OOSE (Object Oriented Software Engineering)

- Được phát triển bởi Ivar Jacobson
- Gồm 5 mô hình:
 - Mô hình yêu cầu (kịch bản sử dụng)
 - Mô hình phân tích (mức khái niệm)
 - Mô hình thiết kế (mức logic)
 - Mô hình mã hóa (mức vật lý)
 - Mô hình kiểm thử

2.2.3. OMT (Object Modeling Technique)

- Được phát triển bởi James Rumbaugh
- Gồm 3 mô hình:
 - Mô hình tĩnh: Mô hình thực thể liên kết
 - Mô hình động: Biểu đồ trạng thái và chuyển tiếp
 - Mô hình chức năng: Biểu đồ luồng dữ liệu

2.3. Ngôn ngữ mô hình hóa UML

- Giới thiệu
- Các loại biểu đồ UML
- Một số công cụ hỗ trợ

2.3.1. Giới thiệu

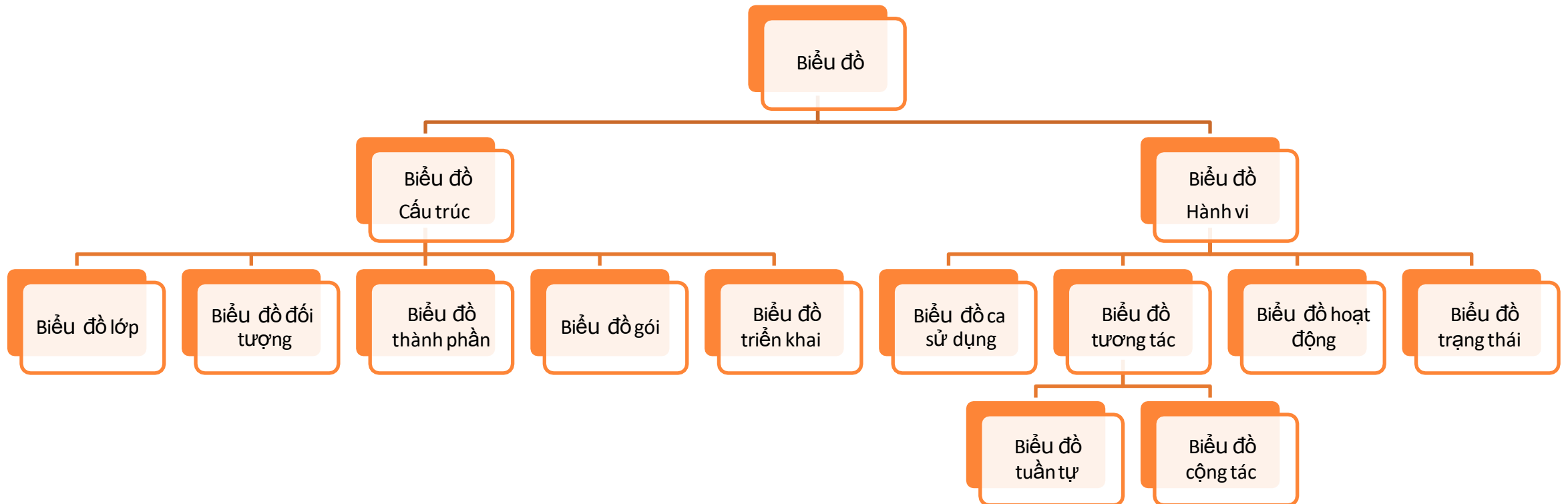
- UML (Unified Modeling Language) được phát triển bởi ba nhà khoa học Grady Booch, Ivar Jacobson và James Rumbaugh vào những năm 1990
- Trở thành một **tiêu chuẩn công nghiệp** được chấp nhận rộng rãi
- UML là một ngôn ngữ mô hình hóa chuẩn hóa được sử dụng để mô tả, đặc tả, thiết kế và tài liệu hóa các hệ thống phần mềm

2.3.1. Giới thiệu

○ Lợi ích của UML:

- Tăng cường khả năng giao tiếp vì UML là một ngôn ngữ chung
- UML cung cấp các công cụ và kỹ thuật mạnh mẽ để phân tích và thiết kế hệ thống
- UML giúp xác định các thành phần của hệ thống có thể tái sử dụng, giảm thiểu chi phí và thời gian phát triển
- UML cung cấp các mô hình chi tiết giúp kiểm thử và bảo trì hệ thống dễ dàng hơn

2.3.2. Các loại biểu đồ UML



2.3.2. Các loại biểu đồ UML

- Biểu đồ Ca sử dụng (Use case Diagram)
- Biểu đồ Lớp (Class Diagram)
- Biểu đồ Trạng thái (State Diagram)
- Biểu đồ Hoạt động (Activity Diagram)

2.3.2. Các loại biểu đồ UML

- Biểu đồ Tương tác (interaction diagram)
 - Biểu đồ Tuần tự (Sequence Diagram)
 - Biểu đồ Cộng tác (Collaboration Diagram)
- Biểu đồ Thành phần (Component Diagram)
- Biểu đồ Triển khai (Deployment Diagram)

a. Biểu đồ Ca sử dụng (Use case Diagram)

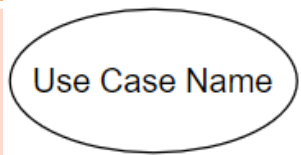
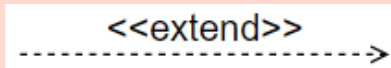
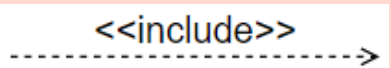
- Là tập hợp các ca sử dụng, các tác nhân và những mối quan hệ giữa chúng, dưới góc nhìn của người dùng
- Mỗi ca sử dụng là **một quá trình các thao tác** để thực hiện một chức năng của hệ thống

a. Biểu đồ Ca sử dụng (Use case Diagram)

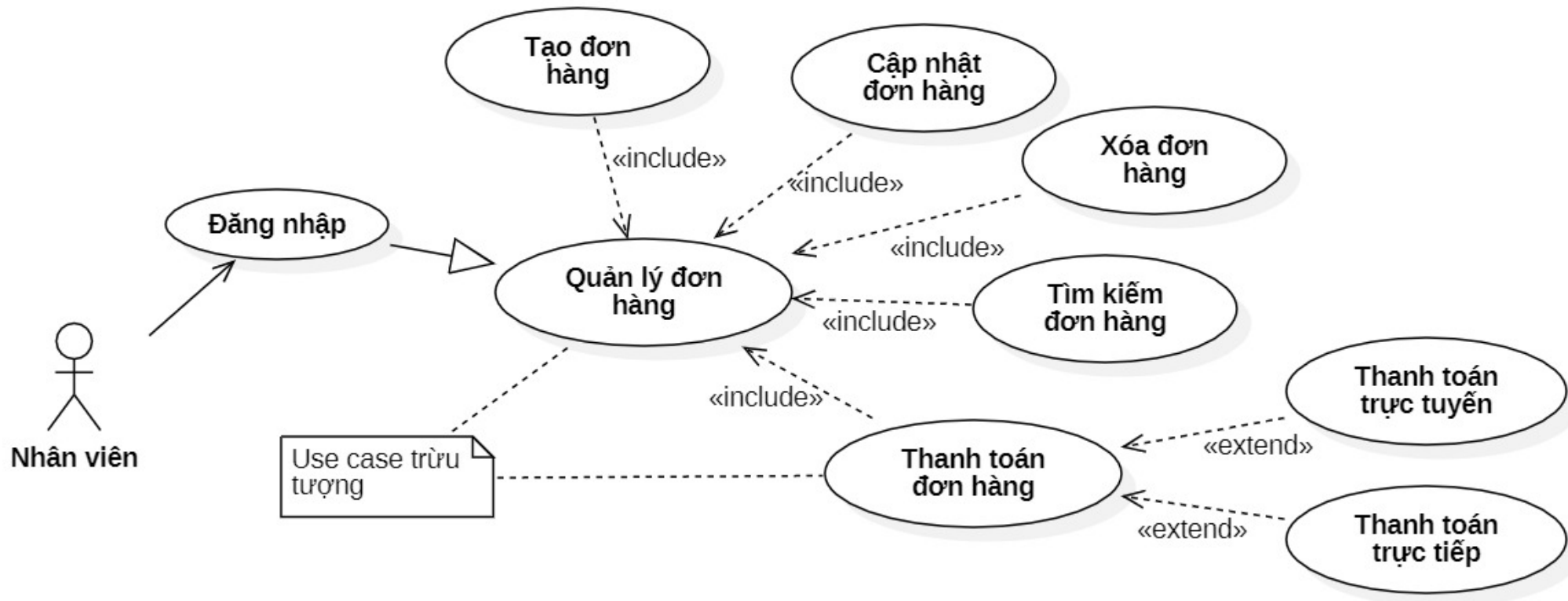
○ Các thành phần:

- Tác nhân (Actor): Người hoặc hệ thống tương tác với hệ thống đang được mô hình hóa
- Ca sử dụng (Use Case): Các chức năng hoặc dịch vụ mà hệ thống cung cấp cho tác nhân
- Mỗi quan hệ (Relationship): Mỗi quan hệ giữa các tác nhân và các ca sử dụng, bao gồm: include, extend và generalization

a. Biểu đồ Ca sử dụng (Use case Diagram)

Phần tử	Ý nghĩa	Cách biểu diễn	Ký hiệu trong biểu đồ
Ca sử dụng (Use case)	Một chức năng xác định của hệ thống	Hình elip chứa tên của use case	
Tác nhân (Actor)	Là một đối tượng bên ngoài hệ thống, tương tác trực tiếp với các use case	Hình người tượng trưng biểu diễn một lớp kiểu Actor	
Mối quan hệ giữa các use case	Tùy từng dạng quan hệ	Extend và Include mũi tên đứt nét Generalization mũi tên đầu tam giác	  

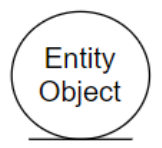
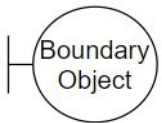
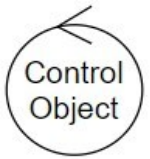
a. Biểu đồ Ca sử dụng (Use case Diagram)



b. Biểu đồ Lớp (Class Diagram)

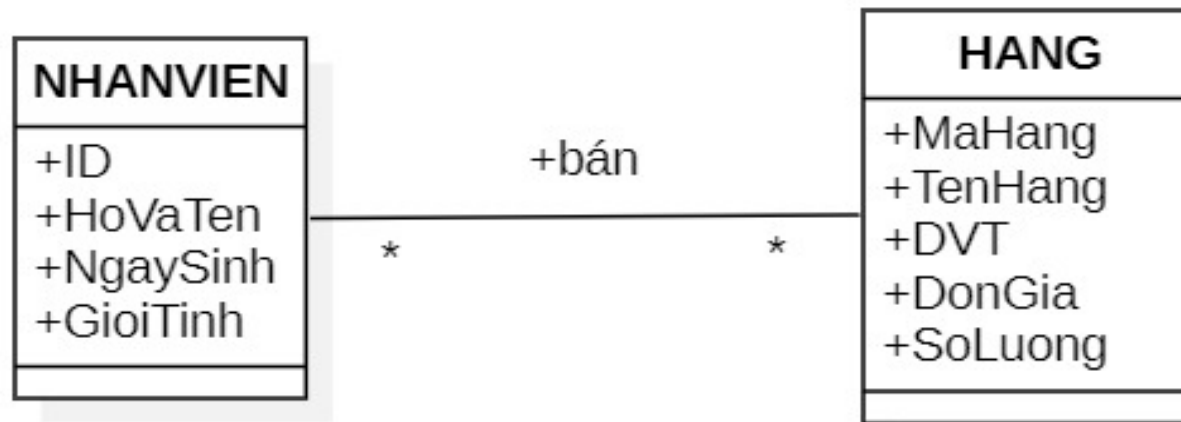
- Mô tả cấu trúc tĩnh của hệ thống thông qua các **lớp** và **mối quan hệ** giữa chúng
- Các thành phần:
 - Lớp (Class): tập hợp các đối tượng có cùng thuộc tính và phương thức
 - Thuộc tính (Attribute): các đặc điểm hoặc dữ liệu của lớp
 - Phương thức (Method): các hành động hoặc chức năng của lớp

b. Biểu đồ Lớp (Class Diagram)

Kiểu lớp	Ý nghĩa	Ký hiệu
Lớp thực thể	Đại diện cho các thực thể chứa thông tin về các đối tượng xác định nào đó	
Lớp biên (lớp giao diện)	Nằm ở ranh giới giữa hệ thống và môi trường bên ngoài, thực hiện tiếp nhận yêu cầu trực tiếp từ các tác nhân và chuyển yêu cầu vào các lớp bên trong hệ thống	
Lớp điều khiển	Thực hiện các chức năng điều khiển hoạt động của hệ thống ứng với chức năng cụ thể nào đó với một nhóm các lớp biên hoặc lớp thực thể xác định	

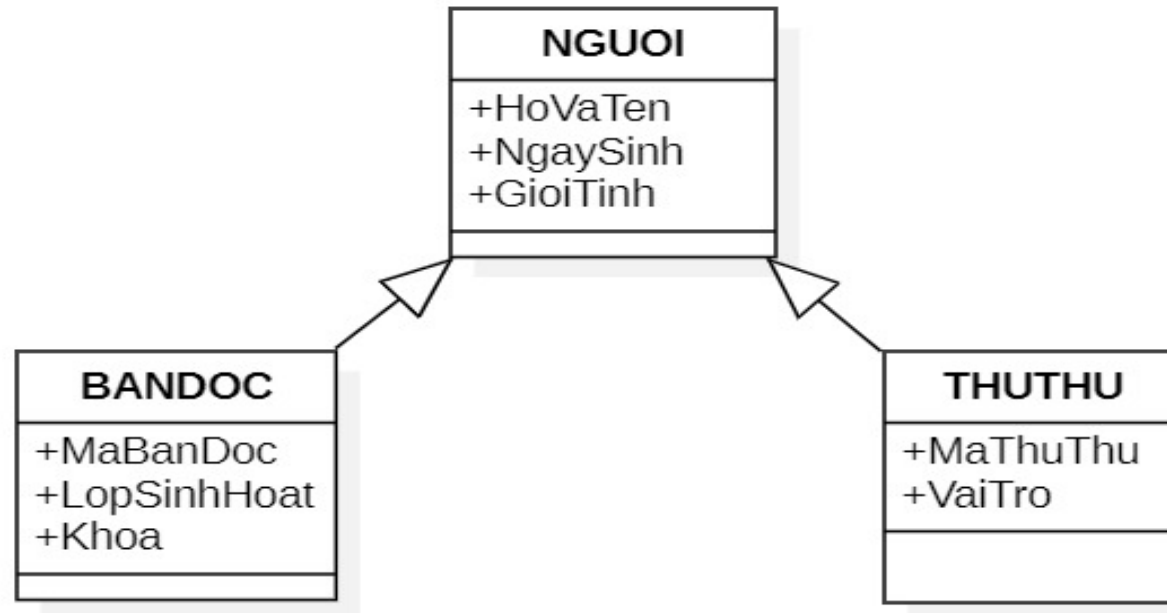
b. Biểu đồ Lớp (Class Diagram)

- Quan hệ Kết hợp (Association): Mỗi quan hệ giữa các lớp, hay **một tập hợp các liên kết** giữa các đối tượng



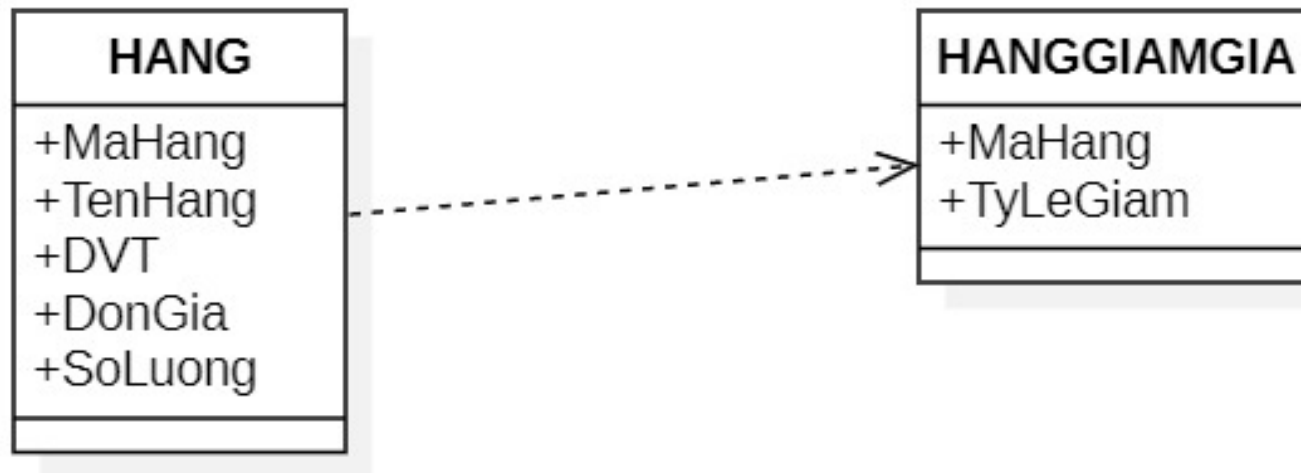
b. Biểu đồ Lớp (Class Diagram)

- Quan hệ tổng quát hóa (Generalization): quan hệ giữa lớp cha và lớp con, thể hiện **tính thừa kế**



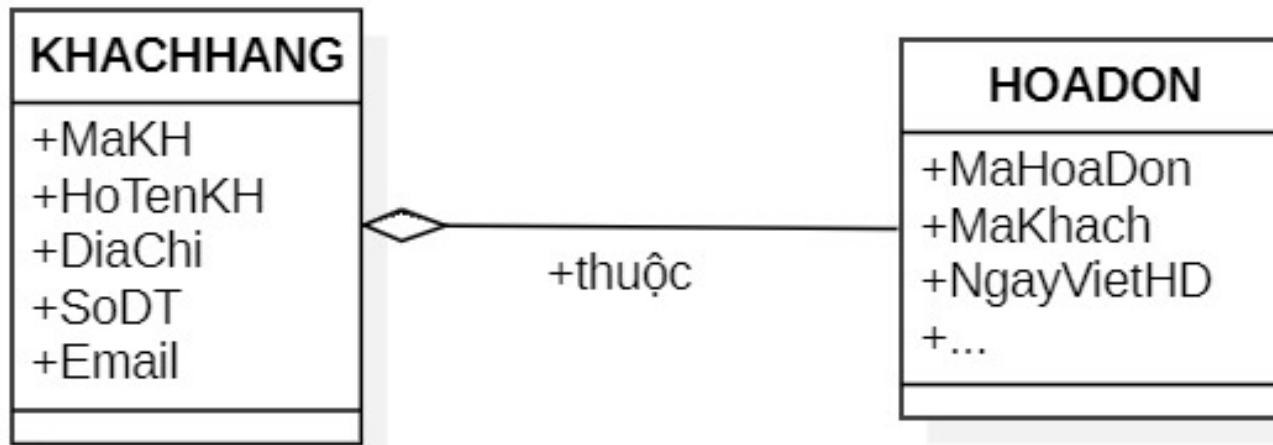
b. Biểu đồ Lớp (Class Diagram)

- Quan hệ Phụ thuộc (Dependency): sự thay đổi của lớp A sẽ ảnh hưởng đến lớp phụ thuộc B



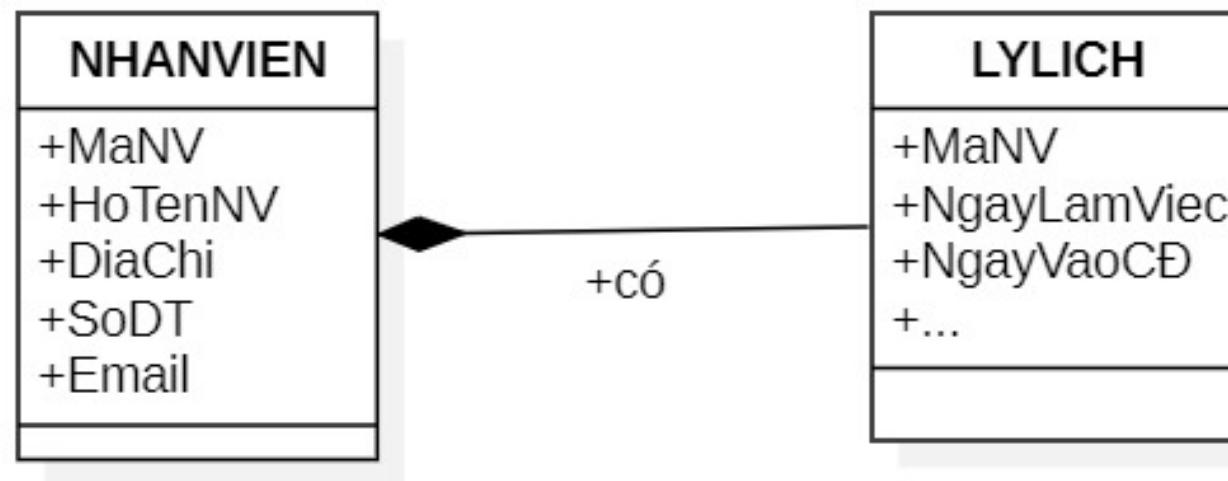
b. Biểu đồ Lớp (Class Diagram)

- Quan hệ Tổng hợp (Aggregation): quan hệ mô tả một lớp A là một phần của lớp B và lớp A có thể tồn tại độc lập



b. Biểu đồ Lớp (Class Diagram)

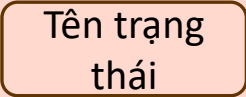
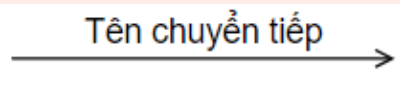
- Quan hệ Hợp thành (Composition): lớp A là một phần của lớp B và sự tồn tại của lớp B điều khiển sự tồn tại của lớp A



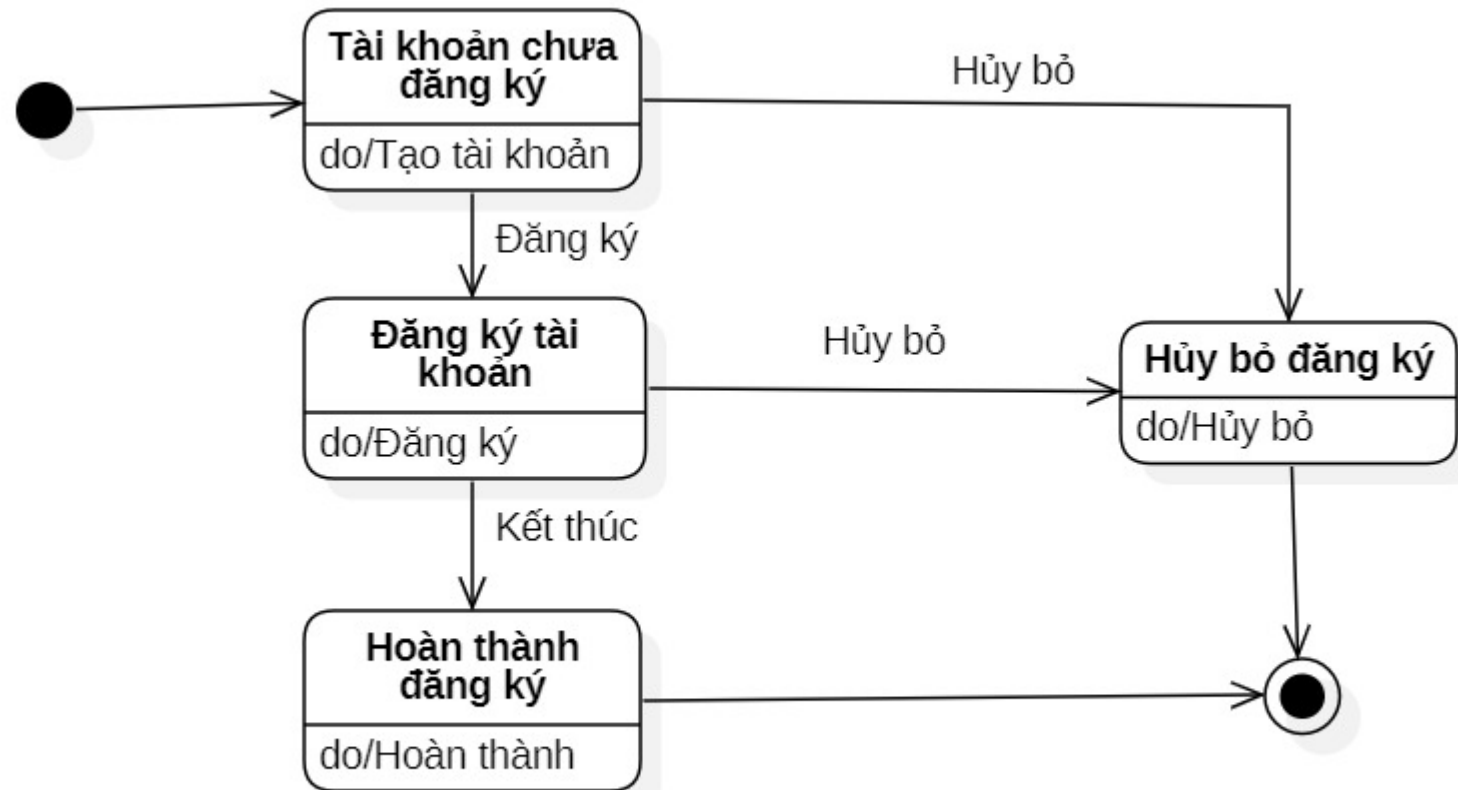
c. Biểu đồ Trạng thái (State Diagram)

- Mô tả các trạng thái khác nhau của một **đối tượng** và các sự kiện gây ra sự chuyển đổi giữa các trạng thái
- Các thành phần:
 - Trạng thái (State): Đại diện cho một tình trạng cụ thể của đối tượng tại một thời điểm
 - Sự kiện (Event): Các sự kiện gây ra sự chuyển đổi giữa các trạng thái

c. Biểu đồ Trạng thái (State Diagram)

Phần tử	Ý nghĩa	Cách biểu diễn	Ký hiệu trong biểu đồ
Trạng thái	Trạng thái của một đối tượng	Hình chữ nhật góc tròn, gồm 3 phần: tên, các biến và các hoạt động	
Trạng thái khởi đầu	Khởi đầu vòng đời của đối tượng	Hình tròn đặc	
Trạng thái kết thúc	Kết thúc vòng đời của đối tượng	Hai hình tròn	
Chuyển tiếp	Chuyển từ trạng thái này sang trạng thái khác	Mũi tên với tên chuyển tiếp bên trên	

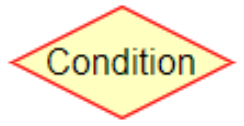
c. Biểu đồ Trạng thái (State Diagram)



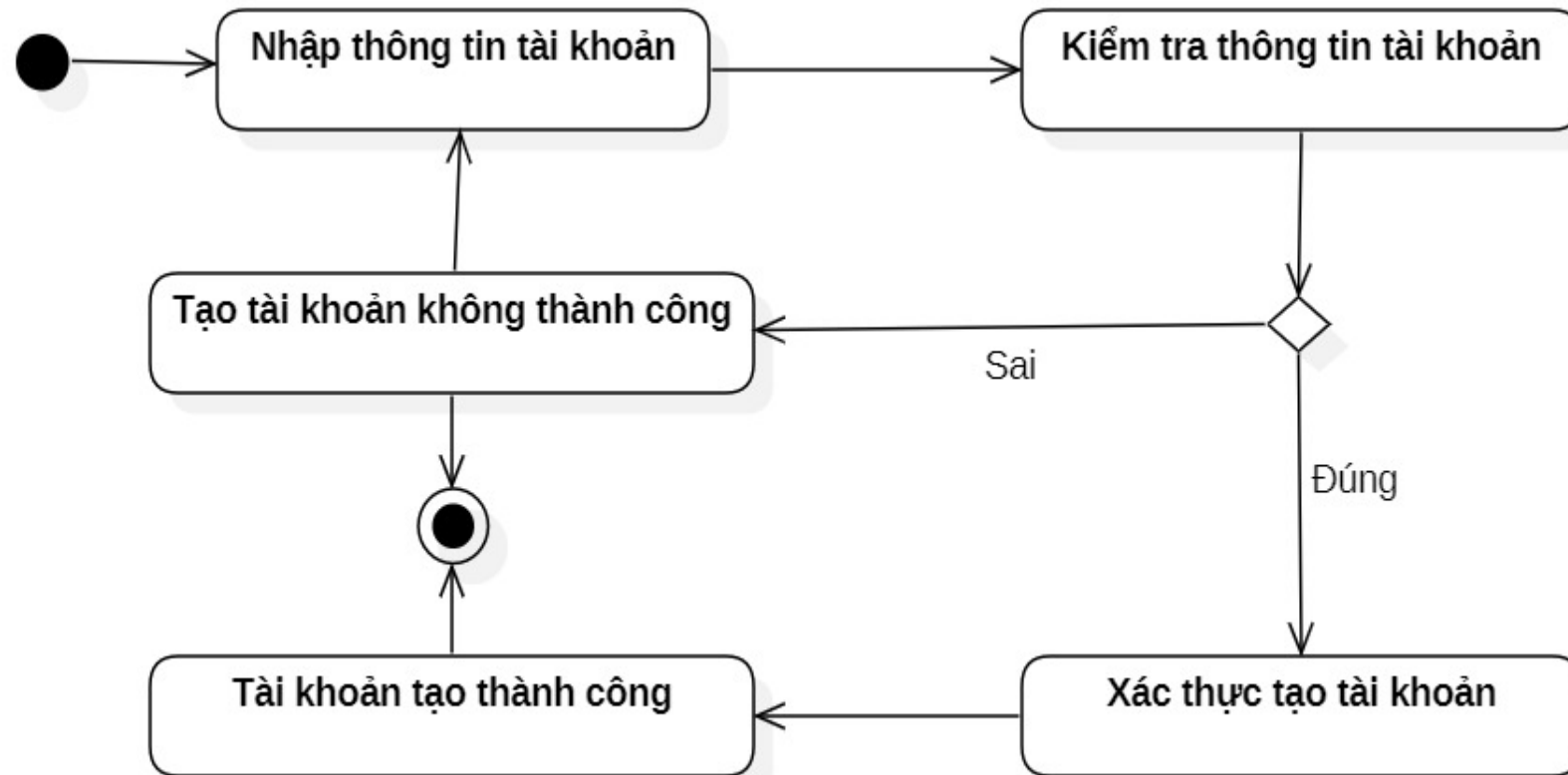
d. Biểu đồ Hoạt động (Activity Diagram)

- Mô tả luồng công việc hoặc hoạt động trong hệ thống, từ khi bắt đầu đến khi kết thúc
- Các thành phần:
 - Hoạt động (Activity): một công việc hoặc hành động cụ thể
 - Chuyển tiếp (Transition): mối quan hệ giữa các hoạt động, chỉ ra luồng công việc
 - Điều kiện (Condition): cần thỏa mãn để chuyển tiếp từ hoạt động này sang hoạt động khác

d. Biểu đồ Hoạt động (Activity Diagram)

Phần tử	Ý nghĩa	Ký hiệu
Hoạt động	Một hoạt động gồm tên và đặc tả của nó	
Trạng thái khởi đầu		
Trạng thái kết thúc		
Chuyển tiếp		
Quyết định	Mô tả lựa chọn điều kiện	
Các luồng (swimlane)	Phân tách các lớp đối tượng tồn tại khác nhau	Phân cách bởi một đường kẻ dọc từ trên xuống dưới

d. Biểu đồ Hoạt động (Activity Diagram)



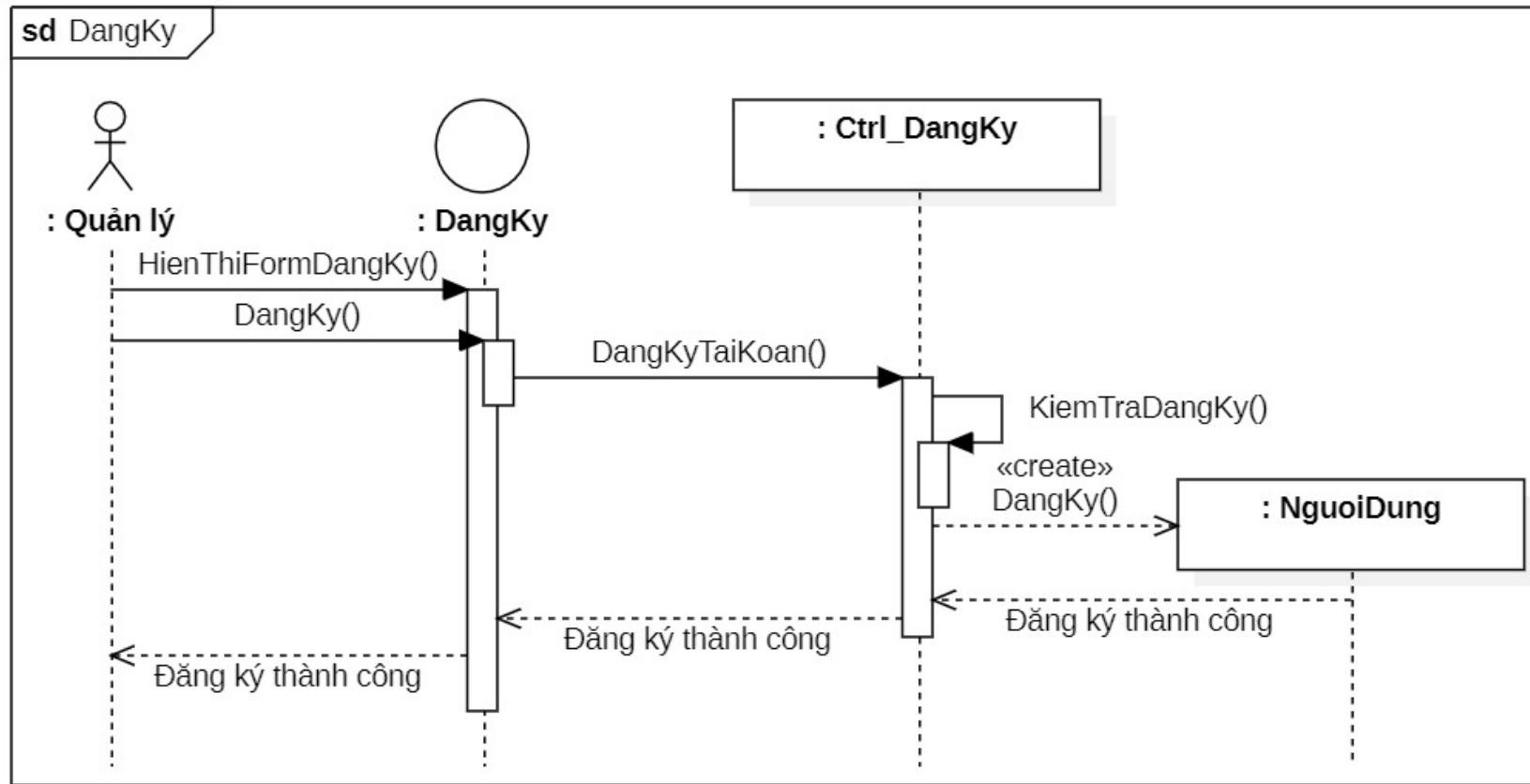
e. Biểu đồ Tuần tự (Sequence Diagram)

- Mô tả sự tương tác giữa các đối tượng theo thứ tự thời gian
- Các thành phần:
 - Đối tượng (Object): Các thực thể tham gia vào tương tác
 - Thông điệp (Message): Các thông điệp được trao đổi giữa các đối tượng
 - Dòng thời gian (Timeline): Thứ tự thời gian của các thông điệp

e. Biểu đồ Tuần tự (Sequence Diagram)

Loại thông điệp	Mô tả	Biểu diễn
Gọi (call)	Một lời gọi từ đối tượng này đến đối tượng kia	
Trả về (return)	Trả về giá trị ứng với lời gọi	
Gửi (send)	Gửi tín hiệu đến một đối tượng	
Tạo (create)	Tạo một đối tượng	
Hủy (destroy)	Hủy một đối tượng	

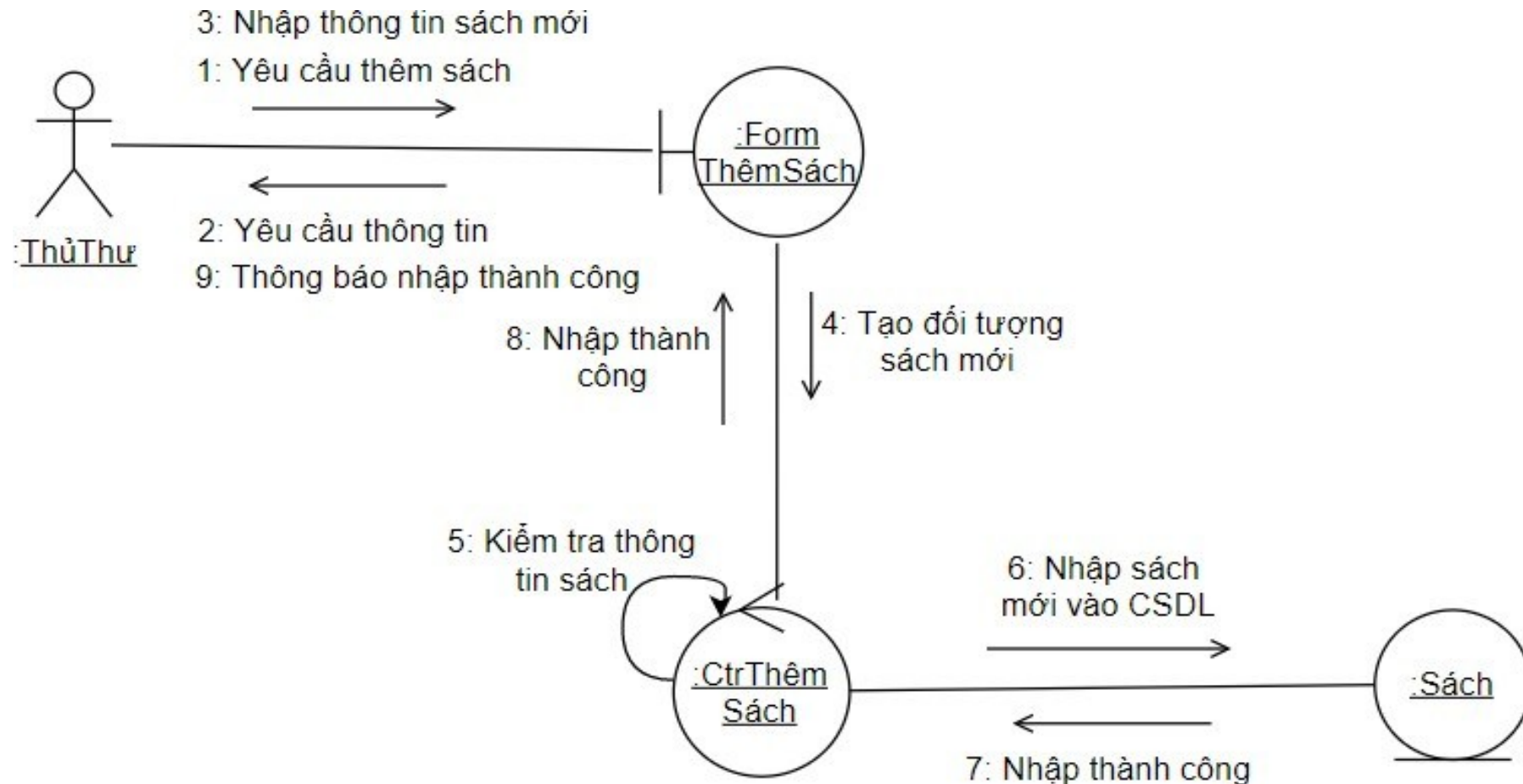
e. Biểu đồ Tuần tự (Sequence Diagram)



f. Biểu đồ Cộng tác (Collaboration Diagram)

- Mô tả sự tương tác giữa các đối tượng và mối quan hệ giữa chúng, tương tự biểu đồ tuần tự
- Các thành phần:
 - Đối tượng (Object): Các thực thể tham gia vào tương tác
 - Liên kết (Link): Mối quan hệ giữa các đối tượng
 - Thông điệp (Message): Các thông điệp được trao đổi giữa các đối tượng

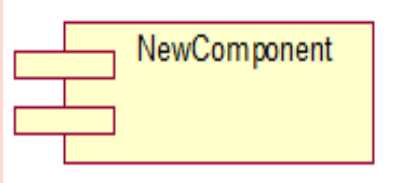
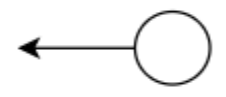
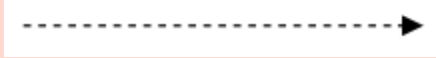
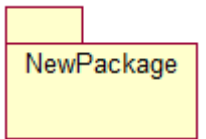
f. Biểu đồ Cộng tác (Collaboration Diagram)



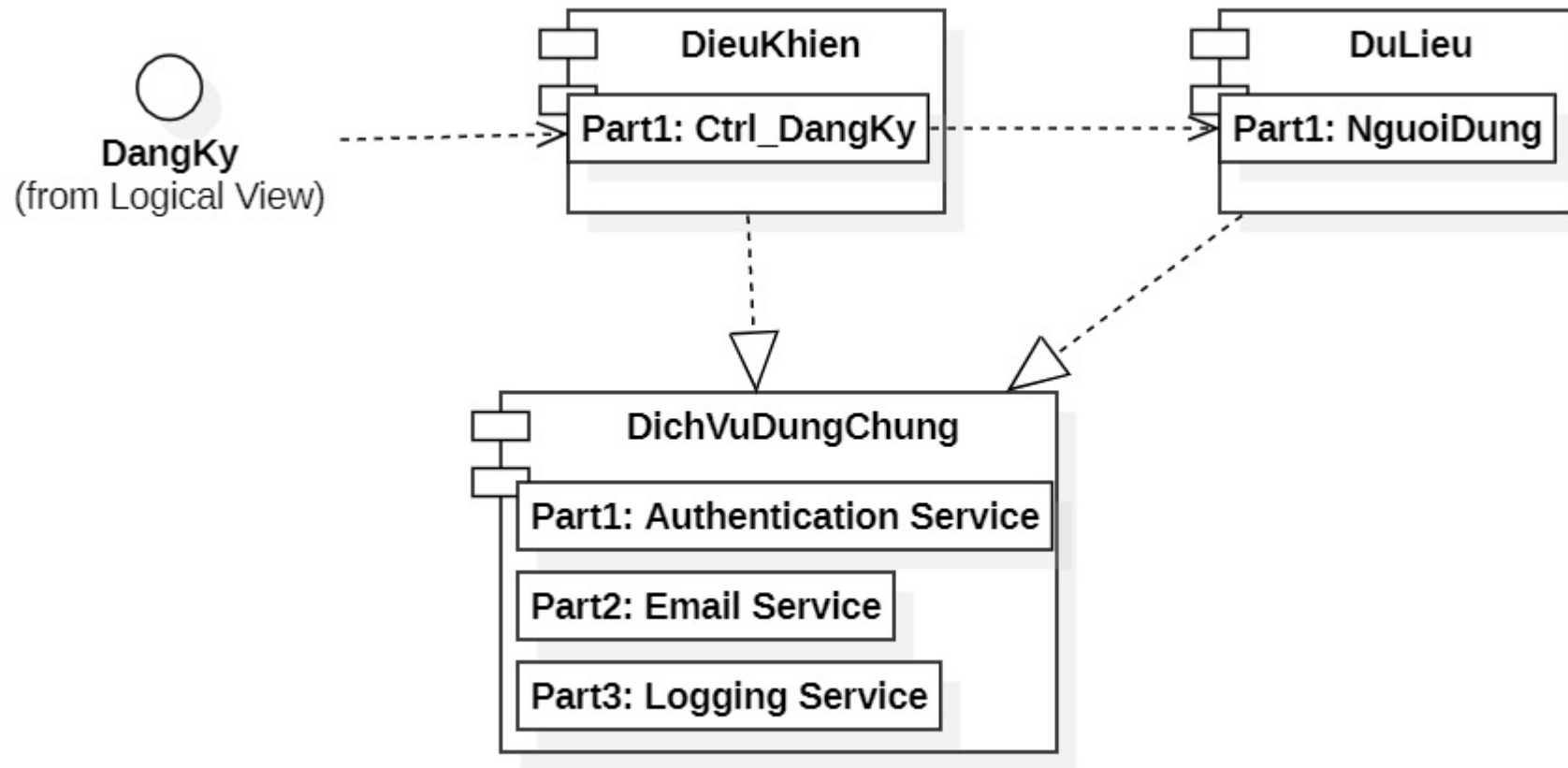
g. Biểu đồ Thành phần (Component Diagram)

- Mô tả cấu trúc vật lý của hệ thống, bao gồm các thành phần phần mềm và mối quan hệ giữa chúng.
- Các thành phần:
 - Thành phần (Component): Đại diện cho một phần của hệ thống phần mềm, có thể là một module, một thư viện hoặc một dịch vụ
 - Giao diện (Interface): Các điểm tương tác giữa các thành phần
 - Mối quan hệ (Relationship): Mối quan hệ giữa các thành phần và giao diện

g. Biểu đồ Thành phần (Component Diagram)

Phần tử	Ý nghĩa	Ký hiệu
Thành phần	Mô tả một thành phần, mỗi thành phần có thể chứa nhiều lớp, hoặc nhiều chương trình con	
Giao tiếp	Mô tả giao tiếp gắn với mỗi thành phần. Các thành phần trao đổi thông tin qua các giao tiếp	
Mối quan hệ phụ thuộc giữa các thành phần	Mối quan hệ giữa các thành phần (nếu có)	
Gói (package)	Được dùng để nhóm các thành phần lại với nhau	

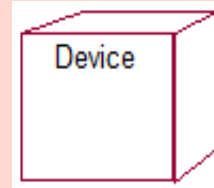
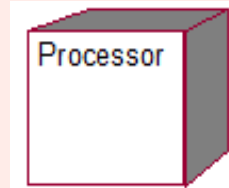
g. Biểu đồ Thành phần (Component Diagram)



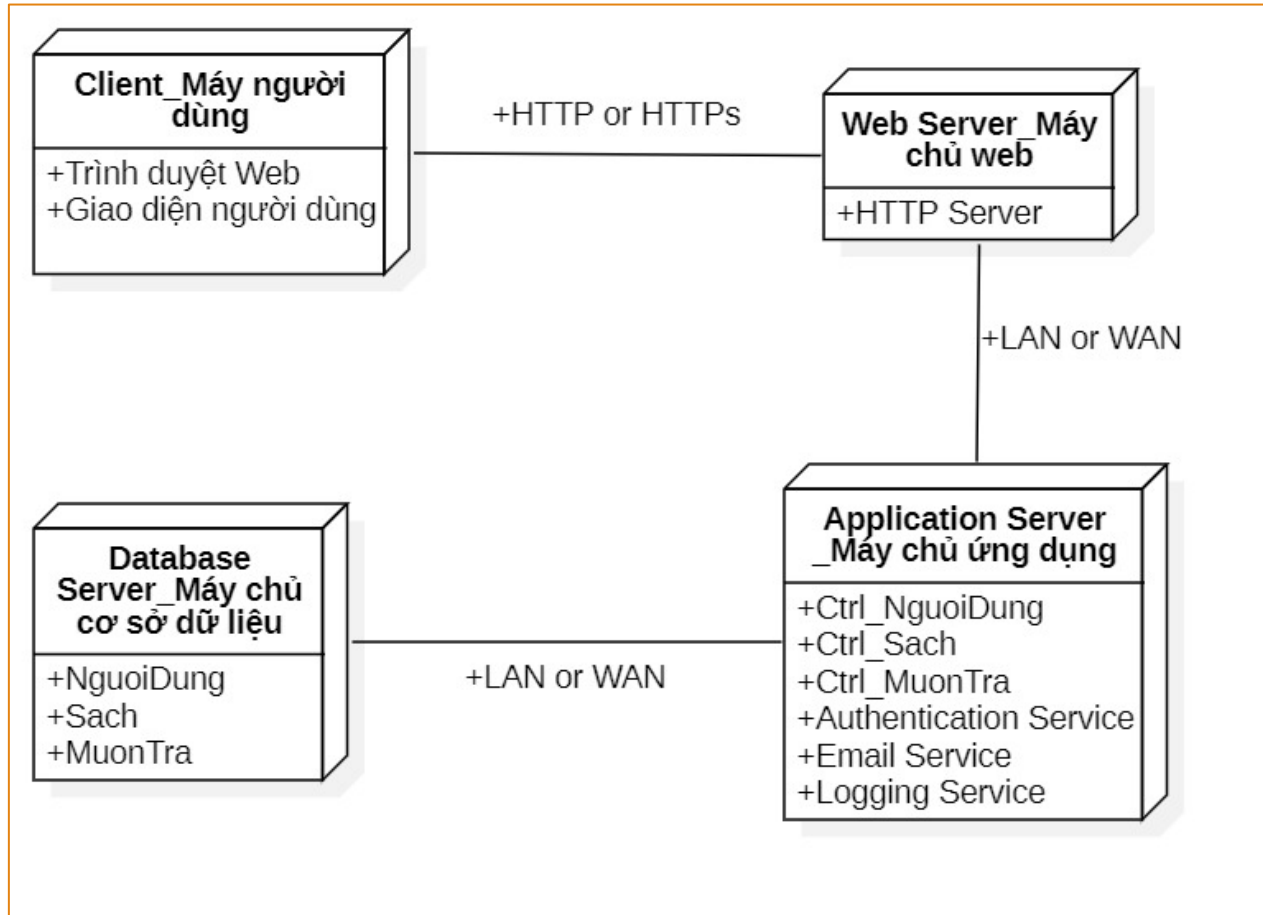
h. Biểu đồ Triển khai (Deployment Diagram)

- Mô tả cấu trúc vật lý của hệ thống, bao gồm các nút phần cứng và phần mềm được triển khai trên đó
- Các thành phần:
 - Nút (Node): Đại diện cho một phần tử vật lý trong hệ thống, có thể là một máy chủ, một thiết bị mạng hoặc một thiết bị đầu cuối
 - Thành phần (Component): Các phần mềm được triển khai trên các nút
 - Mối quan hệ (Relationship): Mối quan hệ giữa các nút và thành phần

h. Biểu đồ Triển khai (Deployment Diagram)

Phần tử	Ý nghĩa	Ký hiệu
Các nút (hay các thiết bị)	Biểu diễn các thành phần không có bộ vi xử lý	
Các thành phần	Biểu diễn các thành phần có bộ vi xử lý	
Các liên kết truyền thông	Nối các thành phần của biểu đồ, thường mô tả một giao thức truyền thông cụ thể	

h. Biểu đồ Triển khai (Deployment Diagram)



2.3. Một số công cụ hỗ trợ

- Hiện nay, có rất nhiều các công cụ hỗ trợ đặc tả, phân tích, thiết kế và tài liệu hóa hệ thống thông tin sử dụng ngôn ngữ mô hình hóa UML
 - Phần mềm: StarUML, Umbrello, UML designer tool, Altova,...
 - Trực tuyến: Draw.IO, UL Diagram, UML Diagram Tool,...
 - Thiết kế giao diện: Figma, Sketch, Zeplin, Invision,...

3. Phân tích hướng đối tượng

- Giới thiệu
- Các bước cơ bản trong phân tích hệ thống
- Biểu đồ ca sử dụng (Use case)
- Mô hình khái niệm (Biểu đồ lớp)

3.1. Giới thiệu

- Phân tích hệ thống thông tin là quá trình nghiên cứu và đánh giá hệ thống thông tin hiện tại để xác định các yêu cầu và đề xuất các giải pháp cải tiến
- Giúp xác định các vấn đề nhằm đảm bảo hệ thống mới sẽ đáp ứng được các yêu cầu của người dùng và tổ chức
- Giúp giảm thiểu rủi ro và đảm bảo tính hiệu quả của hệ thống

3.2. Các bước cơ bản trong phân tích hệ thống

- **Khảo sát hiện trạng:** Thu thập thông tin về hệ thống hiện tại, bao gồm các quy trình, dữ liệu, công nghệ và người dùng
- Phương pháp khảo sát:
 - Phỏng vấn, quan sát (có ghi chép), phân tích tài liệu, khảo sát trực tuyến,...

3.2. Các bước cơ bản trong phân tích hệ thống

- **Xác định yêu cầu:** Xác định các yêu cầu của người dùng và tổ chức đối với hệ thống mới.
- **Phân loại yêu cầu:**
 - Yêu cầu chức năng (functional requirements) và yêu cầu phi chức năng (non-functional requirements).

3.2. Các bước cơ bản trong phân tích hệ thống

- **Phân tích yêu cầu:** Phân tích và mô hình hóa các yêu cầu để hiểu rõ hơn về hệ thống cần phát triển
- **Kỹ thuật phân tích yêu cầu:**
 - Biểu đồ Use case, Mô hình khái niệm (Biểu đồ lớp phân tích)
 - Sử dụng công cụ hỗ trợ để xây dựng biểu đồ

3.3. Biểu đồ ca sử dụng (Use case)

- Ý nghĩa và vai trò
- Các bước xây dựng biểu đồ
- Ca sử dụng
- Tác nhân
- Mỗi quan hệ
- Vẽ biểu đồ ca sử dụng
- Kịch bản cho ca sử dụng

3.3.1. Ý nghĩa và vai trò

- Mô tả sự tương tác giữa người dùng (tác nhân) với hệ thống
- Xác định phạm vi của hệ thống bằng cách mô tả các chức năng chính mà hệ thống phải thực hiện và các tác nhân liên quan đến hệ thống
- Hiểu rõ các yêu cầu của người dùng đối với hệ thống. Hay, các nhà phân tích xác định được những gì người dùng mong muốn từ hệ thống

3.3.1. Ý nghĩa và vai trò

- Là phương tiện trực quan để giao tiếp giữa các bên liên quan, bao gồm các nhà phân tích, nhà phát triển, và người dùng. Giúp đảm bảo rằng tất cả các bên đều hiểu rõ các chức năng và yêu cầu của hệ thống
- Giúp xác định các mối quan hệ giữa các chức năng khác nhau của hệ thống, cũng như giữa các tác nhân và hệ thống

3.3.2. Các bước xây dựng

○ **Bước 1: Xác định Use case**

- Xác định các chức năng chính của hệ thống
- Xác định các ca sử dụng (Use case) tương ứng với các chức năng chính. Đặt tên ngắn gọn từng Use case

○ **Bước 2: Xác định tác nhân (Actor)**

- Xác định các đối tượng tương tác với hệ thống (người dùng, hệ thống khác). Phân loại các tác nhân theo vai trò và quyền hạn.

3.3.2. Các bước xây dựng

○ **Bước 3: Xác định mối quan hệ**

- Xác định mối quan hệ giữa các tác nhân và các Use case
- Xác định mối quan hệ giữa các Use case với nhau (generalization, include, extend)

○ **Bước 4: Vẽ biểu đồ Use Case**

- Sử dụng các ký hiệu và quy ước để biểu diễn các tác nhân, Use case và mối quan hệ giữa chúng

3.3.2. Các bước xây dựng

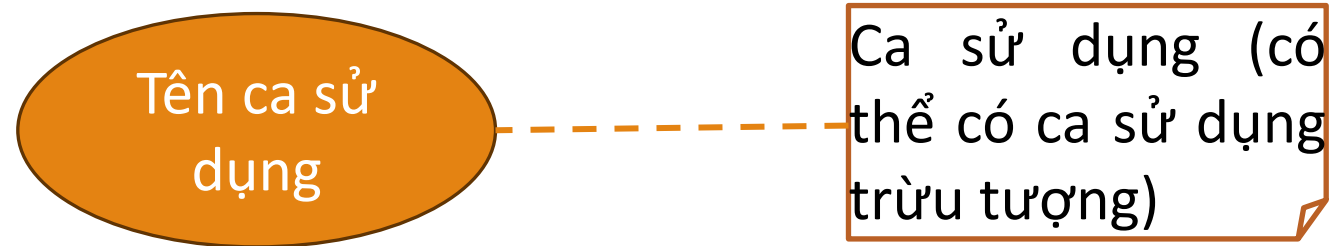
- **Xây dựng các kịch bản (scenario) cho từng ca sử dụng**
 - Mô tả các bước thực hiện của Use case.
 - Xác định các điều kiện tiên quyết và kết quả mong đợi.
 - Mô tả các tình huống chính và các tình huống ngoại lệ
 - Các tình huống ngoại lệ chia làm hai luồng: luồng tương ứng với các tác nhân và luồng tương ứng với hệ thống

3.3.3. Ca sử dụng (Use case)

- Một ca sử dụng tương ứng với một chức năng của hệ thống dưới góc nhìn của người dùng, nó có thể lớn hoặc nhỏ
- Một ca sử dụng chỉ ra làm thế nào **một mục tiêu của người dùng** được thỏa mãn bởi hệ thống
- Cần phân biệt các **mục tiêu** của người dùng với các **tương tác** của họ đối với hệ thống

3.3.3. Ca sử dụng (Use case)

○Ký hiệu



3.3.2. Ca sử dụng (Use case)

○ Ví dụ, để xây dựng ứng dụng của ngân hàng có dịch vụ **chuyển tiền** với các tương tác sau khi đăng nhập như:

- Nhập tài khoản
- Kiểm tra tài khoản
- Nhập số tiền
- Khẳng định chuyển tiền
- Xác nhận giao dịch

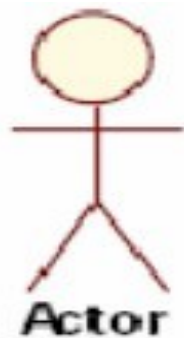
Trong các tương tác có ca sử dụng không? Tại sao?

KHÔNG? Vì mục tiêu của người dùng là “Chuyển tiền” và tập hợp các tương tác đáp ứng cho mục tiêu chuyển tiền của người dùng

3.3.4. Tác nhân (actor)

○ Tác nhân là đối tượng tương tác với hệ thống (người dùng, hệ thống khác). Chẳng hạn, hệ thống bán hàng tại siêu thị: Khách hàng, Người bán hàng, Người quản lý, Hệ thống thanh toán,...

○ Ký hiệu



3.3.4. Tác nhân (actor)

- Phân biệt tác nhân và người dùng
 - Nhiều người sử dụng có thể tương ứng với một tác nhân: *nhiều người bán hàng khác nhau nhưng có cùng vai trò như nhau đối với hệ thống*
 - Một người sử dụng nhưng có thể tương ứng với nhiều tác nhân khác nhau: *cùng một người có thể đồng thời đóng vai trò là người bán hàng và người quản lý*

3.3.5. Mối quan hệ

○ Một số ký hiệu mối quan hệ giữa các ca sử dụng

■ Include: Ca sử dụng này sử dụng lại chức năng của ca sử dụng kia

`<<include>>`
----->

■ Extend: Ca sử dụng này mở rộng từ ca sử dụng kia bằng cách thêm vào một chức năng cụ thể

`<<extend>>`
----->

■ Generalization: Ca sử dụng này được kế thừa các chức năng từ ca sử dụng kia

—————>

3.3.6. Vẽ biểu đồ ca sử dụng

- Sử dụng công cụ (phần mềm trực tiếp, trực tuyến) để biểu diễn tác nhân, use case và mối quan hệ giữa chúng bằng cách sử dụng các ký hiệu và quy ước

3.3.7. Kịch bản cho ca sử dụng

- Ví dụ, xây dựng kịch bản cho Use case Đăng nhập
- Các tác nhân:
 - Người dùng hệ thống: thực hiện thao tác đăng nhập vào hệ thống
- Điều kiện trước:
 - Người dùng đã có tài khoản và thông tin đăng nhập hợp lệ
 - Hệ thống đang hoạt động và sẵn sàng tiếp nhận yêu cầu đăng nhập

3.3.7. Kịch bản cho ca sử dụng

○Điều kiện sau:

- Người dùng đã đăng nhập thành công vào hệ thống
- Hệ thống xác nhận và ghi nhận thông tin đăng nhập của người dùng

○Mô tả tóm tắt:

- Người dùng có tài khoản đăng nhập hệ thống bằng cách vào giao diện Đăng nhập, điền đầy đủ thông tin cần thiết và sau đó kích chọn nút lệnh đăng nhập.

3.3.7. Kịch bản cho ca sử dụng

○ Các sự kiện chính

Hành động của tác nhân	Hành động của hệ thống
1. Truy cập vào giao diện đăng nhập của hệ thống	2. Chuyển người dùng đến giao diện đăng nhập
3. Nhập tên đăng nhập và mật khẩu.	
4. Kích chọn nút Đăng nhập	5. Kiểm tra tính hợp lệ của tên đăng nhập và mật khẩu. Chuyển người dùng đến giao diện chính
	6. Ghi nhận thông tin và cập nhật trạng thái của người dùng trong hệ thống

3.3.7. Kịch bản cho ca sử dụng

○ Các sự kiện ngoại lệ

Hành động của tác nhân	Hành động của hệ thống
	5. Kiểm tra tên đăng nhập và mật khẩu là không hợp lệ. Chuyển người dùng quay lại giao diện đăng nhập để người dùng nhập lại.
3. Người dùng nhập lại tên đăng nhập và mật khẩu hoặc hủy bỏ việc đăng nhập.	

3.3.8. Ví dụ minh họa

- Xây dựng mô hình ca sử dụng hệ thống quản lý bán hàng (cửa hàng, siêu thị,...).
 - Xác định các tác nhân
 - Xác định các ca sử dụng
 - Xác định mối quan hệ giữa các ca sử dụng
 - Vẽ mô hình ca sử dụng
 - Xây dựng kịch bản cho từng ca sử dụng

a. Xác định các tác nhân

- Quản trị viên: Người quản lý toàn bộ hệ thống, có quyền cao nhất.
- Nhân viên bán hàng: Người thực hiện các giao dịch bán hàng và quản lý sản phẩm.
- Khách hàng: Người mua hàng từ hệ thống.

b. Xác định các ca sử dụng

○ Quản trị viên

- Quản lý tài khoản người dùng: Tạo tài khoản; Cập nhật tài khoản; Xóa tài khoản

○ Nhân viên

- Đăng nhập
- Quản lý sản phẩm: Thêm sản phẩm mới; Cập nhật thông tin sản phẩm; Xóa sản phẩm; Xem danh sách sản phẩm

b. Xác định các ca sử dụng

○ Nhân viên

- Quản lý đơn hàng: Tạo đơn hàng; Cập nhật đơn hàng; Xóa đơn hàng; Tìm kiếm đơn hàng; Thanh toán đơn hàng;...
- Thanh toán đơn hàng: Thanh toán trực tiếp; Thanh toán trực tuyến
- Quản lý khách hàng: ...
- Quản lý khuyến mãi: ...
- ...

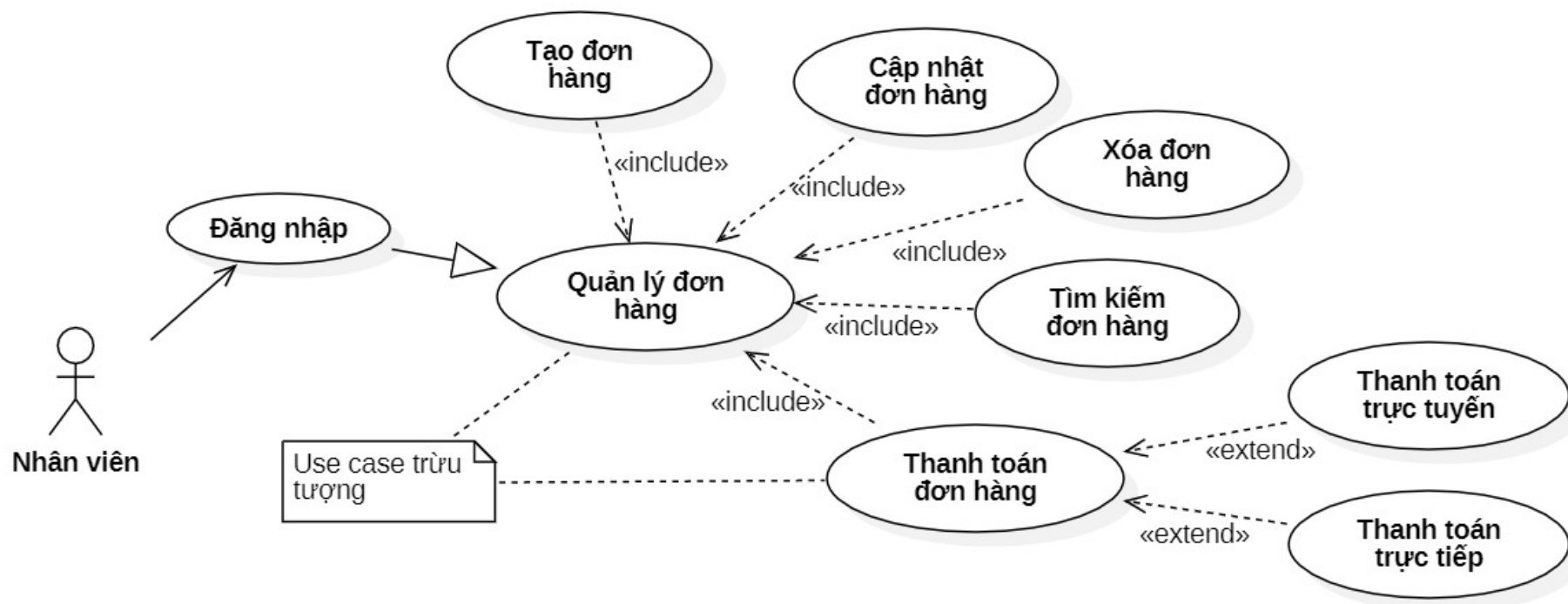
c. Xác định mối quan hệ

○ Nhân viên

- Use case Đăng nhập có mối quan hệ generalization với các use case trườ tượng Quản lý đơn hàng; Quản lý khách hàng;...
- Use case Quản lý đơn hàng có mối quan hệ include với các use case Tạo đơn hàng; Cập nhật đơn hàng; Xóa đơn hàng; Tìm kiếm đơn hàng; Thanh toán đơn hàng;...
- Use case Thanh toán đơn hàng có mối quan hệ extend với các use case Thanh toán trực tiếp; Thanh toán trực tuyến

d. Mô hình ca sử dụng

○ Một phần mô hình ca sử dụng



3.4. Mô hình khái niệm

- Giới thiệu
- Ý nghĩa và vai trò
- Khái niệm
- Thuộc tính
- Phương thức
- Mối quan hệ
- Các bước xây dựng mô hình khái niệm
- Ví dụ minh họa

3.4.1. Giới thiệu

- Mô hình khái niệm (conceptual model) là mô tả các khái niệm trong quan hệ.
- UML không cung cấp mô hình khái niệm, nhưng cung cấp ký hiệu và cú pháp để biểu diễn mô hình này chính là **biểu đồ lớp**
- Mô hình khái niệm được gọi là biểu đồ lớp phân tích (analysis class diagram)

3.4.1. Giới thiệu

- Mô hình khái niệm bao gồm:
 - *Các khái niệm* là các kiểu thực thể thuộc lĩnh vực nghiên cứu
 - *Các thuộc tính và phương thức* của từng khái niệm
 - *Mối quan hệ* giữa các khái niệm
- Mô hình khái niệm sẽ được triển khai dần thành sang biểu đồ lớp thiết kế (design class diagram)

3.4.2. Ý nghĩa và vai trò

- Mô tả cấu trúc tĩnh của hệ thống, bao gồm các lớp, thuộc tính, phương thức, và mối quan hệ giữa các lớp
- Là một phương tiện giao tiếp trực quan giữa các nhà phân tích, nhà phát triển, lập trình viên,... Nó giúp truyền đạt thông tin về cấu trúc dữ liệu, cấu trúc điều khiển và cấu trúc giao diện của hệ thống một cách dễ hiểu và hiệu quả

3.4.3. Khái niệm

- Một khái niệm là một biểu diễn ở mức cao (mức trừu tượng) về một sự vật hoặc một đối tượng
- Để xác định khái niệm, cần xác định khái niệm từ khách hàng (nhà đầu tư dự án).
- Làm sao biết được một khái niệm là đúng hay sai? Cần dựa trên nguyên tắc: *“Nếu khách hàng không hiểu khái niệm, rất có thể đó không phải là khái niệm”*

3.4.3. Khái niệm

- Một khái niệm thường được gắn liền với **vấn đề**.
- Ví dụ, một số khái niệm đúng trong hệ thống quản lý bán hàng
 - **Nhân viên** bán hàng trong hệ thống (lớp thực thể hay lớp dữ liệu)
 - **Giao diện đăng nhập** của hệ thống (lớp biên hay lớp giao diện)
 - Chức năng **Đăng nhập** vào hệ thống (lớp điều khiển)

3.4.3. Khái niệm

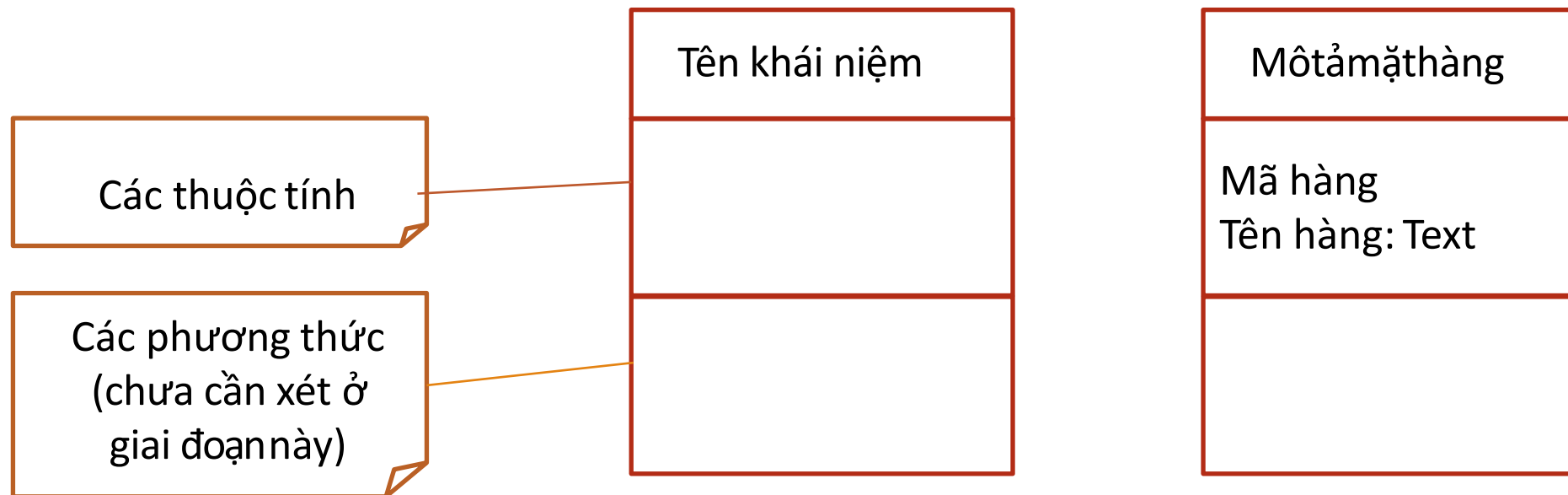
- Ngoài ra, để xác định khái niệm có thể dựa vào kịch bản đặc tả yêu cầu của các ca sử dụng (use case). Khái niệm thường là danh từ hoặc cụm danh từ
- Ví dụ, từ ca sử dụng “*Tạo đơn hàng*”, có thể xác định các khái niệm: Nhân viên, khách hàng, Đơn hàng, Mặt hàng,...

3.4.3. Khái niệm

- **MỘT SỐ GỢI Ý ĐỂ XÁC ĐỊNH KHÁI NIỆM**
- Các đối tượng vật lý (xe ô tô)
- Các vị trí, địa điểm (nhà ga, bến xe)
- Các giao tác (thanh toán)
- Các vai trò của con người (người bán hàng)
- Các hệ thống khác ở bên ngoài
- Các danh từ trừu tượng (thực đơn)
- Các tổ chức (đại học, cửa hàng)
- Các sự kiện (cấp cứu)
- Các nguyên tắc/chính sách (quy định nộp phạt vi phạm giao thông)
- ...

3.4.3. Khái niệm

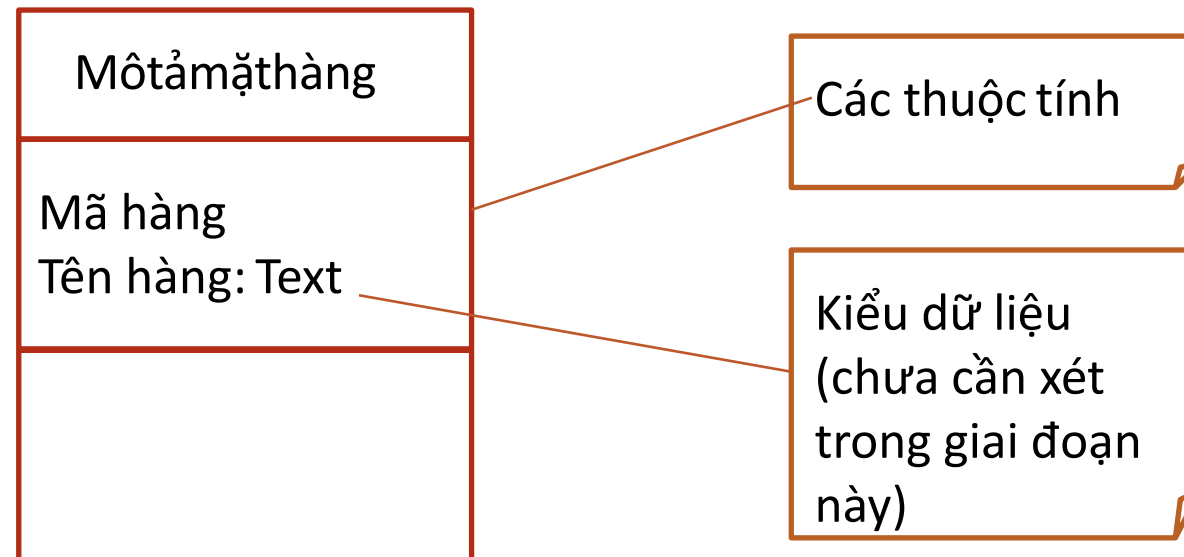
- Biểu diễn **khái niệm** bằng cách sử dụng ký hiệu **lớp** trong biểu đồ lớp. Ví dụ, khái niệm *Mô tả mặt hàng*



3.4.4. Thuộc tính

○ Thuộc tính (attribute) là đặc trưng của một khái niệm dùng biểu diễn dữ liệu cần thiết cho một thể hiện (instance)

○ Ví dụ:



3.4.5. Phương thức

- Phương thức (method) là khả năng thực hiện của một thể hiện của khái niệm. Trong giai đoạn này không cần thiết mô tả phương thức

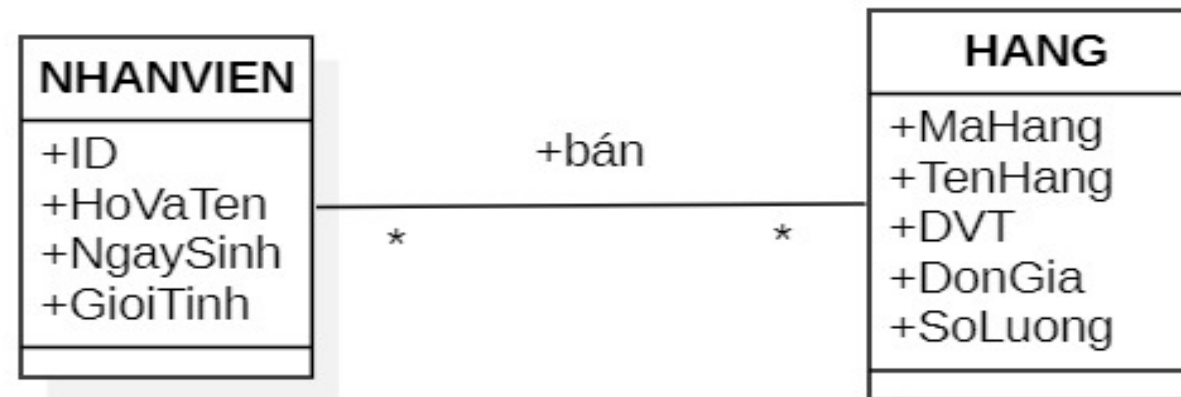
Mô tả mặt hàng
Mã hàng Tên hàng

3.4.6. Mỗi quan hệ

- Mỗi quan hệ kết hợp
- Mỗi quan hệ tổng quát hóa
- Quan hệ phụ thuộc
- Mỗi quan hệ tổng hợp và hợp thành

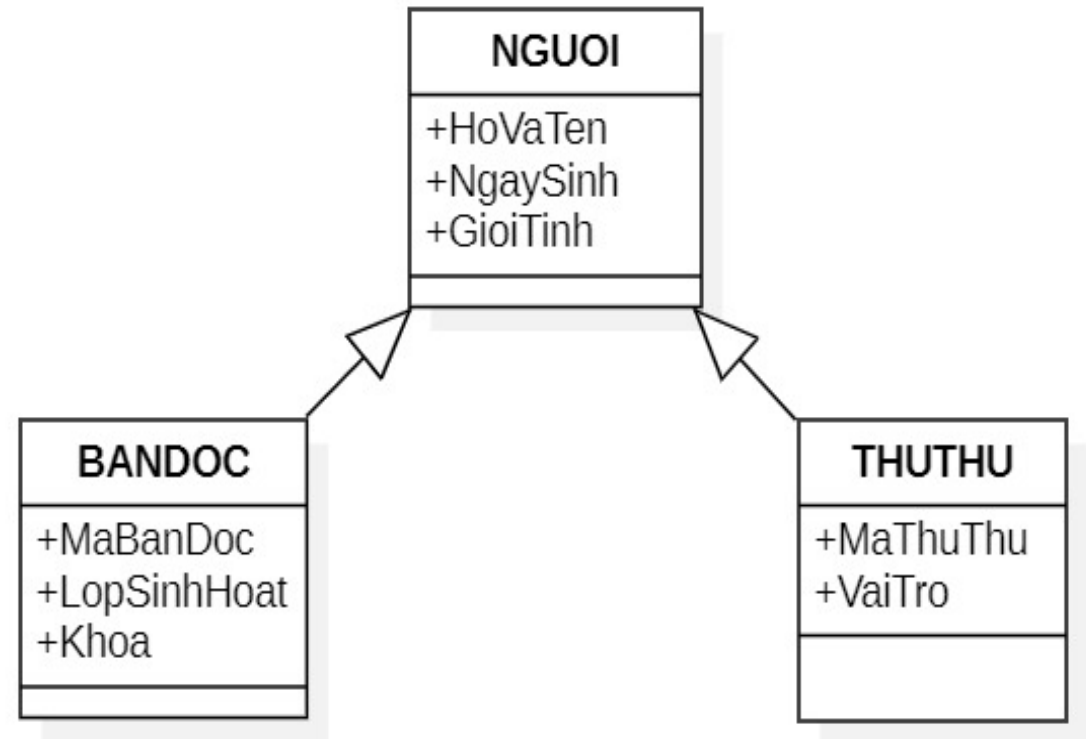
3.4.6. Mỗi quan hệ

- Quan hệ Kết hợp (Association): Mỗi quan hệ giữa các lớp, hay **một tập hợp các liên kết** giữa các đối tượng. Được biểu diễn bằng đường thẳng



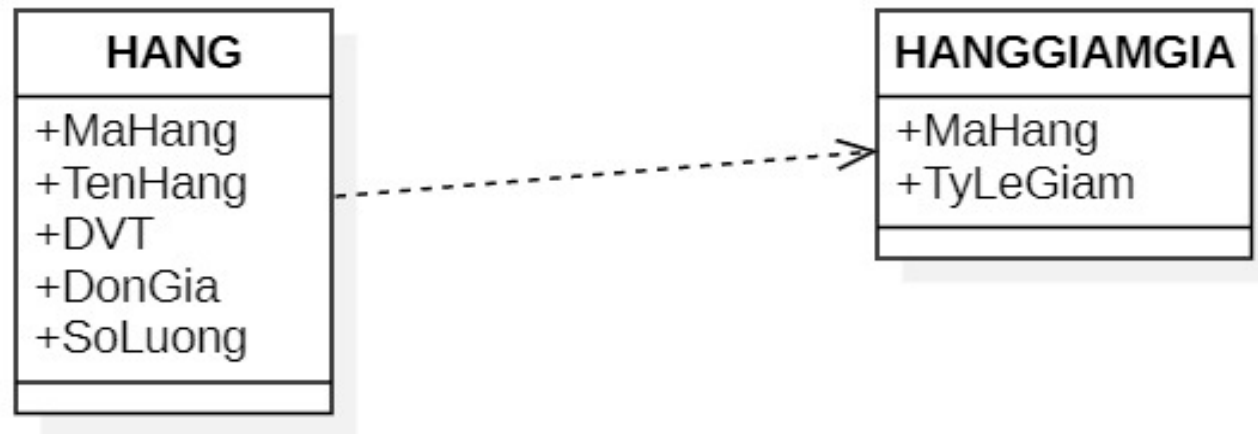
3.4.6. Mối quan hệ

- Quan hệ tổng quát hóa (Generalization): quan hệ giữa lớp cha và lớp con, thể hiện **tính thừa kế**. Được biểu diễn bằng đường thẳng với mũi tên rỗng



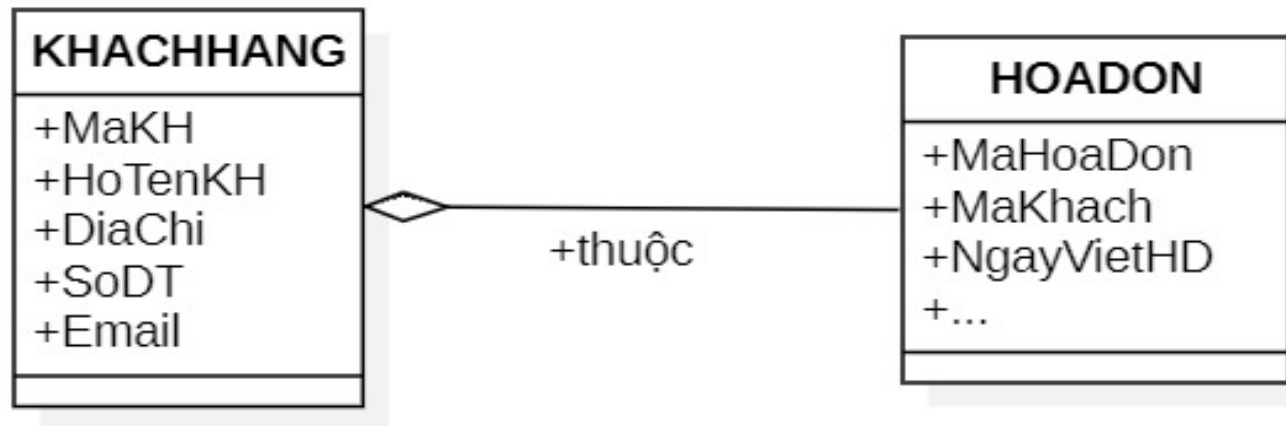
3.4.6. Mỗi quan hệ

- Quan hệ Phụ thuộc (Dependency): sự thay đổi của lớp A sẽ ảnh hưởng đến lớp phụ thuộc B. Được biểu diễn bằng đường mũi tên nét đứt



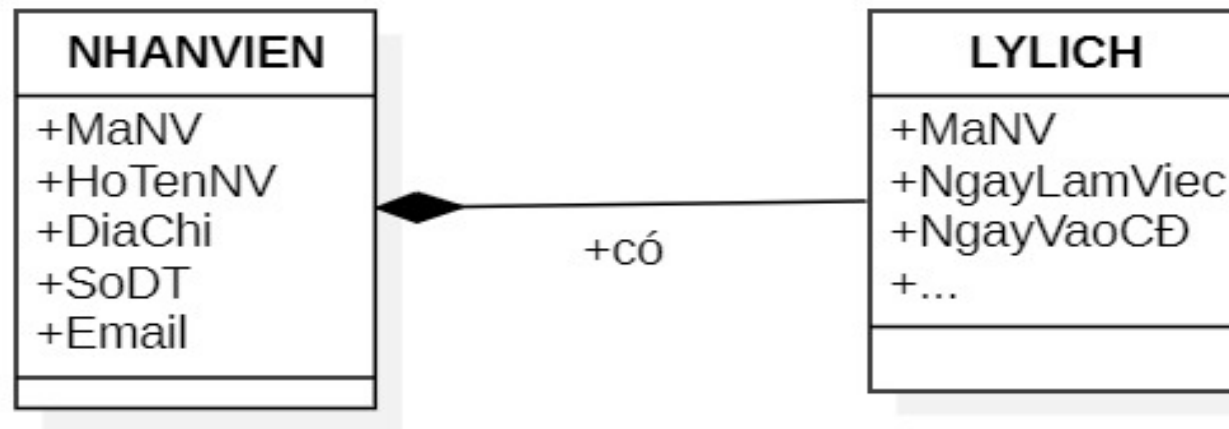
3.4.6. Mỗi quan hệ

- Quan hệ Tổng hợp (Aggregation): quan hệ mô tả một lớp A là một phần của lớp B và lớp A có thể tồn tại độc lập. Được biểu diễn bằng đường thẳng với hình thoi rỗng



3.4.6. Mỗi quan hệ

- Quan hệ Hợp thành (Composition): lớp A là một phần của lớp B và sự tồn tại của lớp B điều khiển sự tồn tại của lớp A. được biểu diễn bằng đường thẳng với hình thoi đầy



3.4.7. Các bước xây dựng mô hình khái niệm

- Xác định khái niệm
 - Xác định các khái niệm chính trong hệ thống
 - Xác định các thuộc tính và các phương thức liên quan đến từng khái niệm.
- Xác định mối quan hệ
 - Xác định mối quan hệ giữa từng khái niệm
 - Xác định loại mối quan hệ cụ thể

3.4.7. Các bước xây dựng mô hình khái niệm

- Vẽ mô hình khái niệm (biểu đồ lớp)
 - Sử dụng các ký hiệu và quy ước để vẽ mô hình khái niệm
 - Biểu diễn các khái niệm, các thuộc tính và mối quan hệ ràng buộc giữa chúng

3.4.8. Ví dụ minh họa

- Xây dựng mô hình khái niệm (biểu đồ lớp phân tích) hệ thống quản lý bán hàng
 - Xác định các khái niệm
 - Xác định các thuộc tính và phương thức (nếu có thể) của từng khái niệm
 - Xác định mối quan hệ ràng buộc giữa khái niệm
 - Vẽ mô hình khái niệm

a. Xác định các khái niệm

- Một số khái niệm thực thể dữ liệu (**lớp thực thể**)
 - Sản phẩm
 - Đơn hàng
 - Chi tiết đơn hàng
 - Khách hàng
 - Nhân viên

b. Xác định các thuộc tính và phương thức

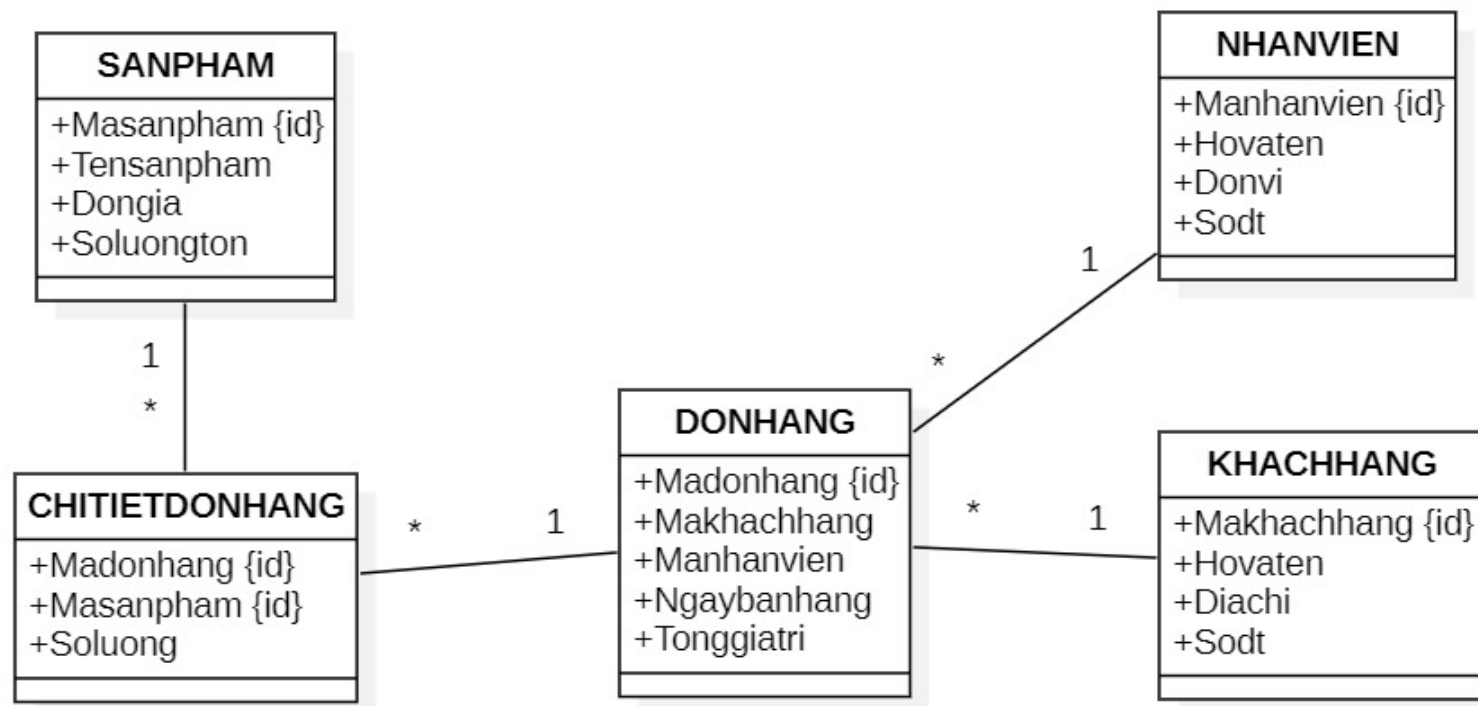
- Sản phẩm(mã sản phẩm, tên sản phẩm, đơn giá, số lượng tồn kho)
- Đơn hàng(mã đơn hàng, mã khách hàng, mã nhân viên, ngày bán hàng, tổng giá trị)
- Chi tiết đơn hàng(mã sản phẩm, mã đơn hàng, số lượng)
- Khách hàng(mã khách hàng, tên khách hàng, địa chỉ, số điện thoại)
- Nhân viên(mã nhân viên, tên nhân viên, đơn vị, số điện thoại)

c. Xác định mối quan hệ

- Mối quan hệ giữa khách hàng và đơn hàng (một-nhiều)
- Mối quan hệ giữa nhân viên và đơn hàng (một-nhiều)
- Mối quan hệ giữa đơn hàng và chi tiết đơn hàng (một-nhiều)
- Mối quan hệ giữa sản phẩm và chi tiết đơn hàng (một-nhiều)

b. Vẽ mô hình khái niệm

- Một phần mô hình khái niệm thực thể dữ liệu



4. Thiết kế hướng đối tượng

- Tổng quan
- Biểu đồ trạng thái
- Biểu đồ hoạt động
- Biểu đồ lớp thiết kế (chi tiết)
- Biểu đồ tương tác
 - Biểu đồ tuần tự
 - Biểu đồ cộng tác
- Biểu đồ gói
- Biểu đồ thành phần
- Biểu đồ triển khai

4.1. Tổng quan

- Mục tiêu xác định hệ thống sẽ được xây dựng như thế nào dựa trên kết quả của việc phân tích
- Từng bước xây dựng các mô hình, từ đó đưa ra các phần tử hỗ trợ giúp cấu thành nên một hệ thống thực sự
- Hình thành chiến lược cài đặt hệ thống

4.1. Tổng quan

- Các bước thiết kế
 - Xây dựng biểu đồ trạng thái
 - Xây dựng biểu đồ hoạt động
 - Xây dựng biểu đồ lớp thiết kế (biểu đồ lớp chi tiết)
- Xây dựng các biểu đồ tương tác: biểu đồ tuần tự và biểu đồ cộng tác
- Xây dựng biểu đồ thành phần
- Xây dựng biểu đồ triển khai

4.2. Biểu đồ trạng thái

- Tổng quan
- Ý nghĩa và vai trò
- Trạng thái
- Chuyển tiếp, sự kiện và hành động
- Một số trường hợp đặc biệt
- Xây dựng biểu đồ trạng thái
- Ví dụ minh họa

4.2.1. Tổng quan

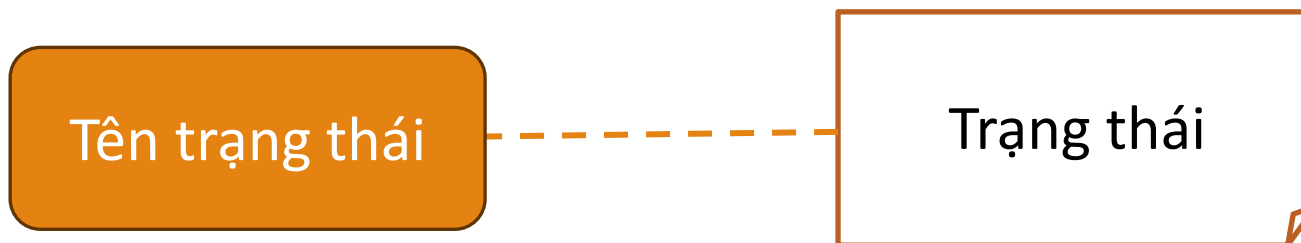
- Biểu đồ trạng thái là một loại biểu đồ UML dùng để mô tả hành vi của một đối tượng qua các trạng thái và sự kiện kích hoạt
- Biểu đồ trạng thái biểu diễn các trạng thái của đối tượng và việc sự chuyển tiếp giữa các trạng thái đó khi hành động diễn ra một sự kiện

4.2.2. Ý nghĩa và vai trò

- Mô tả chi tiết các trạng thái khác nhau mà một đối tượng có thể trải qua trong suốt vòng đời của nó
- Giúp người thiết kế và phát triển hiểu rõ cách hệ thống hoạt động dưới các tình huống khác nhau
- Giúp xác định các sự kiện (events) kích hoạt sự chuyển tiếp giữa các trạng thái của đối tượng

4.2.3. Trạng thái

- Một trạng thái là một tình huống mà đối tượng của một lớp tồn tại ở một thời điểm nào đó. Ví dụ, "Đang chờ", "Đang xử lý", "Hoàn thành".
- Tại một thời điểm đối tượng chờ đợi sự kiện để có thể thực hiện một hành động nào đó
- Ký hiệu:

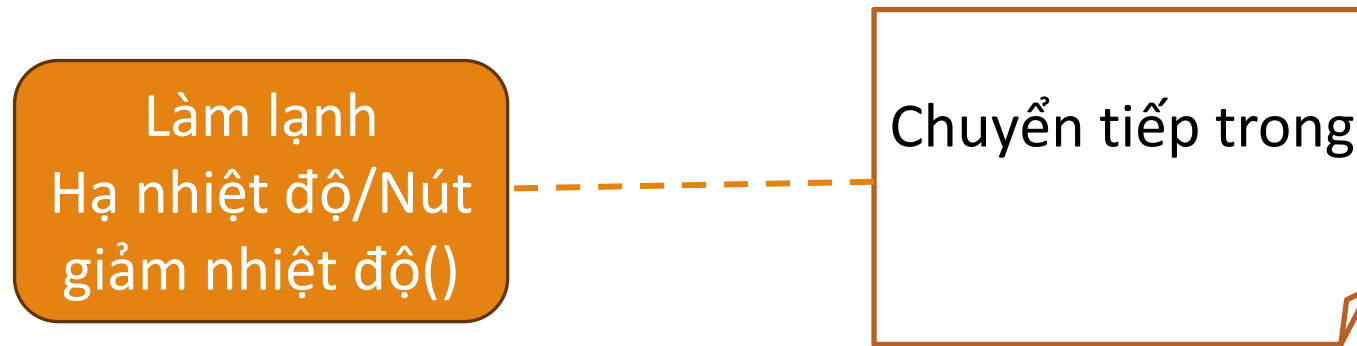


4.2.4. Chuyển tiếp, sự kiện và hành động

- Các chuyển tiếp (transition) gắn liền với các hành động kích hoạt bởi sự kiện
- Có 2 loại:
 - Chuyển tiếp trong (interne transition) một trạng thái là cho phép trả lời một sự kiện mà không rời trạng thái đó
 - Chuyển tiếp giữa các trạng thái hay chuyển tiếp ngoài (externe transition) mô tả một sự chuyển tiếp trạng thái

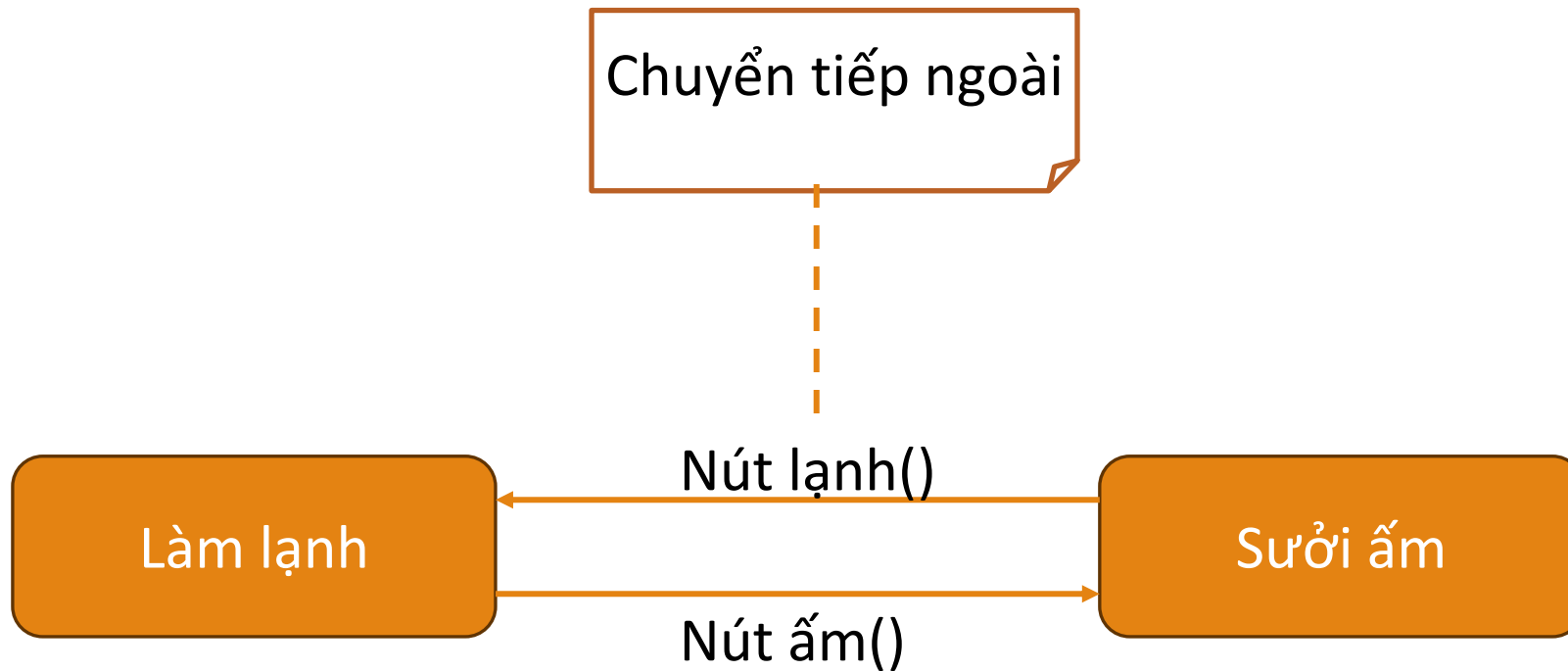
4.2.4. Chuyển tiếp, sự kiện và hành động

- Ví dụ, các trạng thái của một máy điều hòa 2 chiều



4.2.4. Chuyển tiếp, sự kiện và hành động

- Ví dụ, các trạng thái của một máy điều hòa 2 chiều



4.2.4. Chuyển tiếp, sự kiện và hành động

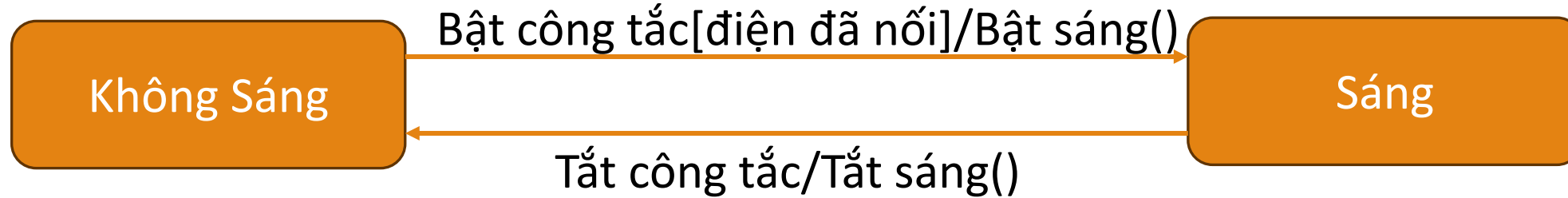
○ Một chuyển tiếp có dạng tổng quát sau:

Sự kiện[điều kiện]/hành động()

- Sự kiện: tên sự kiện để chuyển tiếp đến một trạng thái
- Điều kiện (nếu có): biểu thức logic phải đúng để có thể vượt qua một chuyển tiếp đến một trạng thái khác
- Hành động: là một thao tác cụ thể cần thực hiện để chuyển tiếp qua trạng thái khác

4.2.4. Chuyển tiếp, sự kiện và hành động

- Ví dụ, biểu đồ trạng thái của đối tượng “bóng đèn”



4.2.5. Một số trường hợp đặc biệt

- Có ba sự kiện đặc biệt của một trạng thái (không có biểu thức điều kiện)
 - **entry** và **exit** cho phép mô tả hành động cần phải thực hiện lúc **vào** hoặc lúc **ra** trạng thái
 - **do** đặc tả một hành động cần phải được thực hiện khi **ở trong** trạng thái

4.2.5. Một số trường hợp đặc biệt

- Ví dụ, trạng thái của đối tượng hộp văn bản “Nhập mật khẩu”

Nhập mật khẩu

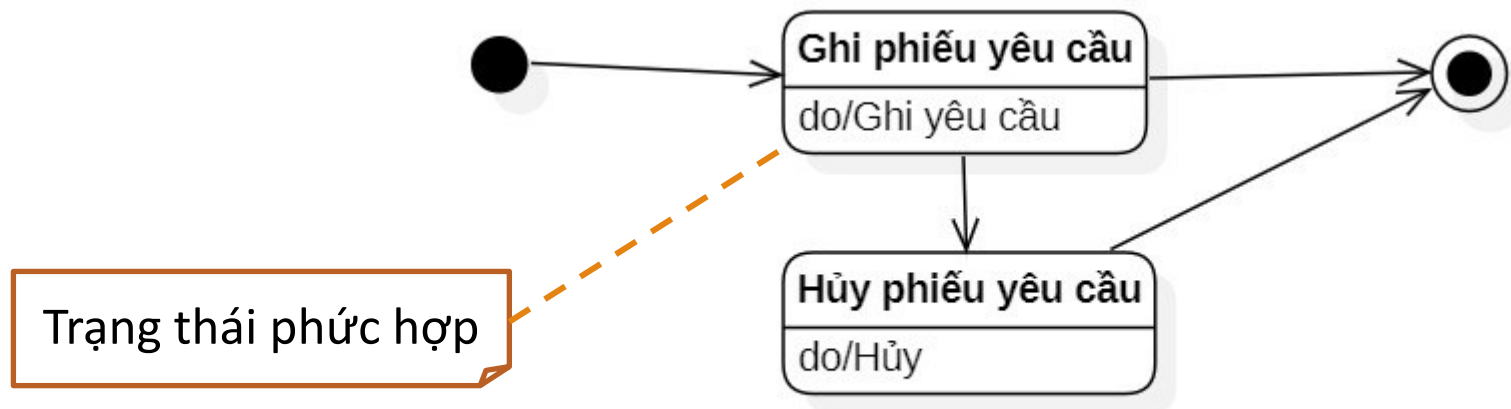
entry / không hiển thị các ký tự nhập vào

exit / hiển thị các ký tự nhập vào

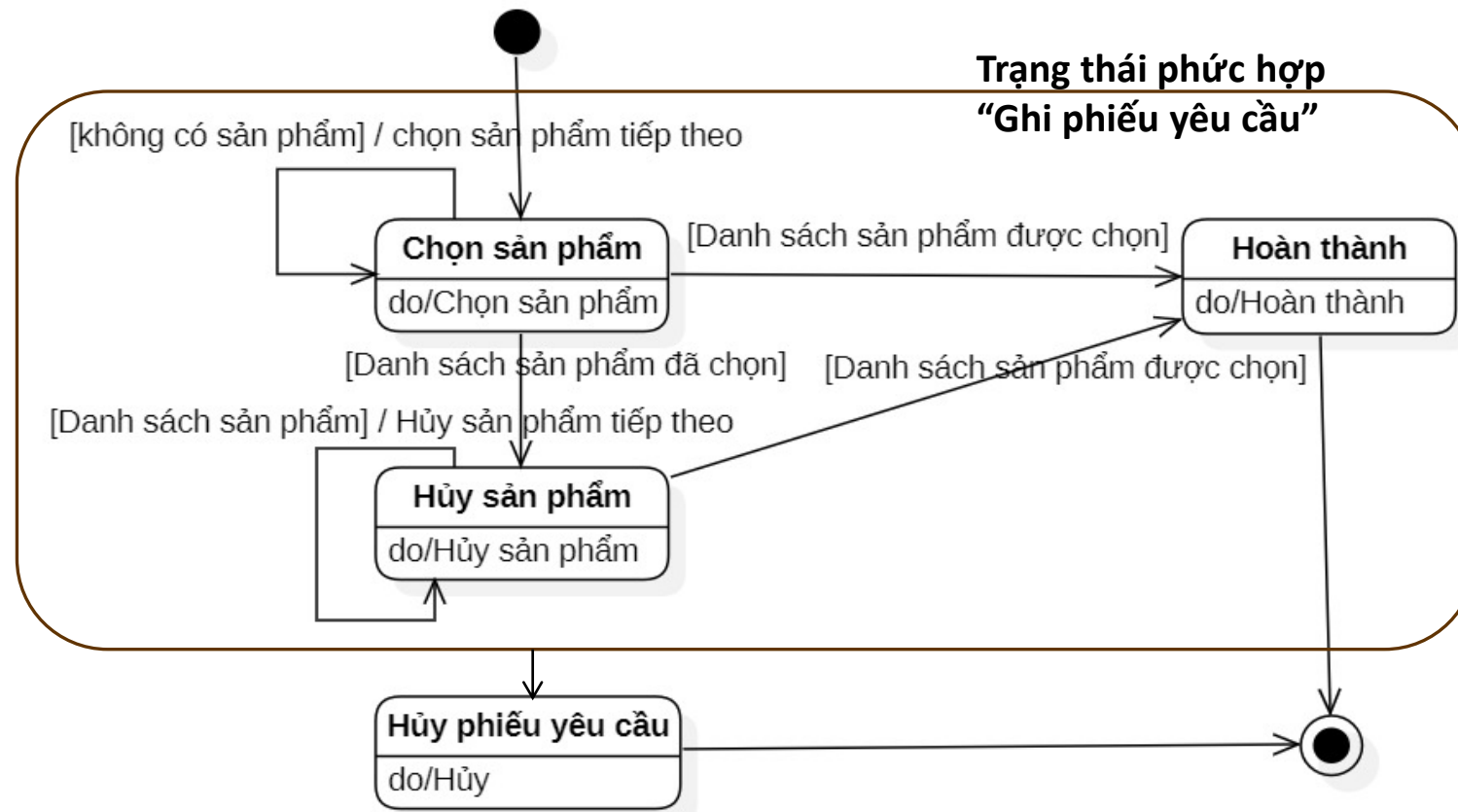
do / thực thi quản lý ký tự nhập vào của người dùng

4.2.5. Một số trường hợp đặc biệt

- Nhiều trạng thái và chuyển tiếp nối các trạng thái này có thể nhóm lại tạo nên một **trạng thái phức hợp** (composite state)
- Ví dụ, biểu đồ trạng thái của đối tượng Phiếu yêu cầu



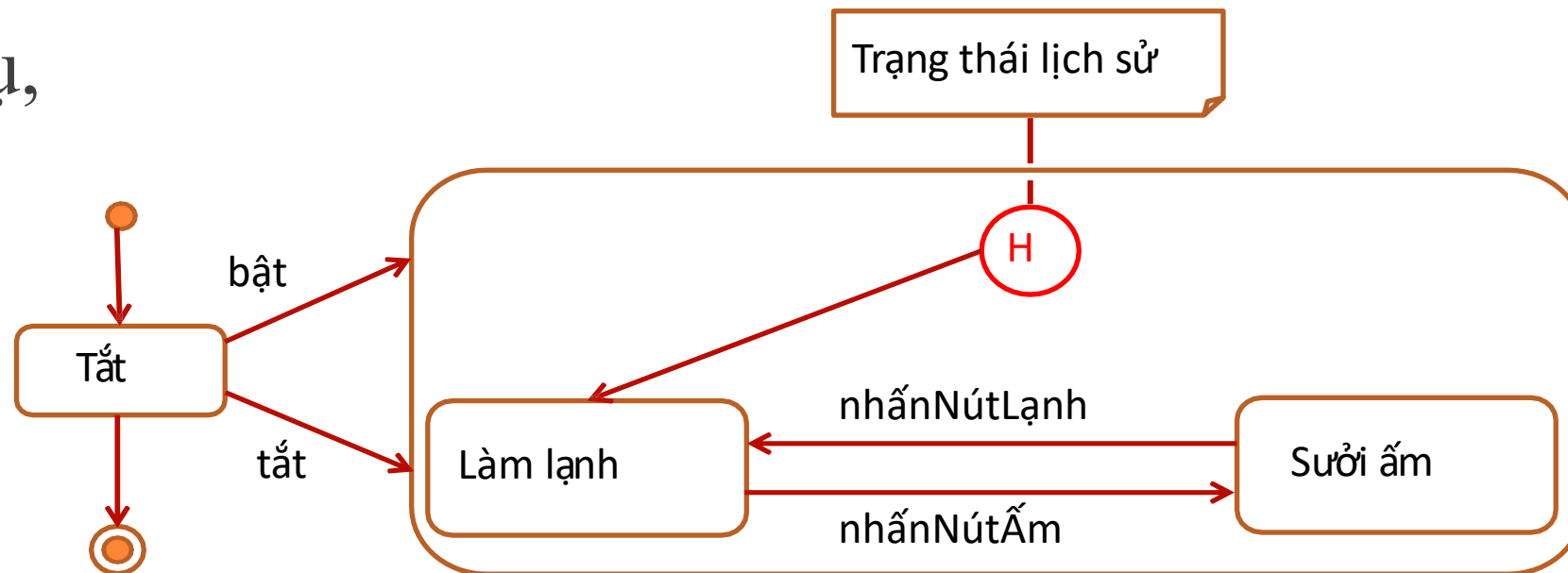
4.2.5. Một số trường hợp đặc biệt



4.2.5. Một số trường hợp đặc biệt

- Trạng thái cần ghi nhớ cuối cùng khi đi ra khỏi một trạng thái phức hợp là **trạng thái lịch sử** (history state)

- Ví dụ,



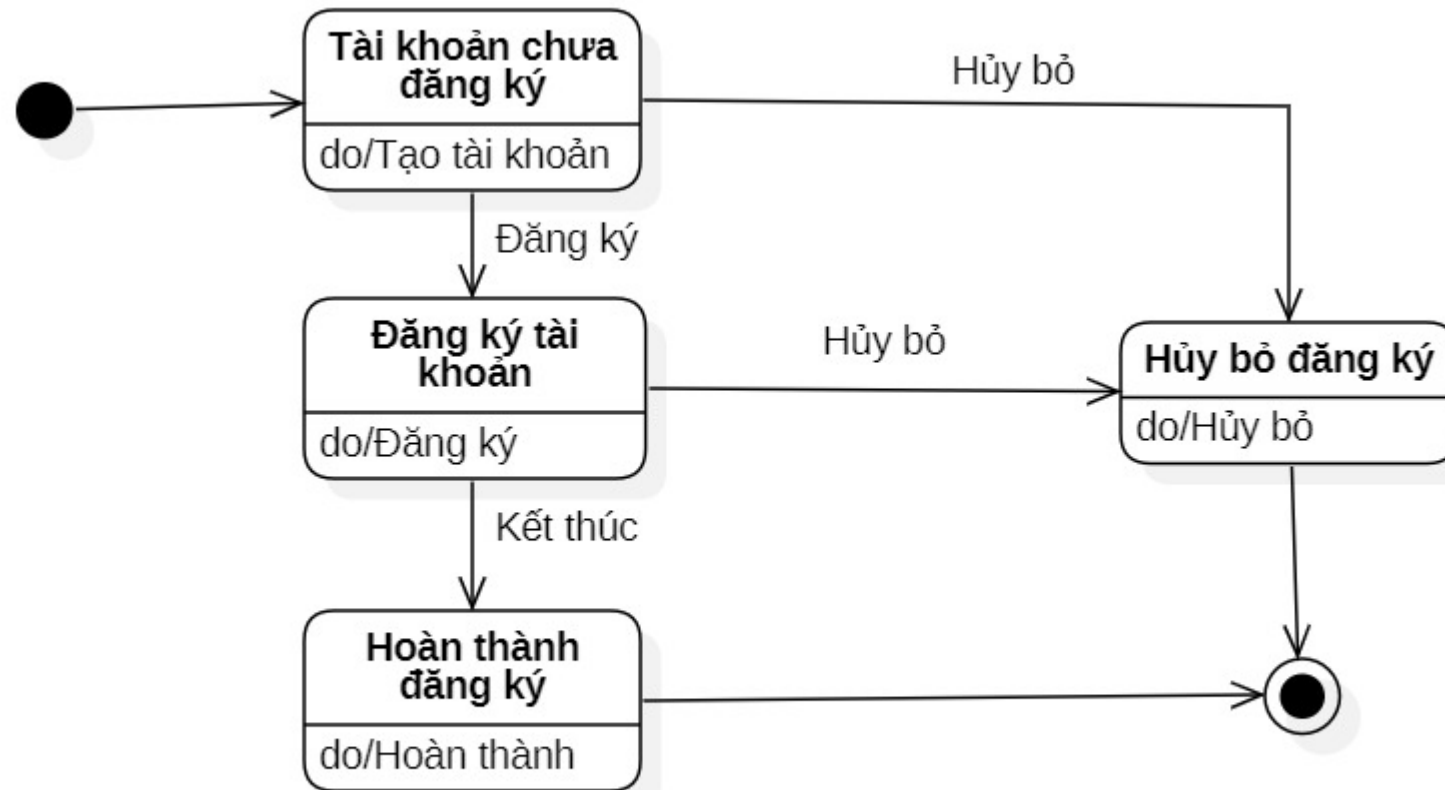
4.2.6. Xây dựng biểu đồ trạng thái

- Xác định các trạng thái: Liệt kê tất cả các trạng thái mà đối tượng có thể có
- Xác định các sự kiện, hành động: Liệt kê các sự kiện có thể kích hoạt, các hành động xảy ra trong quá trình chuyển tiếp từ trạng thái này sang trạng thái khác
- Xây dựng biểu đồ trạng thái: Sử dụng các ký hiệu UML để vẽ biểu đồ trạng thái.

4.2.7. Ví dụ minh họa

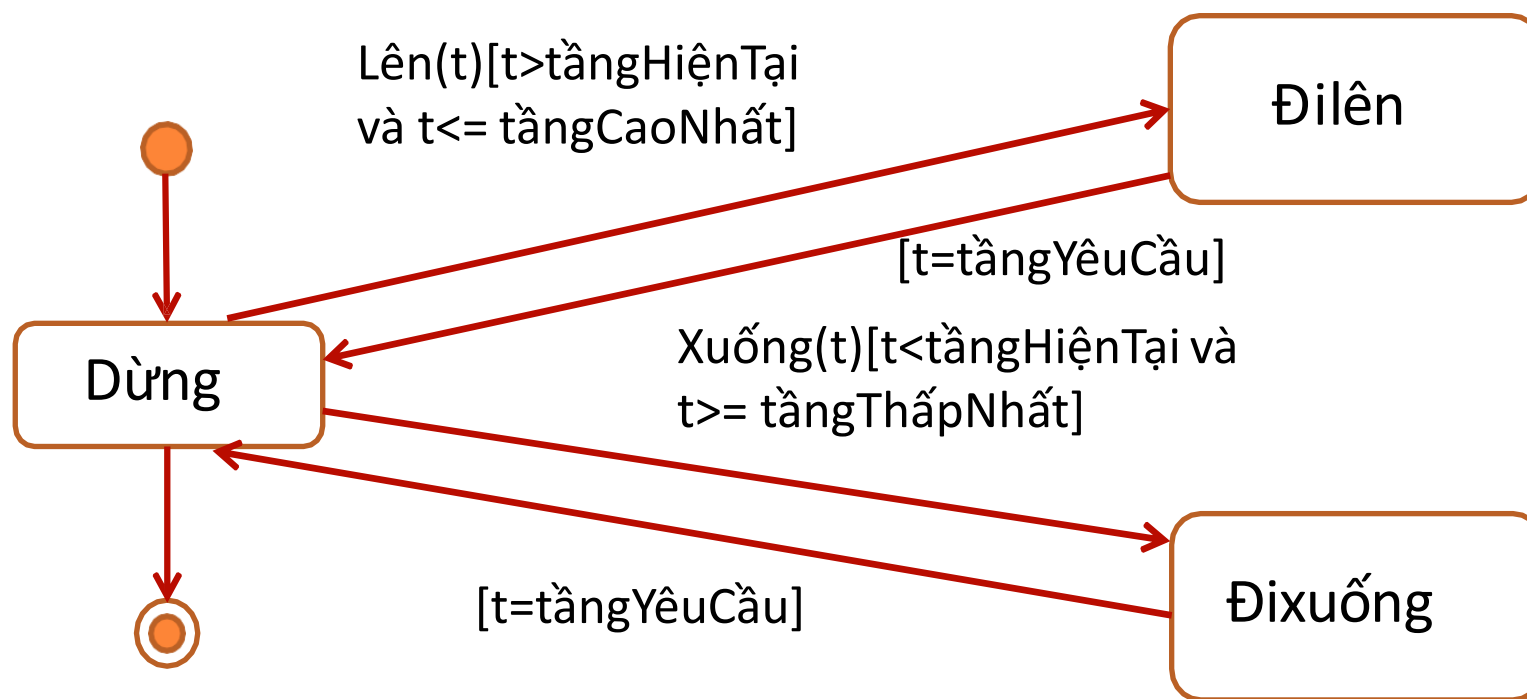
- Ví dụ 1, xây dựng biểu đồ trạng thái của đối tượng **Tài khoản người dùng** khi thực hiện đăng ký tài khoản
 - Các trạng thái: Tài khoản chưa đăng ký, Đăng ký tài khoản, Đăng ký thành công, Hủy bỏ đăng ký
 - Xác định các sự kiện, hành động: Tạo tài khoản(), Đăng ký(), Hoàn thành(), Hủy bỏ()

4.2.7. Ví dụ minh họa



4.2.7. Ví dụ minh họa

- Ví dụ 2, xây dựng biểu đồ trạng thái của đối tượng **Thang máy**

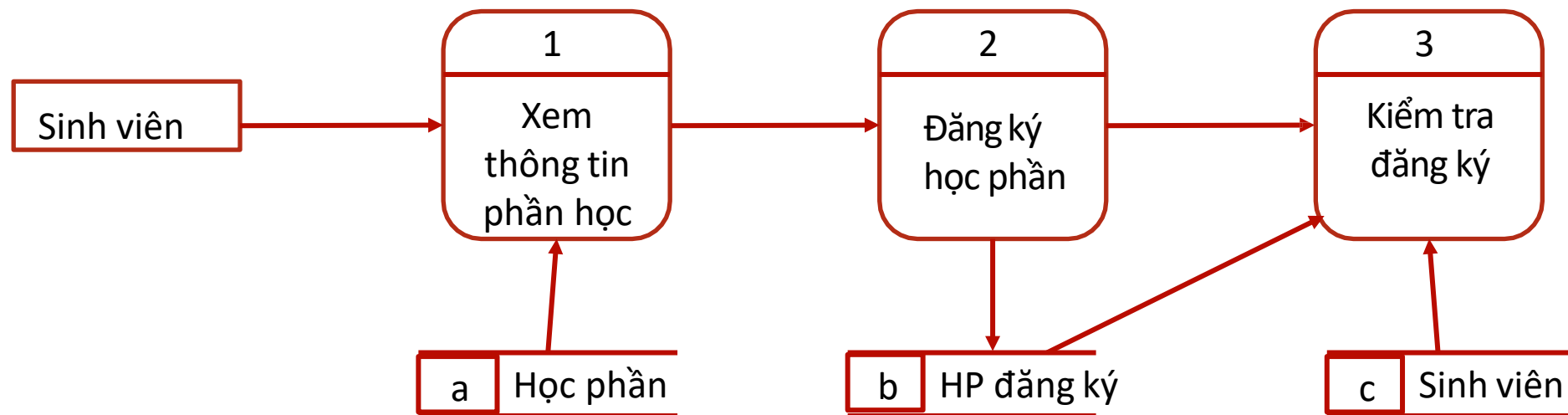


4.3. Biểu đồ hoạt động

- Giới thiệu
- Ý nghĩa và vai trò
- Hoạt động và chuyển tiếp
- Một số kiểu biểu đồ
- Xây dựng biểu đồ
- Ví dụ minh họa

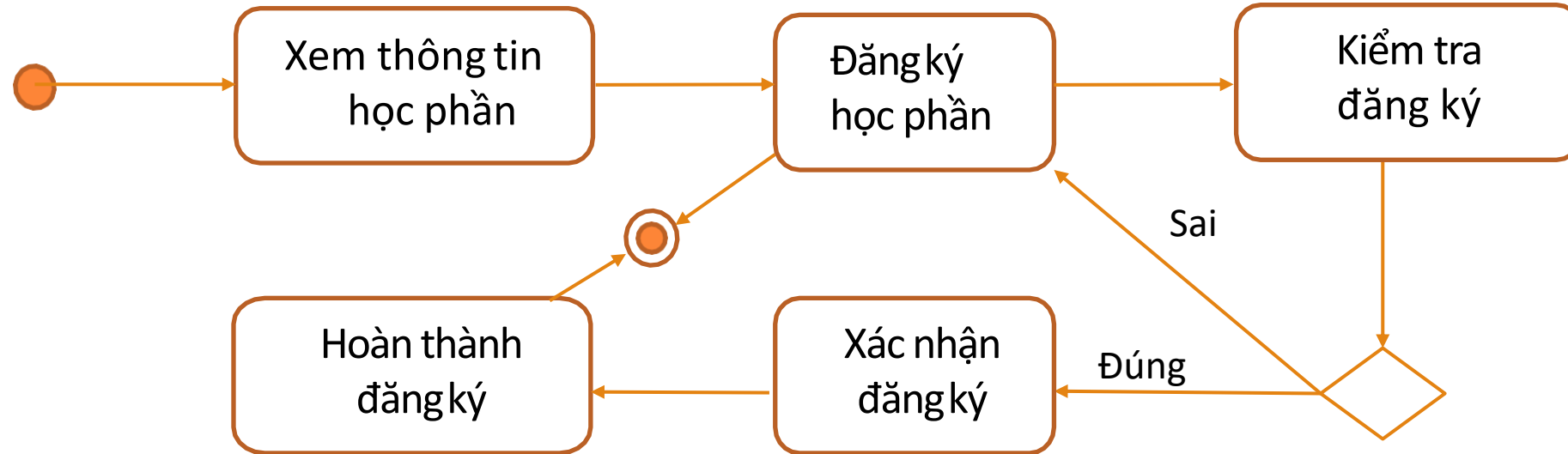
4.3.1. Giới thiệu

- Theo phương pháp phân tích hướng cấu trúc, cần xây dựng **biểu đồ luồng dữ liệu**, ví dụ:



4.3.1. Giới thiệu

- Theo phương pháp phân tích hướng đối tượng, có thể xây dựng **biểu đồ hoạt động**, ví dụ:



4.3.1. Giới thiệu

- Biểu đồ hoạt động (activity diagram) mô tả hoạt động của hệ thống so với **một** hoặc **nhiều** ca sử dụng
- Một số thành phần:
 - Các hoạt động của hệ thống và tác nhân
 - Trình tự thực hiện các hoạt động
 - Phụ thuộc có thể có giữa các hoạt động này

4.3.2. Ý nghĩa và vai trò

- Mô tả rõ ràng các bước, quy trình hoạt động của hệ thống tương ứng với một công việc ở mức trừu tượng cao có *mục tiêu xác định*. Cho phép người thiết kế và người dùng có cái nhìn tổng quan và chi tiết về cách hệ thống hoạt động
- Xác định rõ ràng các hoạt động của tác nhân (actor) và các hoạt động của hệ thống

4.3.2. Ý nghĩa và vai trò

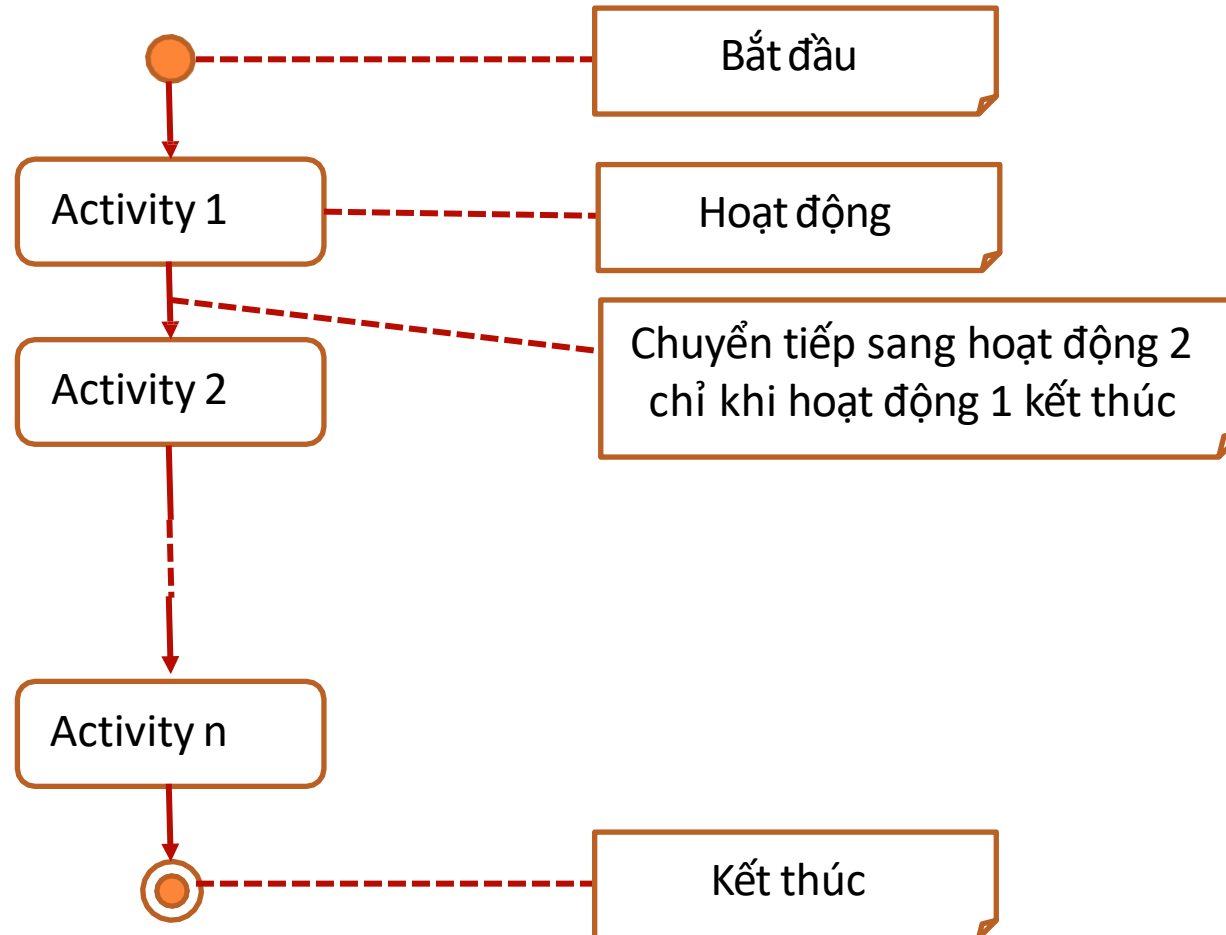
- Nhận diện các lỗ hổng hoặc vấn đề có thể tồn tại trong quy trình hoạt động của hệ thống hiện tại
- Là một công cụ giao tiếp trực quan giữa các nhà phát triển, người dùng, và các bên liên quan khác. Giúp truyền đạt thông tin một cách dễ hiểu và hiệu quả
- Được sử dụng để xác định các kịch bản kiểm thử và kiểm tra tính đúng đắn của hệ thống

4.3.3. Hoạt động và chuyển tiếp

- Một hoạt động tương ứng với một công việc xác định
- Một hoạt động không tương ứng với một thao tác trong mô hình khái niệm
 - Thao tác liên quan đến khái niệm
 - Hoạt động liên quan đến hệ thống hay tác nhân
- Từ biểu đồ hoạt động, có thể xác định thêm các thao tác của khái niệm trong mô hình khái niệm

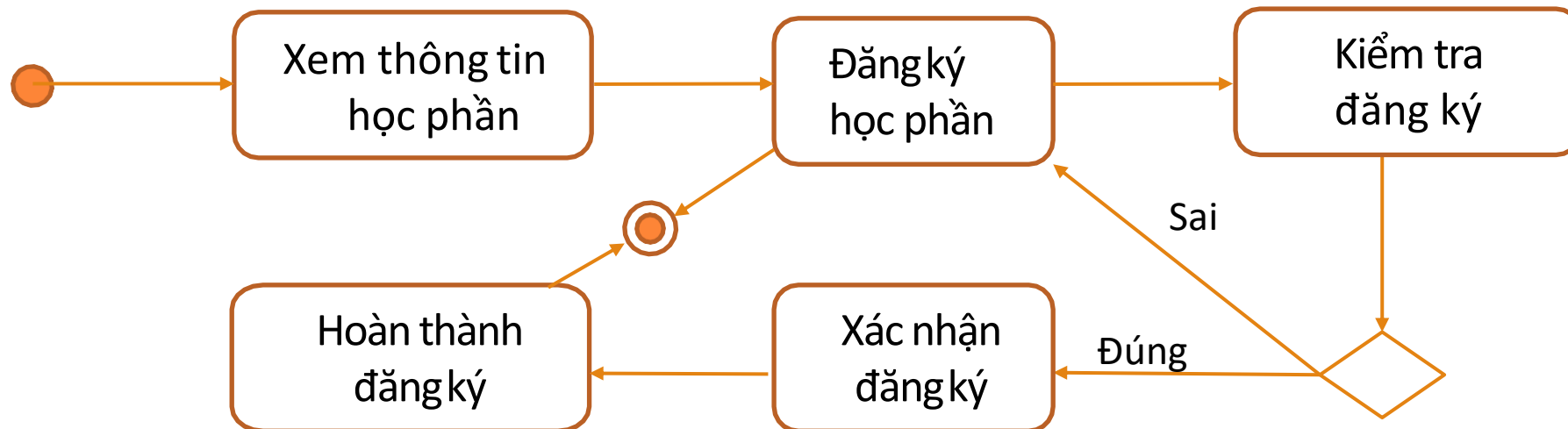
4.3.3. Hoạt động và chuyển tiếp

○ Ký hiệu



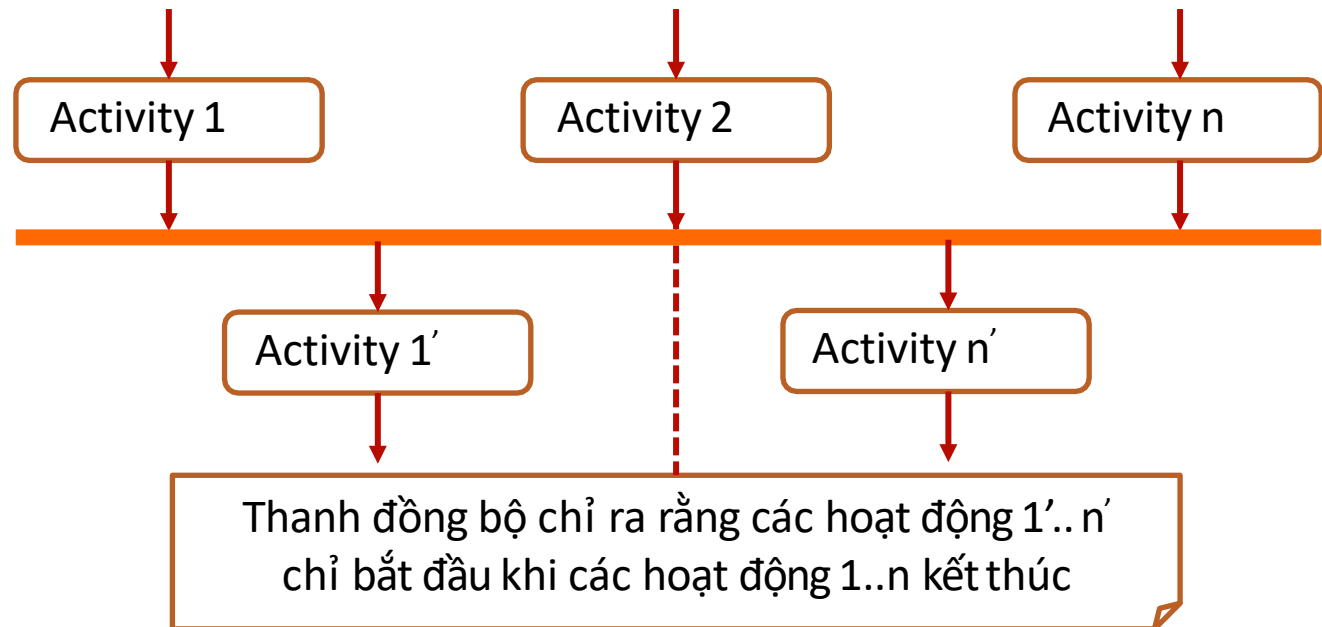
4.3.3. Hoạt động và chuyển tiếp

- Ví dụ, biểu đồ hoạt động “Đăng ký học phần”. Từ *Xem thông tin học phần* có thể xác định được thao tác *XemDShocphan()* của khái niệm HOCPHAN



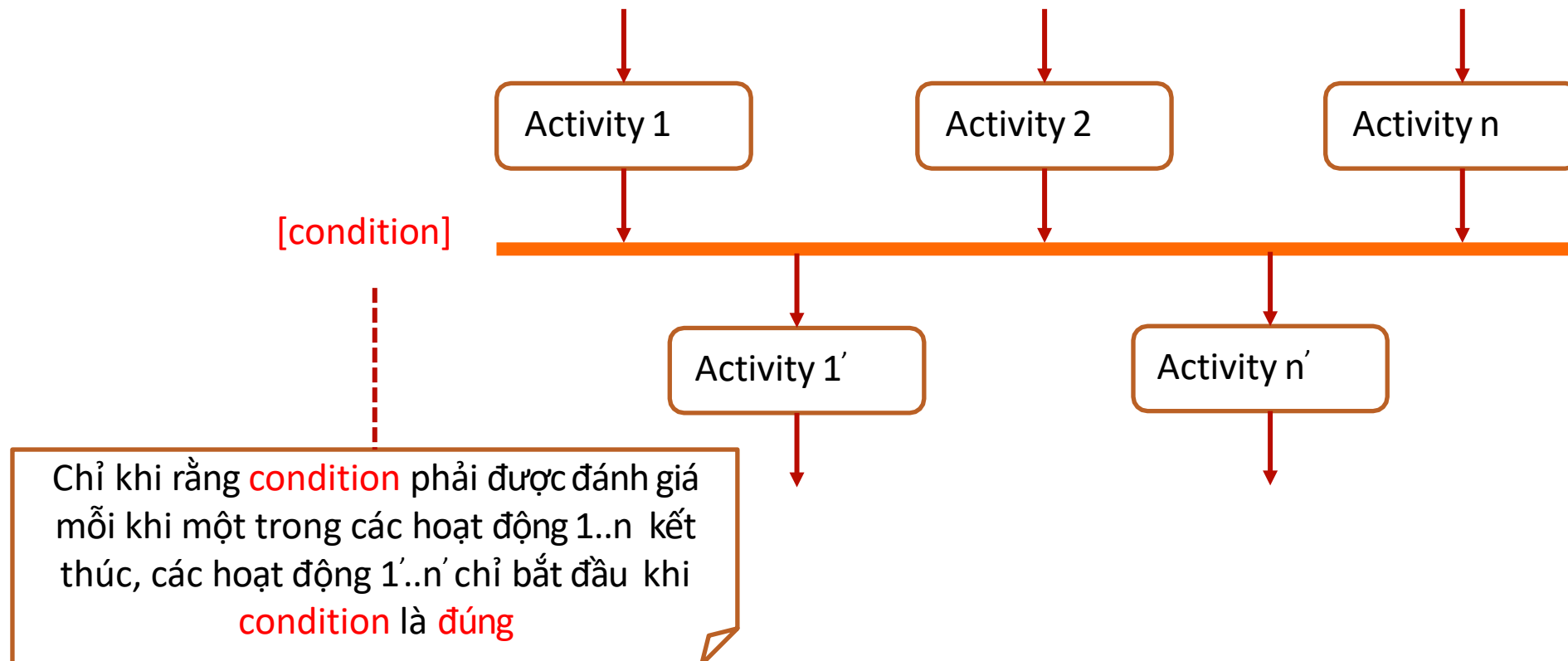
4.3.4. Một số kiểu biểu đồ

- Đồng bộ hóa các hoạt động: Các hoạt động 1'..n' (hoặc 1..n) có thể thực hiện theo **bất cứ thứ tự** nào. Hoặc chúng thực hiện **đồng thời**



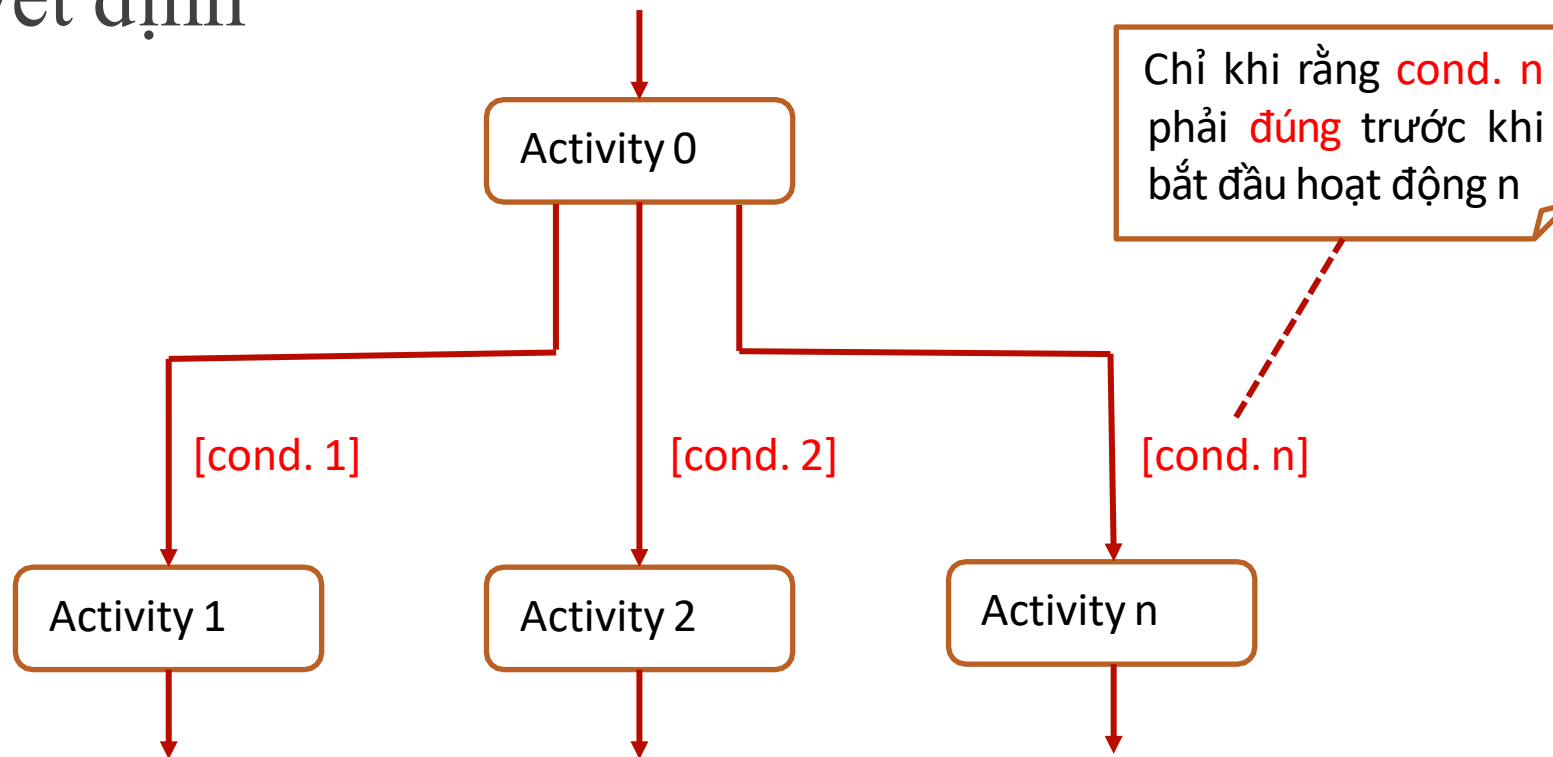
4.3.4. Một số kiểu biểu đồ

○ Đồng bộ hóa có điều kiện



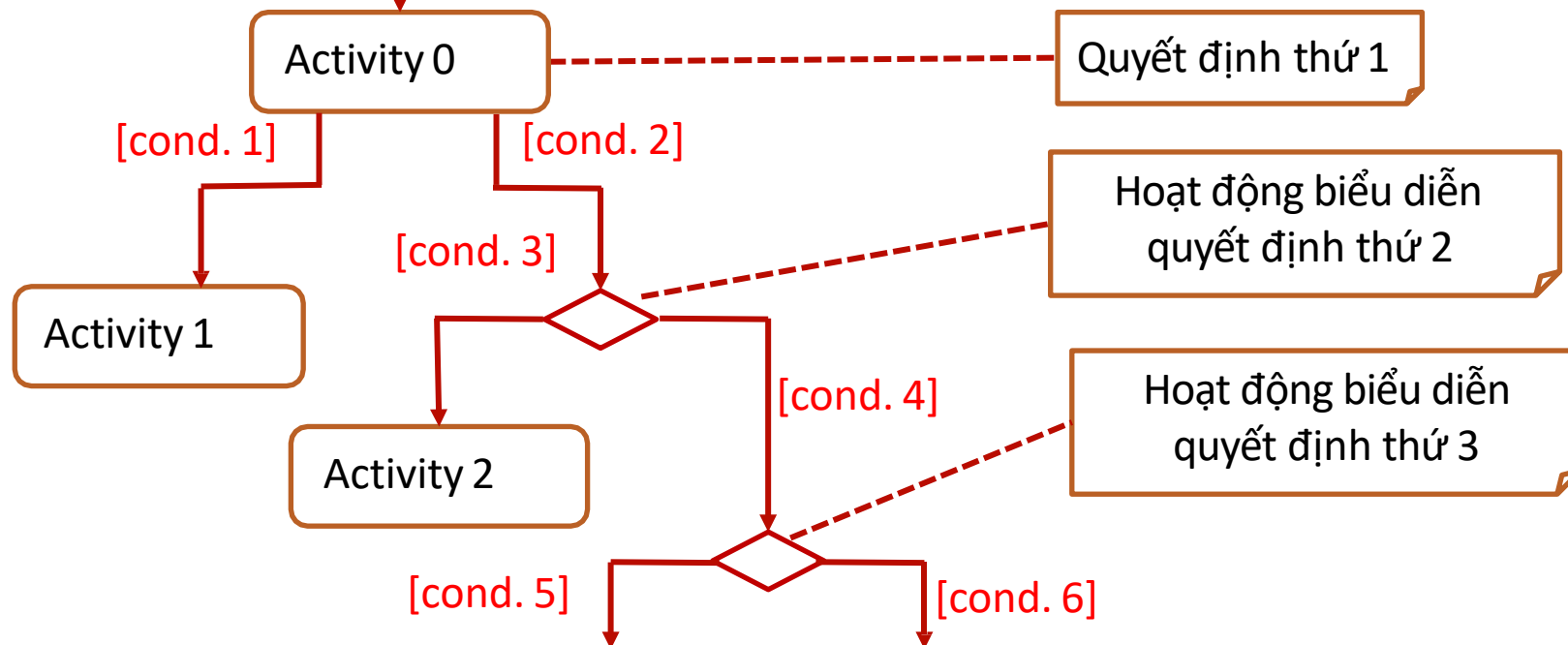
4.3.4. Một số kiểu biểu đồ

○ Quyết định



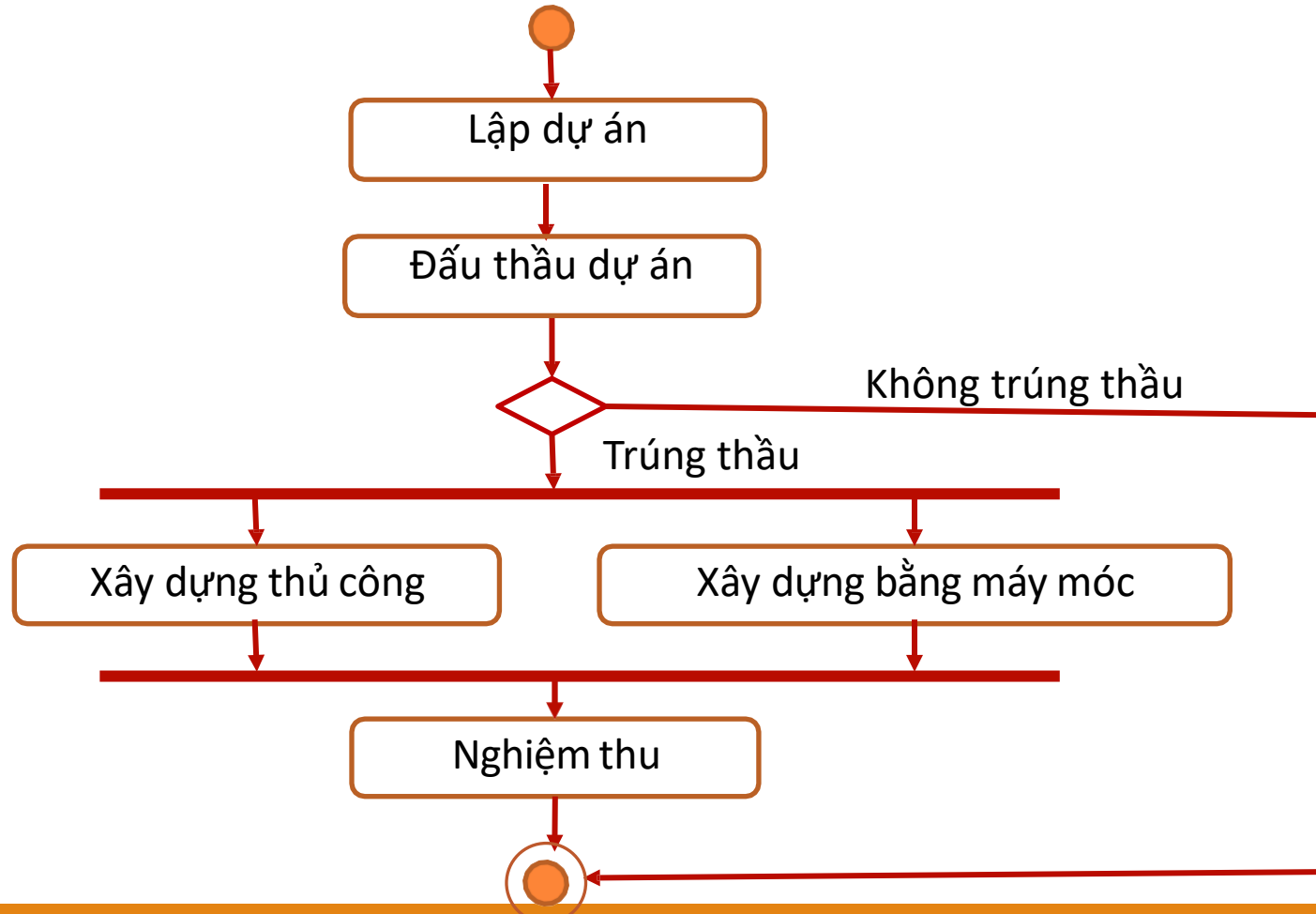
4.3.4. Một số kiểu biểu đồ

- Quyết định kết hợp: trong trường hợp nếu có nhiều quyết định đi liên nhau, thì cần phải biểu diễn các hoạt động riêng



4.3.4. Một số kiểu biểu đồ

○ Ví dụ:



4.3.5. Xây dựng biểu đồ

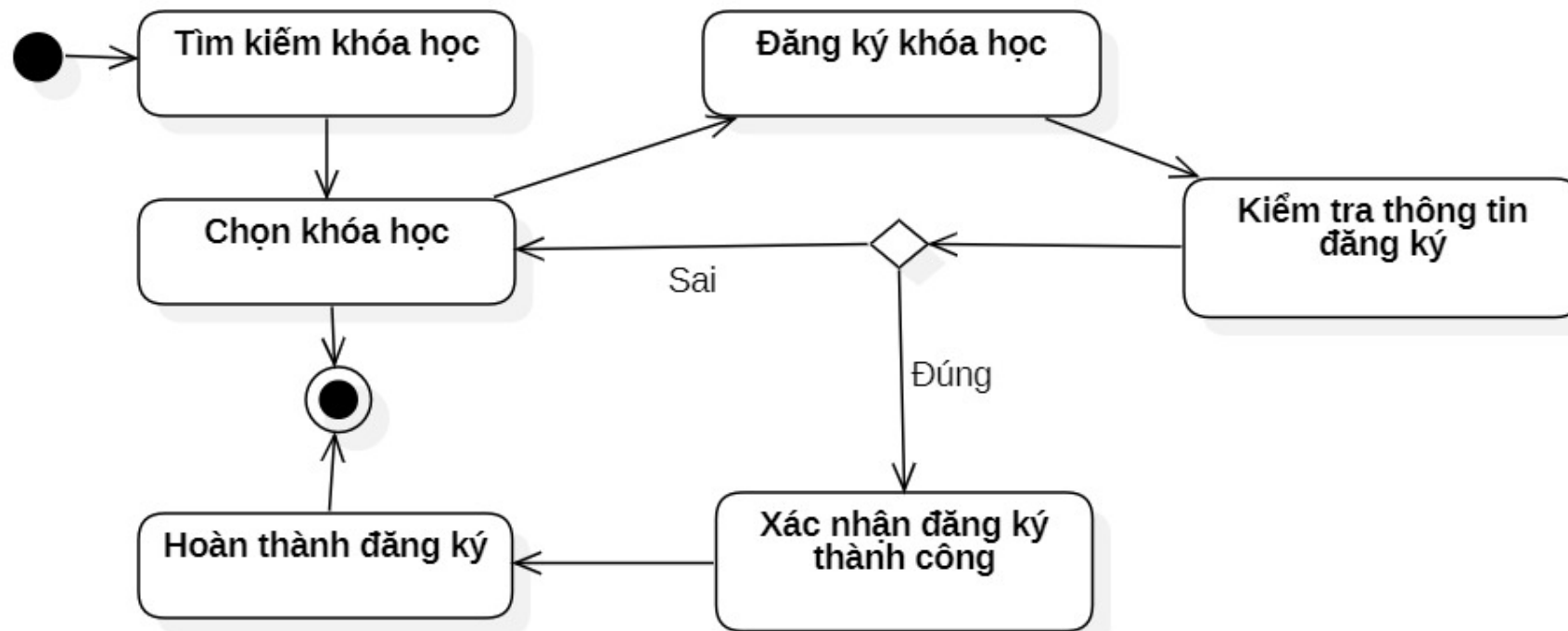
- Bước 1: Xác định các hoạt động chính để đạt được **một mục tiêu cụ thể**
 - Liệt kê tất cả các hoạt động của tác nhân và hệ thống cần thực hiện để đạt được một mục tiêu cụ thể
- Bước 2: Xác định các tác nhân liên quan.
 - Nhận diện các tác nhân (người dùng, hệ thống, ...) tương tác với hệ thống.

4.3.5. Xây dựng biểu đồ

- Bước 3: Xác định thứ tự thực hiện các hoạt động.
 - Sắp xếp các hoạt động theo thứ tự thực hiện logic
- Bước 4: Mô tả mối quan hệ phụ thuộc giữa các hoạt động.
 - Xác định các mối quan hệ phụ thuộc giữa các hoạt động

4.3.6. Ví dụ minh họa

- Ví dụ, biểu đồ hoạt động đăng ký khóa học trực tuyến



4.4. Biểu đồ lớp thiết kế

- Giới thiệu
- Các thành phần của biểu đồ
- Các bước xây dựng biểu đồ
- Ví dụ minh họa

4.4.1. Giới thiệu

- Trong pha phân tích đã xây dựng mô hình khái niệm (biểu đồ lớp phân tích)
- Pha thiết kế, xây dựng biểu đồ lớp thiết kế (design class diagram), đây là bước chi tiết hóa mô hình khái niệm
- Biểu đồ lớp thiết kế là nền tảng cho việc mã hóa

4.4.2. Các thành phần của biểu đồ

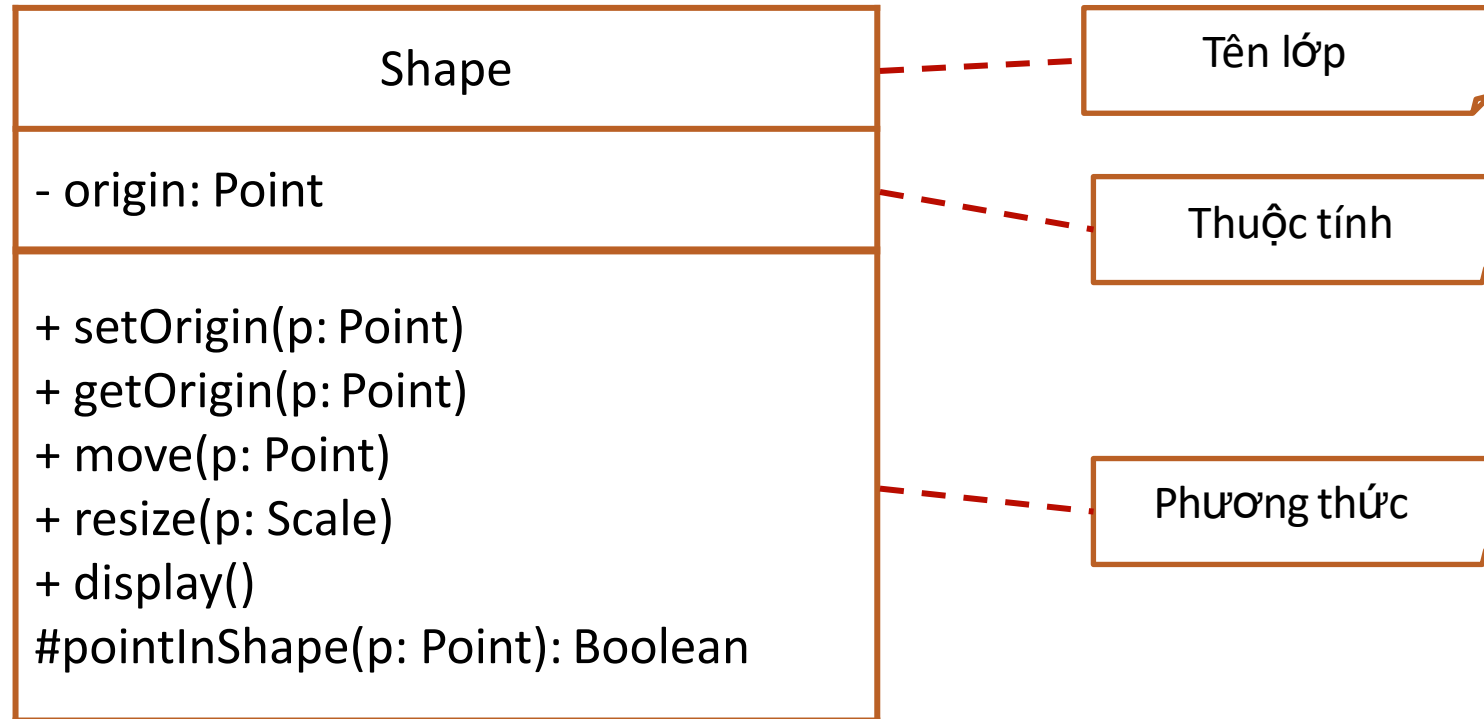
- Tất cả ký hiệu và quy tắc đối với mô hình khái niệm đều được sử dụng để xây dựng biểu đồ lớp thiết kế
- Lớp (Class): Tên lớp, thuộc tính và phương thức.
- Thuộc tính (Attributes): Đặc điểm hoặc trạng thái của đối tượng.
- Phương thức (Methods/Operations): Hành vi mà lớp có thể thực hiện.

4.4.2. Các thành phần của biểu đồ

- Mức khả biến (visibility) của mỗi thuộc tính hay mỗi phương thức:
 - Private (-): Chỉ có thể truy cập trong nội bộ lớp.
 - Protected (#): Có thể truy cập trong lớp và các lớp con.
 - Public (+): Có thể truy cập từ bên ngoài lớp.

4.4.2. Các thành phần của biểu đồ

○ Ví dụ, lớp Shape



4.4.2. Các thành phần của biểu đồ

- Mỗi quan hệ giữa các lớp:
 - Quan hệ kết hợp (Association): Thể hiện sự liên kết giữa các lớp.
 - Quan hệ kế thừa (Generalization): Một lớp con kế thừa thuộc tính và phương thức từ lớp cha.
 - Quan hệ phụ thuộc (Dependency): Một lớp phụ thuộc vào một lớp khác.

4.4.2. Các thành phần của biểu đồ

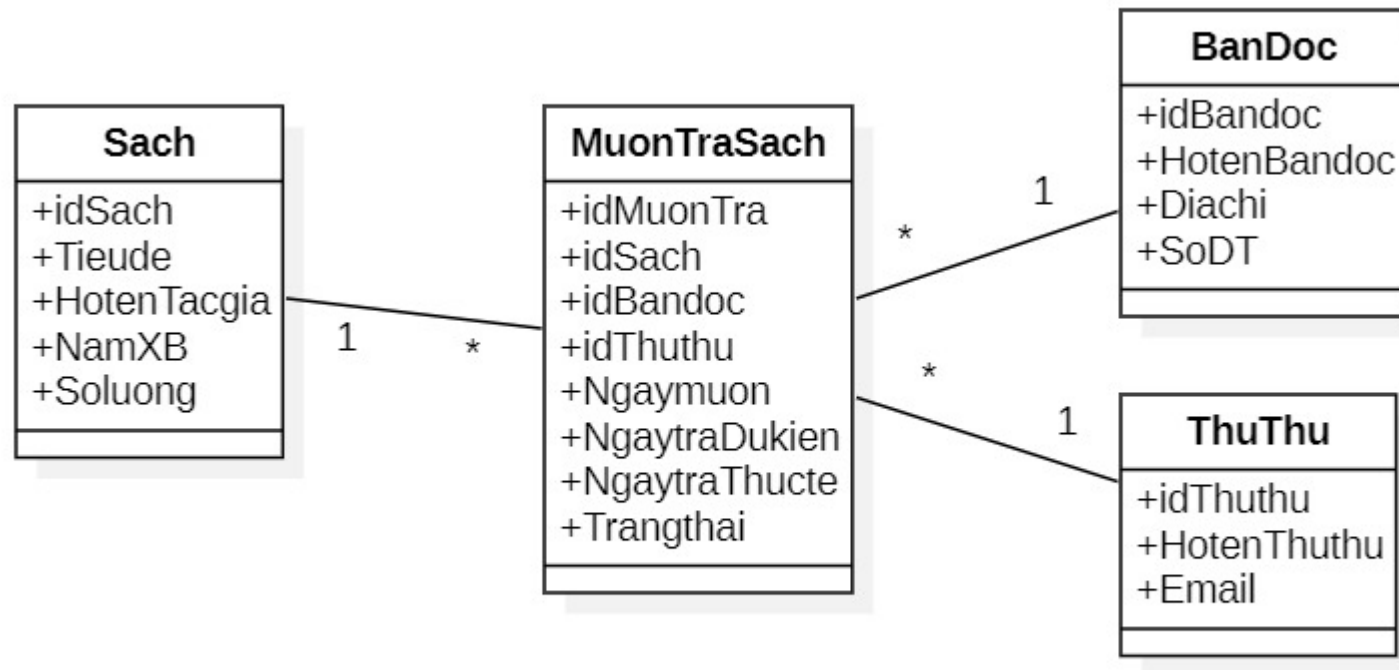
- Mỗi quan hệ giữa các lớp:
 - Quan hệ tổng hợp (Aggregation): Một lớp chứa một hoặc nhiều thực thể của lớp khác.
 - Quan hệ hợp thành (Composition): Một lớp chứa lớp khác và có vòng đời phụ thuộc.

4.4.3. Các bước xây dựng biểu đồ

- Xác định các lớp và mối quan hệ giữa các lớp: Chuyển đổi từ mô hình khái niệm sang các lớp thiết kế.
- Xác định thuộc tính và phương thức của từng lớp: Dựa vào yêu cầu của hệ thống.
- Xác định mức khả biến của các thuộc tính và phương thức: Quyết định mức truy cập phù hợp.
- Cần xem xét lại thiết kế để tối ưu hóa hiệu suất và khả năng mở rộng.

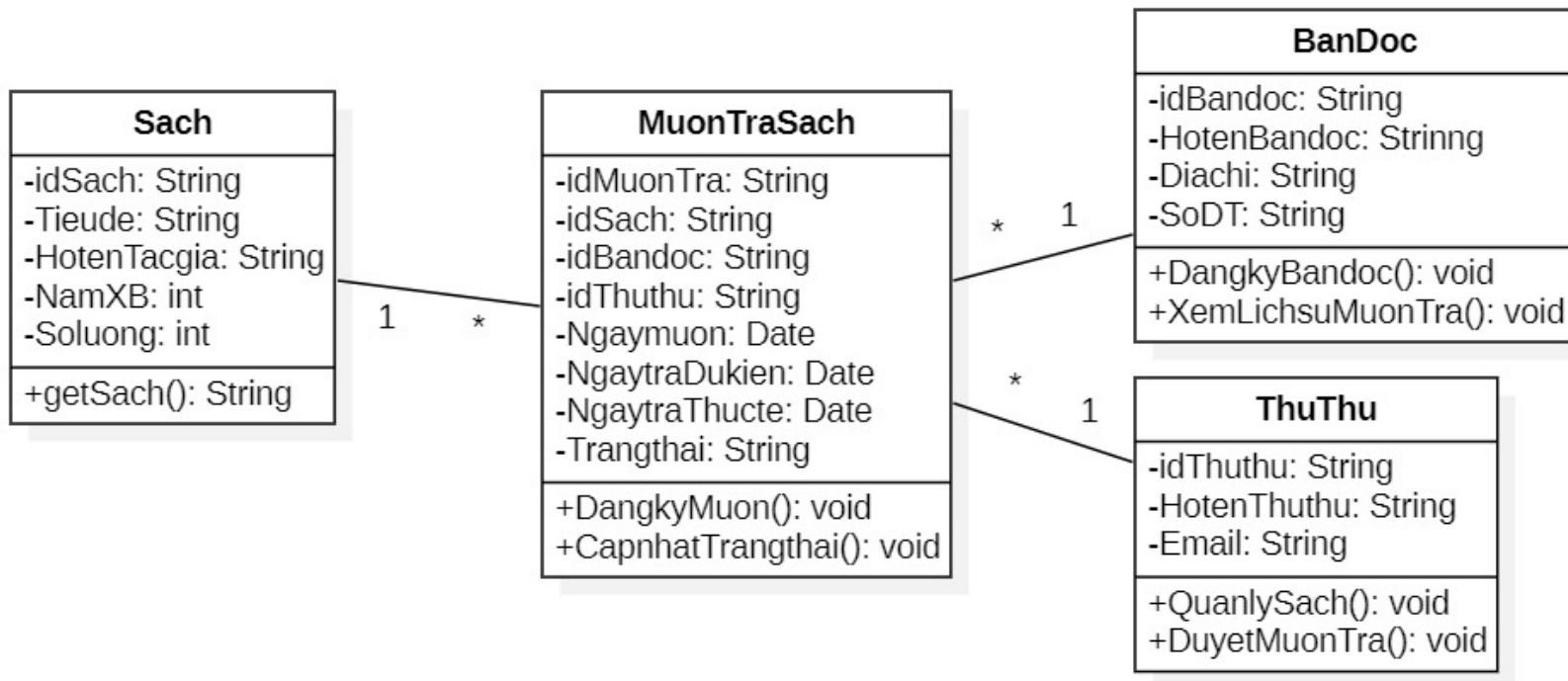
4.4.4. Ví dụ minh họa

- Một phần mô hình khái niệm thực thể Hệ thống thư viện



4.4.4. Ví dụ minh họa

○ Biểu đồ lớp thiết kế được xây dựng



4.5. Biểu đồ tương tác

- Giới thiệu
- Ý nghĩa và vai trò
- Biểu đồ tuần tự
- Biểu đồ cộng tác

4.5.1. Giới thiệu

- Biểu đồ tương tác mô tả các hành động của các đối tượng để thực hiện một chức năng của hệ thống
- Các hành động của đối tượng
 - gửi các thông điệp (message) giữa các đối tượng
 - tạo (create) và hủy (destroy) các đối tượng

4.5.2. Ý nghĩa và vai trò

- Giúp diễn tả quá trình trao đổi thông tin giữa các đối tượng trong hệ thống.
- Mô tả chi tiết cách các đối tượng tác động lẫn nhau trong một quy trình.
- Nhìn thấy một cách trực quan, logic và hỗ trợ giao tiếp giữa các nhóm phát triển hệ thống và khách hàng.

4.5.3. Biểu đồ tuần tự

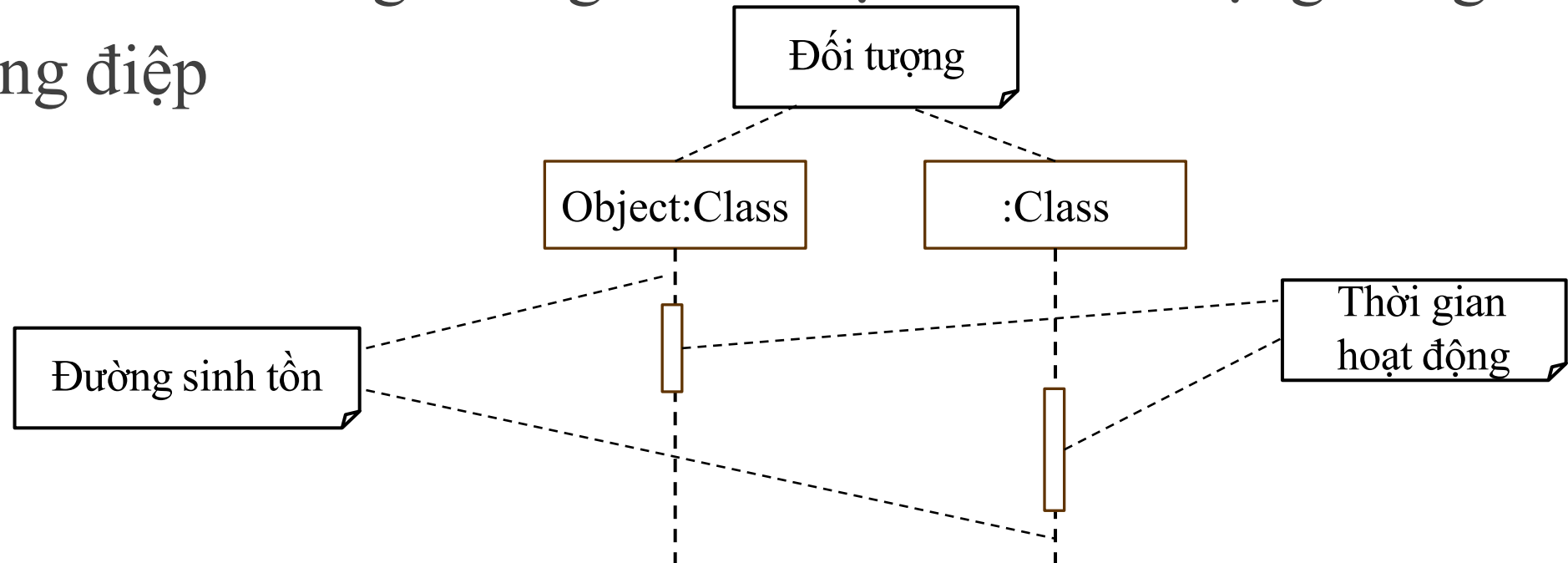
- Ý nghĩa
- Các thành phần của biểu đồ
- Các bước xây dựng biểu đồ
- Ví dụ minh họa

4.5.3.1. Ý nghĩa

- Biểu đồ tuần tự (Sequence Diagram) giúp hệ thống hóa quá trình giao tiếp giữa các đối tượng, dùng để mô tả trình tự gửi nhận giữa các đối tượng trong hệ thống theo thời gian.
- Biểu đồ tuần tự bao gồm:
 - Các đối tượng (Object), đường sinh tồn (Lifeline), thời gian hoạt động (Activation Bar)
 - Các thông điệp (Messages) trao đổi giữa các đối tượng

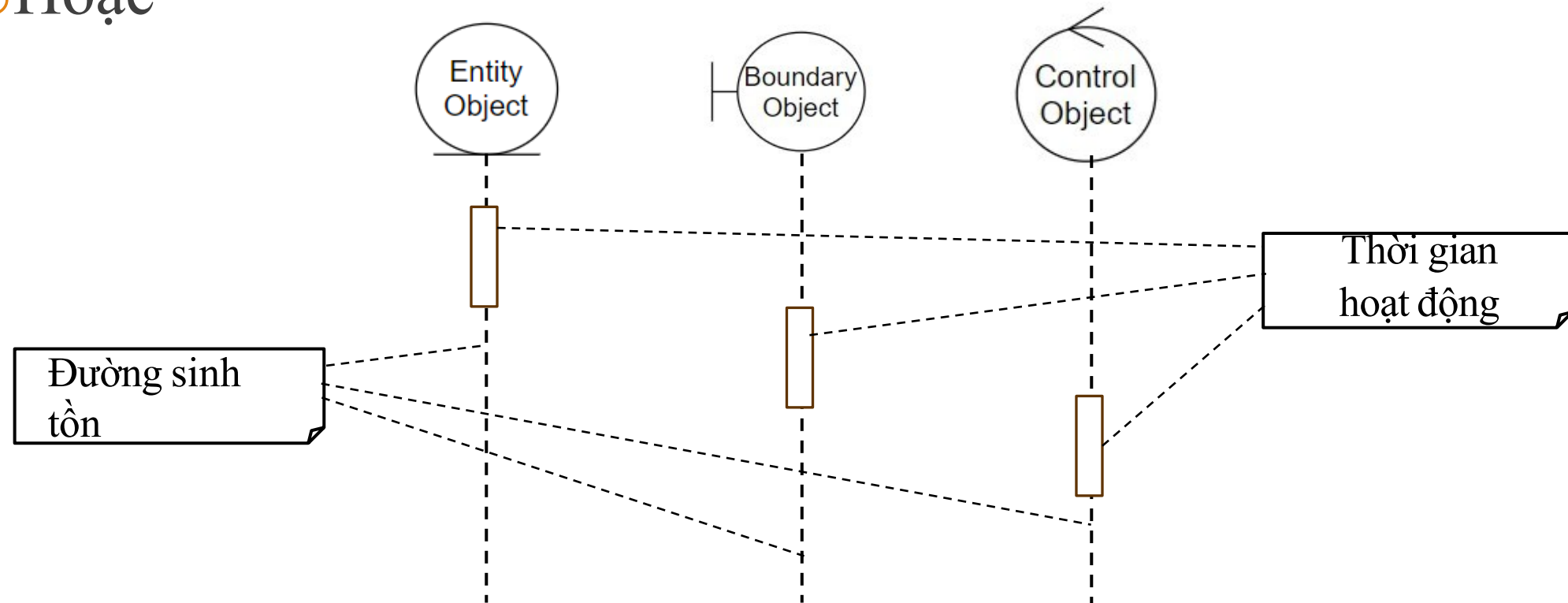
4.5.3.2. Các thành phần của biểu đồ

- Mỗi đối tượng có một đường sinh tồn và thời gian hoạt động biểu diễn khoảng thời gian tồn tại của đối tượng đang xử lý thông điệp



4.5.3.2. Các thành phần của biểu đồ

○Hoặc



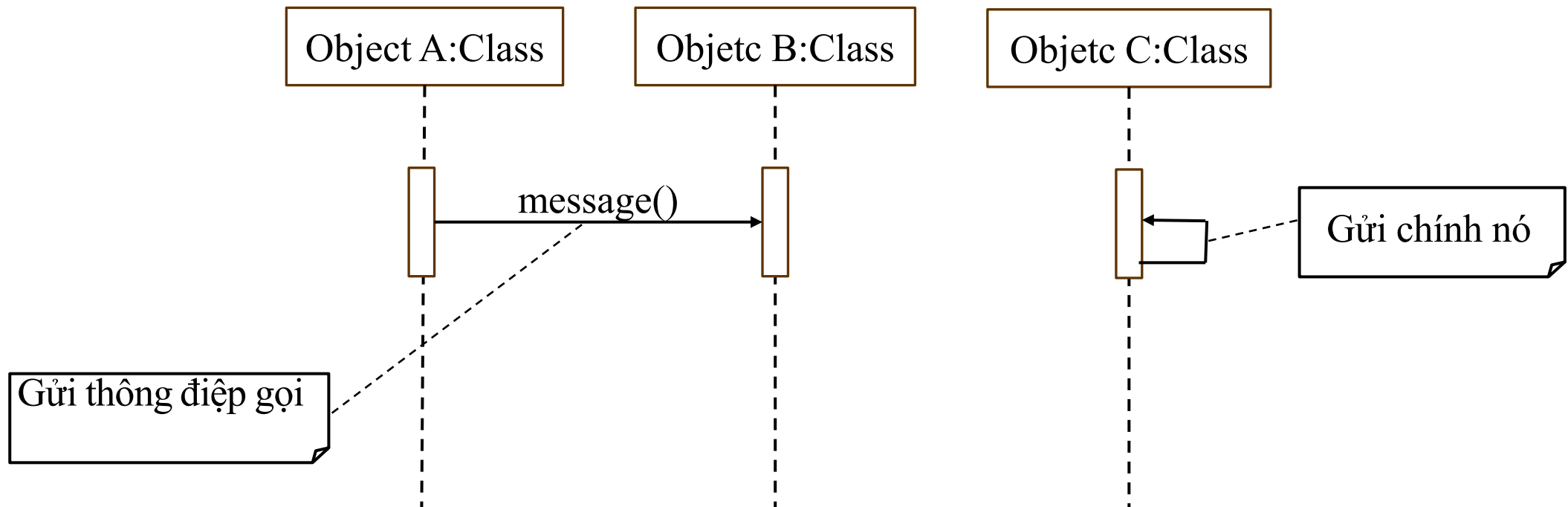
4.5.3.2. Các thành phần của biểu đồ

- Các loại thông điệp trao đổi giữa các đối tượng
 - Gọi (call)
 - Trả về (return)
 - Gửi (send)
 - Tạo (create)
 - Hủy (destroy)

4.5.3.2. Các thành phần của biểu đồ

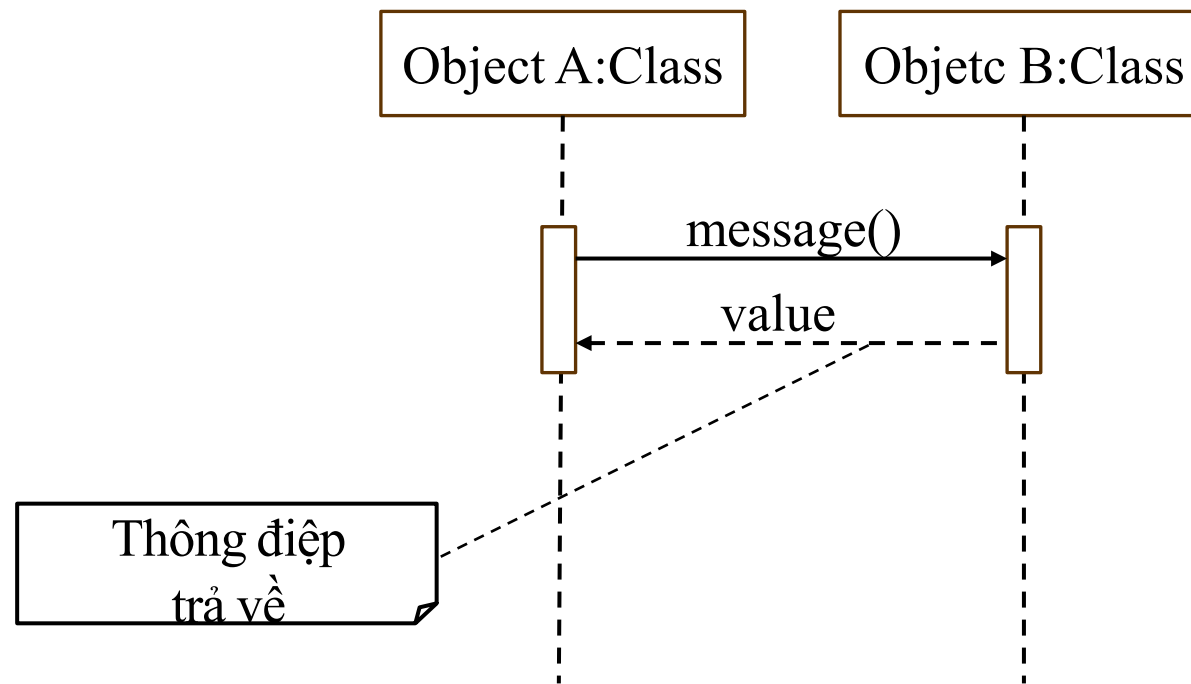
- Thông điệp gọi (call): gọi một phương thức trên đối tượng. Đối tượng gọi phải đợi thông điệp được thực hiện kết thúc mới có thể thực hiện công việc khác (thông điệp đồng bộ_Synchronous message).
- Một đối tượng có thể gửi thông điệp gọi cho chính nó (Self-message)

4.5.3.2. Các thành phần của biểu đồ



4.5.3.2. Các thành phần của biểu đồ

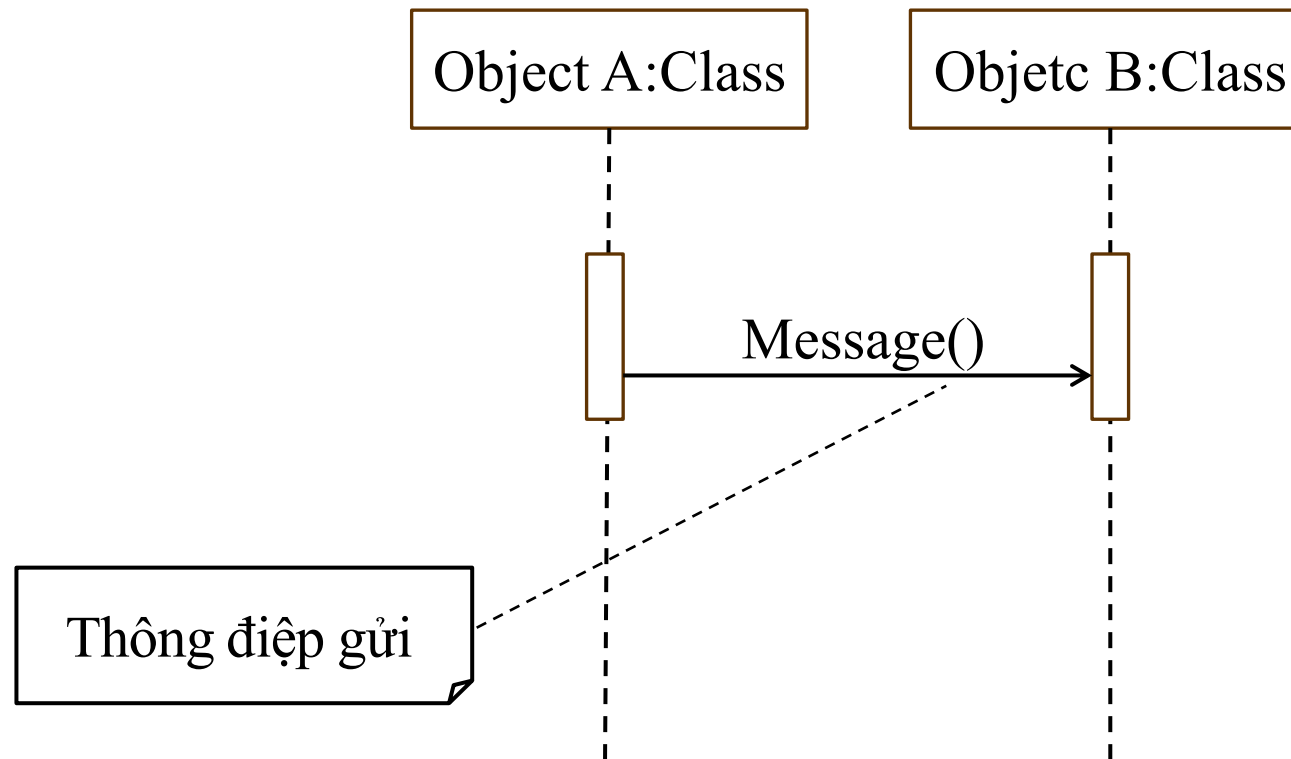
- Thông điệp trả về (return): trả về một giá trị cho đối tượng gọi



4.5.3.2. Các thành phần của biểu đồ

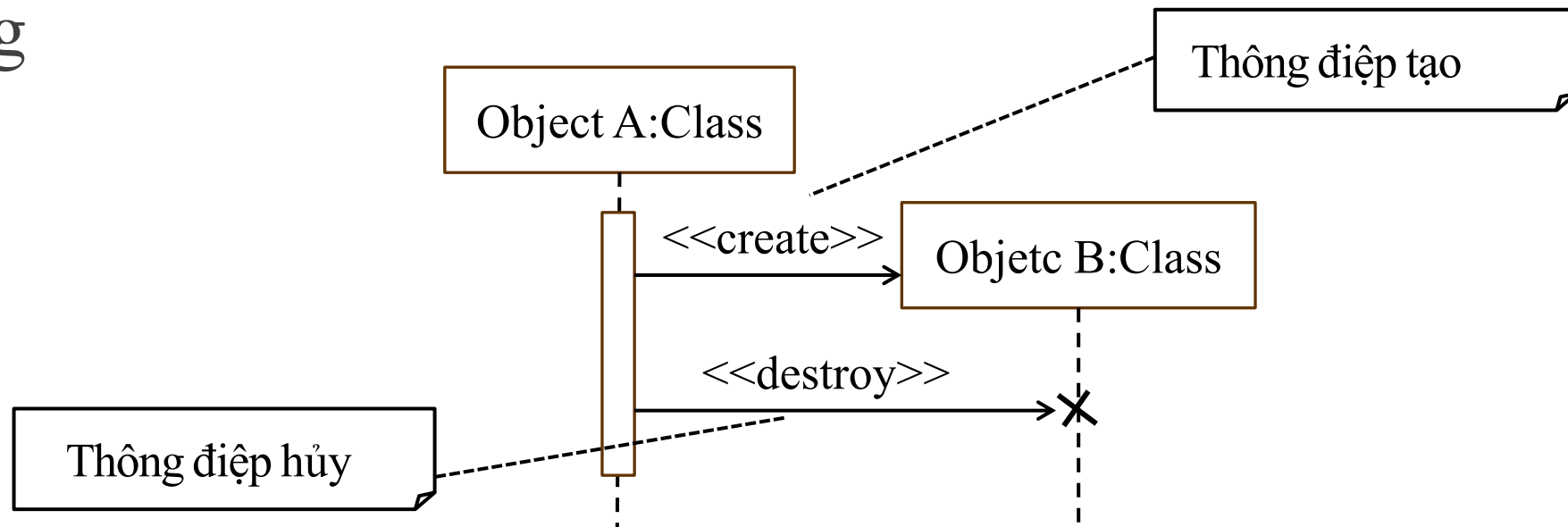
- Thông điệp gửi (send): gửi một tín hiệu đến một đối tượng. Đối tượng gửi thông điệp gửi, có thể tiếp tục thực hiện công việc khác mà không chờ phản hồi (thông điệp không đồng bộ _Asynchronous message).

4.5.3.2. Các thành phần của biểu đồ



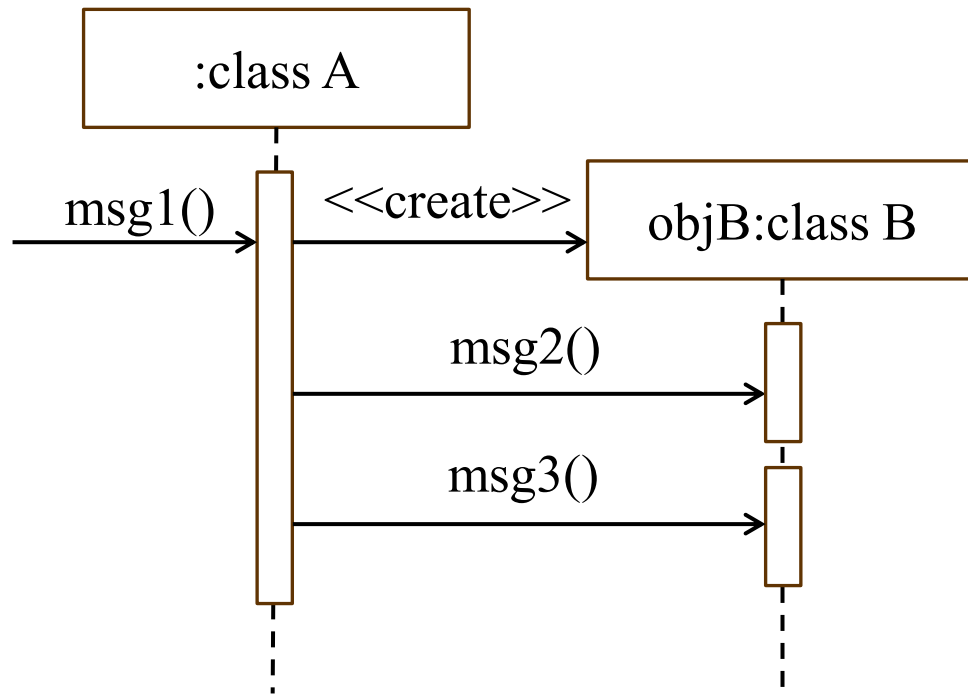
4.5.3.2. Các thành phần của biểu đồ

- Thông điệp tạo (create): gọi phương thức tạo một đối tượng
- Thông điệp hủy (destroy): gọi phương thức hủy một đối tượng



4.5.3.3. Các thành phần của biểu đồ

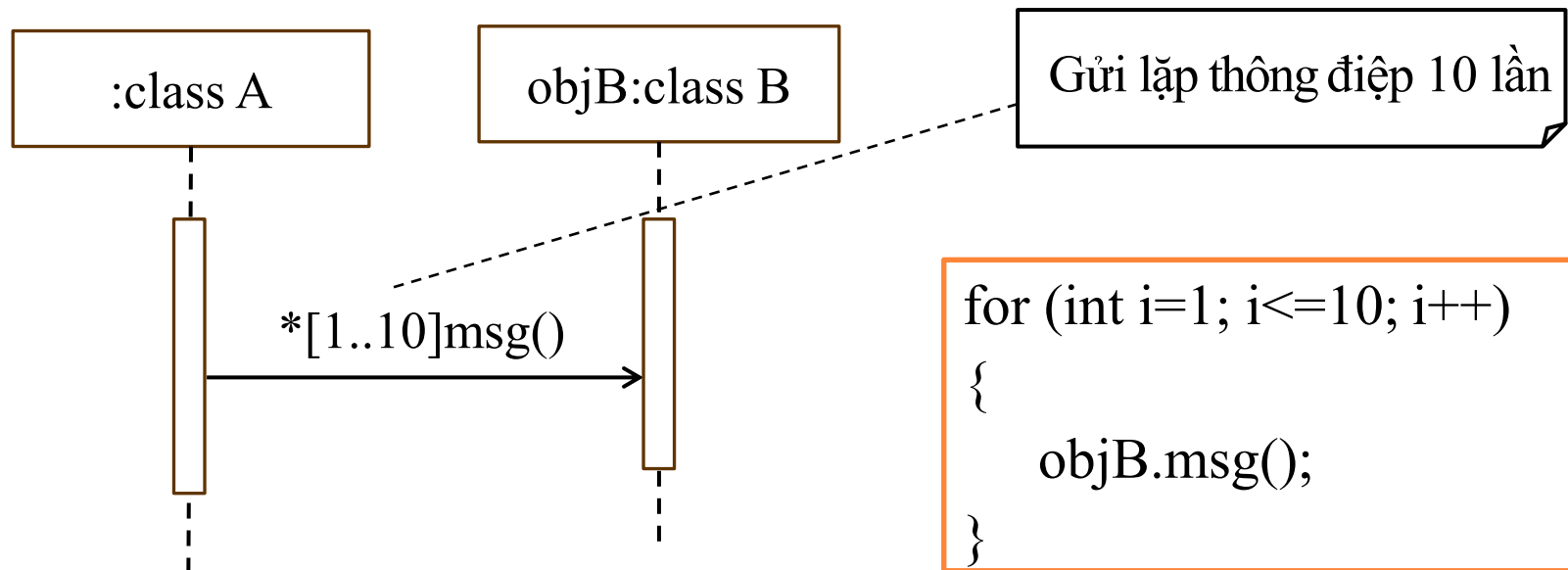
○ Ví dụ 1:



```
public class A
{
    private B objB;
    public void msg1()
    {
        objB = new B();
        objB.msg2();
        objB.msg3();
    }
}
public class B
{
    ...
    public void msg2(){...}
    public void msg3(){...}
}
```

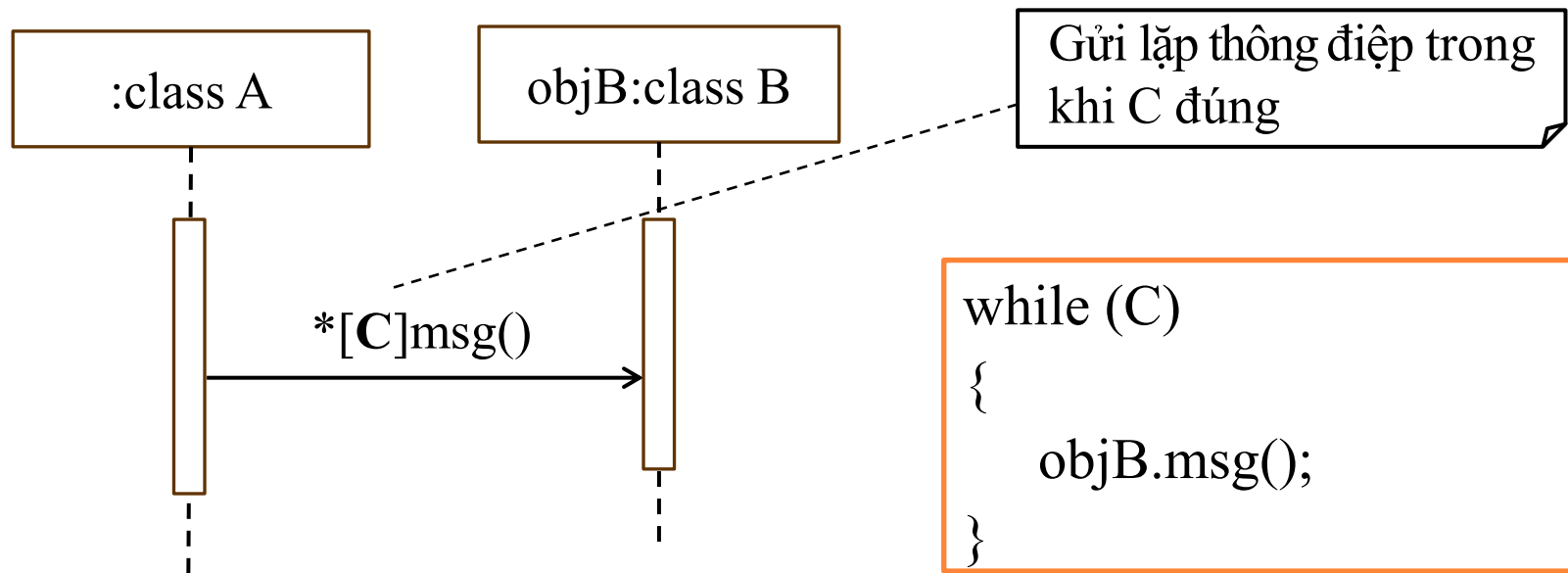

4.5.3.3. Các thành phần của biểu đồ

- Ví dụ 2: Thông điệp gửi có thể được gửi “lặp” nhiều lần



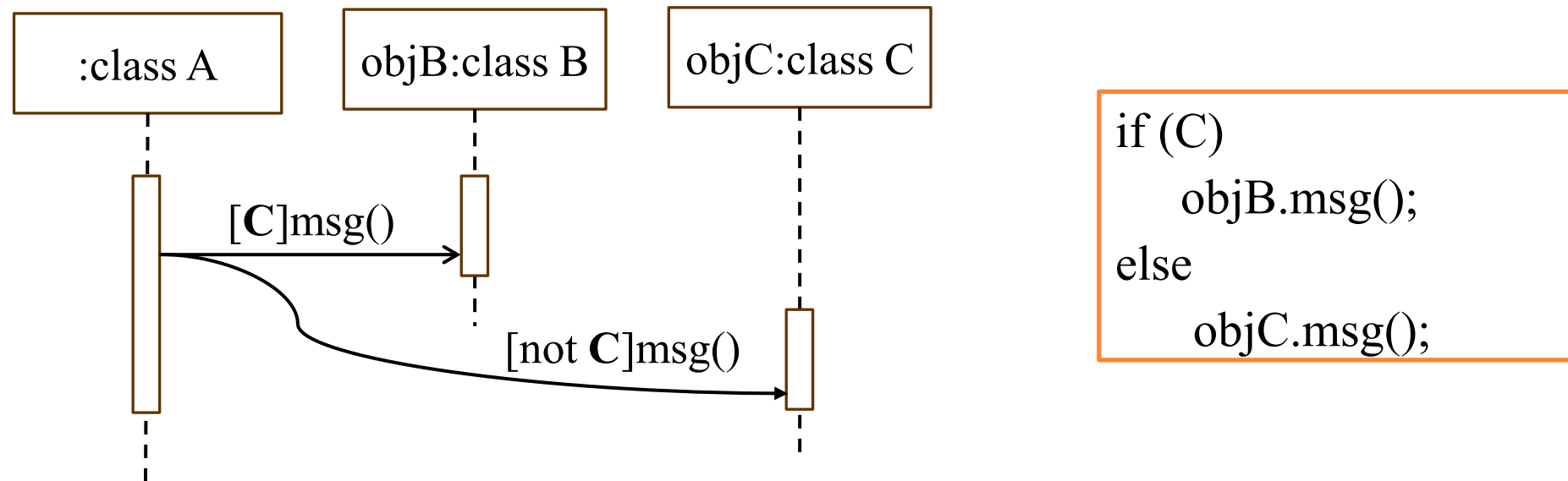
4.5.3.3. Các thành phần của biểu đồ

- Ví dụ 3: Thông điệp gửi có thể được gửi “lặp” nhiều lần phụ thuộc vào một điều kiện



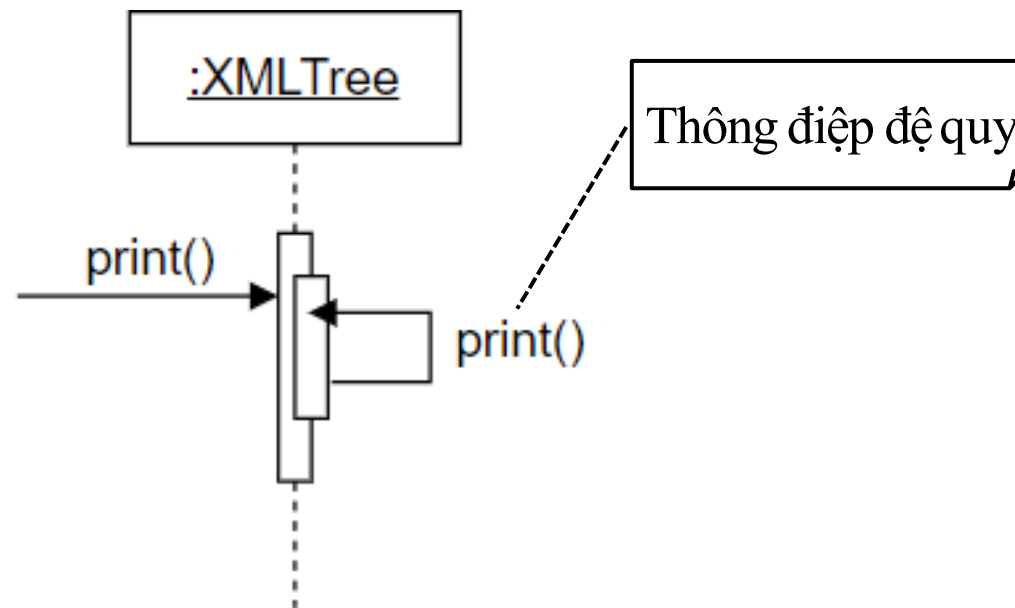
4.5.3.3. Các thành phần của biểu đồ

- Ví dụ 4: Thông điệp gửi có thể được gửi phụ thuộc vào điều kiện rẽ nhánh



4.5.3.3. Các thành phần của biểu đồ

- Ví dụ 5: Thông điệp gửi được gọi là đệ quy

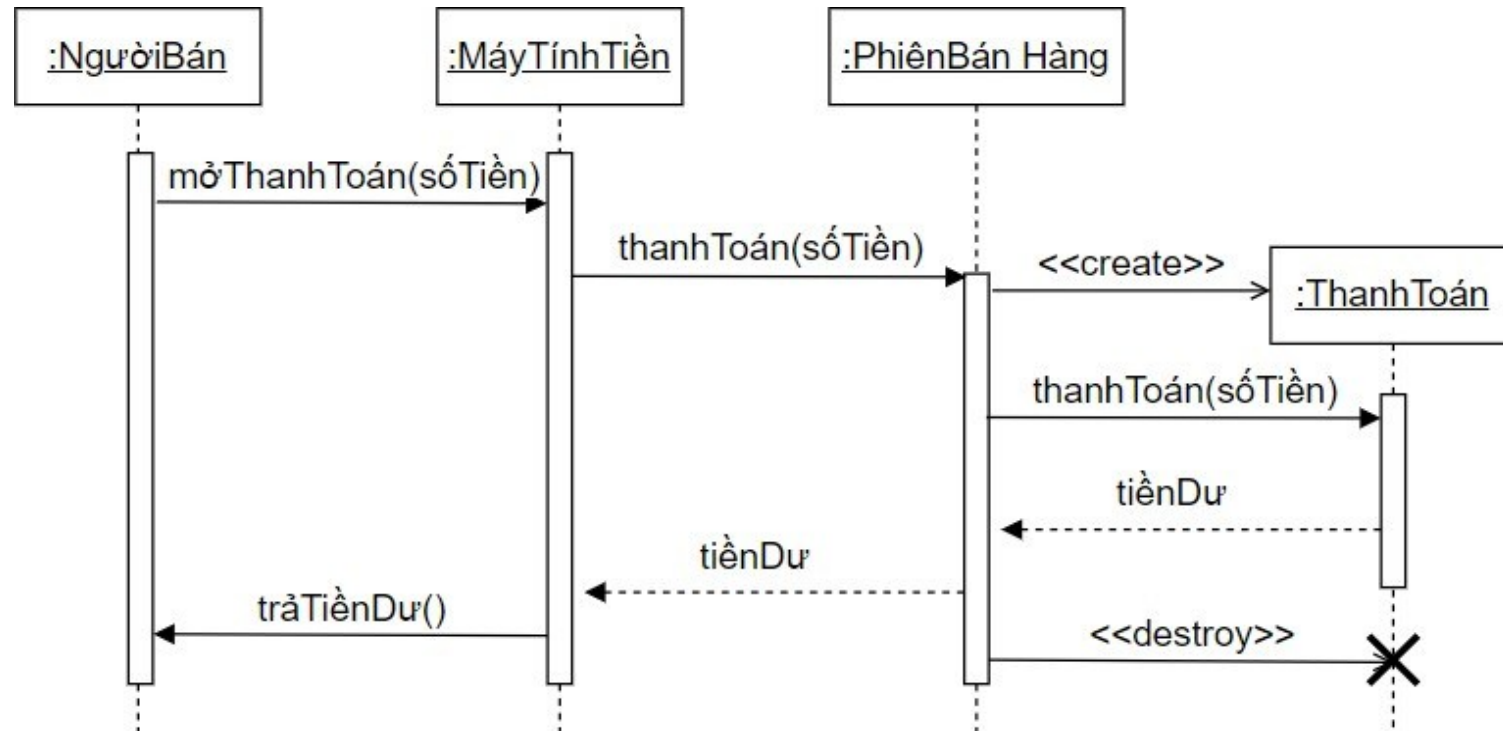


4.5.3.3. Các bước xây dựng biểu đồ

- Xác định các đối tượng trong tương tác.
- Xác định các thông điệp trao đổi giữa các đối tượng.
- Sắp xếp thông điệp theo trật tự thời gian.
- Xác định trạng thái và thời gian hoạt động của các đối tượng.
- Vẽ biểu đồ theo chuẩn UML.

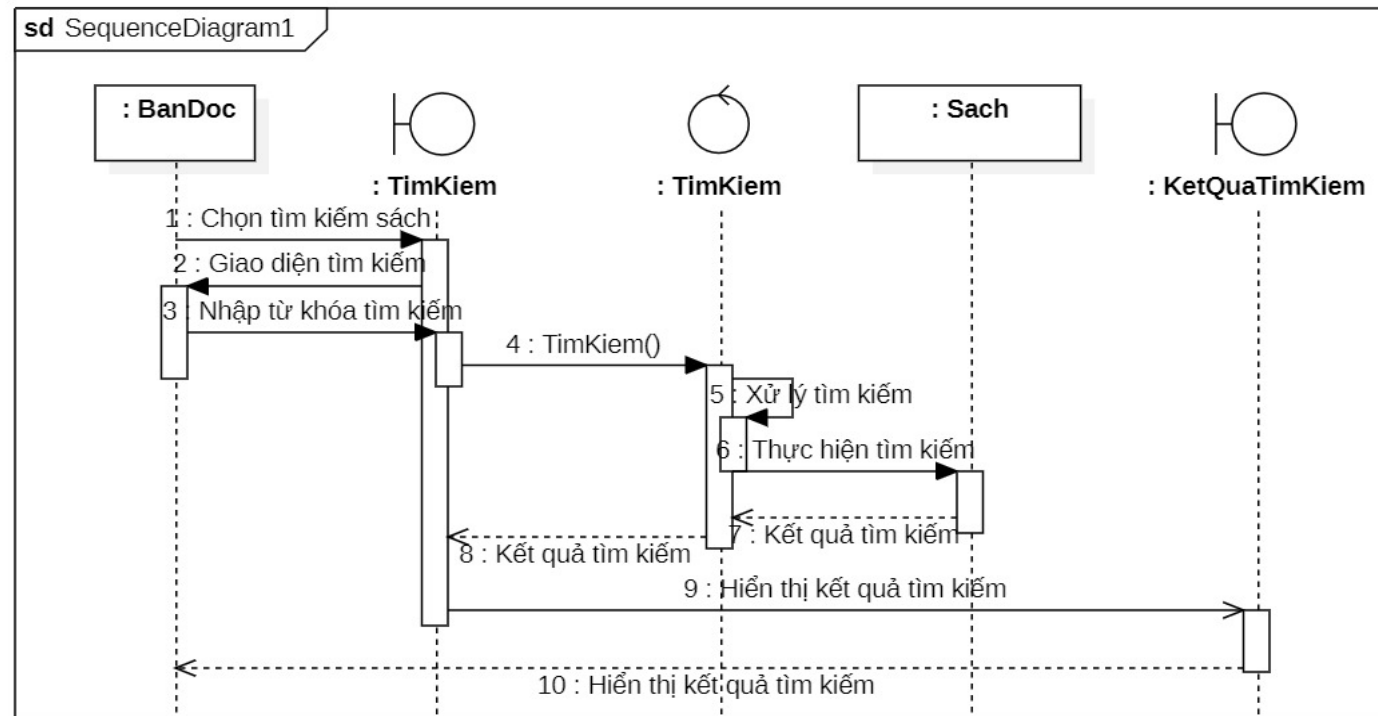
4.5.3.4. Ví dụ minh họa

- Biểu đồ tuần tự tác vụ “Thanh toán trực tiếp” tiền mua hàng



4.5.3.4. Ví dụ minh họa

- Biểu đồ tuần tự tác vụ “Tìm kiếm sách” của bạn đọc



4.5.4. Biểu đồ cộng tác

- Ý nghĩa
- Các thành phần của biểu đồ
- Các bước xây dựng biểu đồ
- Ví dụ minh họa

4.5.4.1. Ý nghĩa

- Biểu đồ cộng tác (collaboration diagram) mô tả sự tương tác giữa các đối tượng bằng việc nhấn mạnh cấu trúc kết hợp giữa các đối tượng và thông điệp trao đổi giữa chúng

4.5.4.2. Các thành phần của biểu đồ

- Đối tượng (Object): Các thành phần tham gia tương tác.
- Liên kết (Đường nối - Link): Thể hiện mối quan hệ giữa các đối tượng.
- Thông điệp (Message): Thông tin trao đổi giữa các đối tượng, được đánh số thứ tự tương tác.
- Số thứ tự (Sequence Number): Xác định trình tự trao đổi thông điệp.

4.5.4.2. Các thành phần của biểu đồ

- Đối tượng (Object): Các thành phần tham gia tương tác.
- Liên kết (Đường nối - Link): Thể hiện mối quan hệ giữa các đối tượng.
- Thông điệp (Message): Thông tin trao đổi giữa các đối tượng, được đánh số thứ tự tương tác.
- Số thứ tự (Sequence Number): Xác định trình tự trao đổi thông điệp.

4.5.4.2. Các thành phần của biểu đồ

- Cấu trúc thông điệp dạng tổng quát:

precondition/condition **sequence*** *||iteration :
result:=message(parameters)

- **precondition** (Điều kiện tiên quyết): Là điều kiện cần thỏa mãn trước khi thông điệp có thể được gửi, được đặt trước /
 - Ví dụ, giả sử thông điệp 1 là: dangnhap(), **1**/... (chỉ gửi thông điệp nếu đăng nhập thành công)

4.5.4.2. Các thành phần của biểu đồ

○ **condition** (Điều kiện): điều kiện đúng, thông điệp sẽ được gửi, được đặt trong []

■ Ví dụ: [**soluong > 0**] (chỉ gửi thông điệp nếu hàng còn trong kho)

○ **sequence** (Thứ tự thực thi): Đánh số thứ tự thực hiện thông điệp

■ Ví dụ, thông điệp **1:KiemtraKho()**; thông điệp **2.1:Giamgia()** và **2.2:Tinh tong()** nằm trong luồng của thông điệp **2:Taodonhang()**

4.5.4.2. Các thành phần của biểu đồ

- *****: thông điệp được gửi đi nhiều lần một cách tuần tự
 - Ví dụ, 1.3*call() (thông điệp này được gửi đi nhiều lần)
- ***||[iteration]**: Nếu một thông điệp được gửi nhiều lần trong một vòng lặp với cú pháp [iteration]
 - Ví dụ, *||[i=1..5] 1.2:close() (thông điệp này được gửi đi 5 lần)

4.5.4.2. Các thành phần của biểu đồ

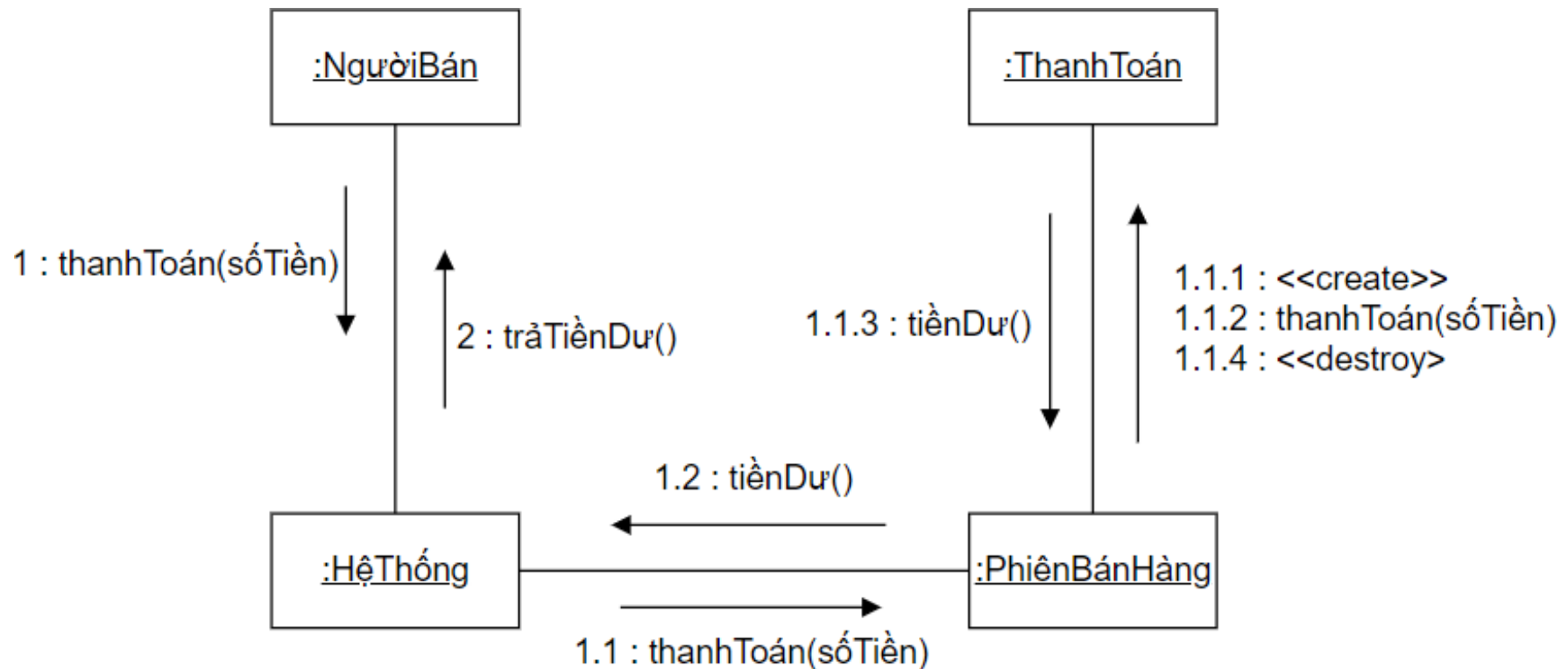
- **result** (Giá trị trả về): Là giá trị mà đối tượng nhận trả về cho đối tượng gửi sau khi thực hiện thông điệp
 - Ví dụ, **tongcong** := tongtien(thanhtien)
- **message()** (Tên thông điệp): là hành động hoặc phương thức được gọi
 - Ví dụ, xulythanhtoan() (phương thức xử lý thanh toán)
- **parameters** (Tham số): Là dữ liệu truyền vào thông điệp
 - Ví dụ, xulythanhtoan(soThe,soTien) (số thẻ và số tiền cần thanh toán)

4.5.4.3. Các bước xây dựng biểu đồ

- Xác định các đối tượng tham gia tương tác.
- Xác định mối quan hệ giữa các đối tượng.
- Xác định các thông điệp trao đổi.
- Xác định thứ tự trao đổi thông điệp.
- Vẽ biểu đồ hoàn chỉnh.

4.5.4.4. Ví dụ minh họa

- Biểu đồ cộng tác của tác vụ “Thanh toán trực tiếp” tiền mua hàng



4.6. Biểu đồ thành phần

- Ý nghĩa
- Các thành phần của biểu đồ
- Các bước xây dựng biểu đồ
- Ví dụ minh họa

4.6.1. Ý nghĩa

- Được sử dụng để mô tả các thành phần của hệ thống thông tin và cách chúng tương tác với nhau
- Giúp thể hiện cấu trúc của hệ thống cần phát triển bằng cách biểu diễn các mô-đun chính và cách chúng liên kết với nhau thông qua giao diện (interface)
- Giúp việc phát triển và bảo trì hệ thống dễ dàng hơn

4.6.2. Các thành phần của biểu đồ

- Thành phần (Component)
 - Đại diện cho một mô-đun hoặc phần của hệ thống có thể được phát triển và tái sử dụng độc lập
- Giao diện (Interface)
 - Là điểm truy cập vào thành phần, giúp thành phần giao tiếp với các thành phần khác.

4.6.2. Các thành phần của biểu đồ

- Mỗi quan hệ giữa các thành phần
 - Phụ thuộc (Dependency): Một thành phần sử dụng dịch vụ của thành phần khác.
 - Hiện thực giao diện (Realization): Thành phần triển khai một giao diện cụ thể.

4.6.3. Các bước xây dựng biểu đồ

- Bước 1: Xác định các thành phần chính của hệ thống
 - Xác định các mô-đun phần mềm có thể tách riêng thành các thành phần độc lập. Ví dụ: Quản lý sản Phẩm; Quản lý Giỏ hàng; Quản lý đơn hàng; Cơ sở dữ liệu: Khách hàng, Sản phẩm, Đơn hàng,...
- Bước 2: Xác định giao diện giữa các thành phần
 - Xác định cách các thành phần giao tiếp với nhau thông qua giao diện hoặc cổng. Ví dụ: Trang giỏ hàng, Trang đơn hàng,...

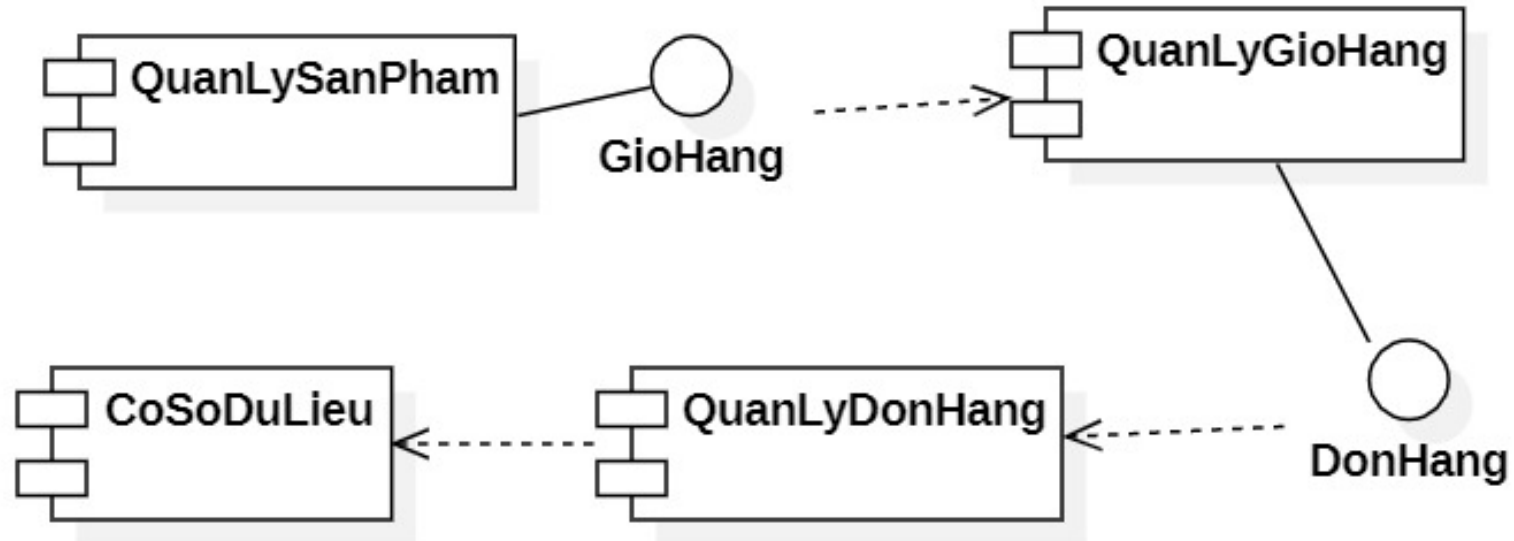
4.6.3. Các bước xây dựng biểu đồ

- Bước 3: Thiết lập các kết nối giữa các thành phần
 - Sử dụng mối quan hệ phụ thuộc hoặc hiện thực giao diện để mô tả sự tương tác giữa các thành phần. Ví dụ: Trang sản phẩm thực hiện giao diện Trang giỏ hàng
- Bước 4: Vẽ biểu đồ
 - Sử dụng các ký hiệu UML để vẽ biểu đồ thành phần với ba tầng kiến trúc.

4.6.4. Ví dụ minh họa

- Quản lý sản phẩm thực hiện giao diện Trang giỏ hàng để quản lý giỏ hàng.
- Quản lý giỏ hàng thực hiện giao diện Trang đơn hàng thực hiện quản lý đơn hàng để lưu đơn hàng của khách hàng vào cơ sở dữ liệu.
- Quản lý đơn hàng thực hiện thao tác trên cơ sở dữ liệu.

4.6.4. Ví dụ minh họa



4.7. Biểu đồ gói

- Ý nghĩa và vai trò
- Nguyên tắc đóng gói
- Các thành phần của biểu đồ
- Các bước xây dựng biểu đồ
- Ví dụ minh họa

4.7.1. Ý nghĩa và vai trò

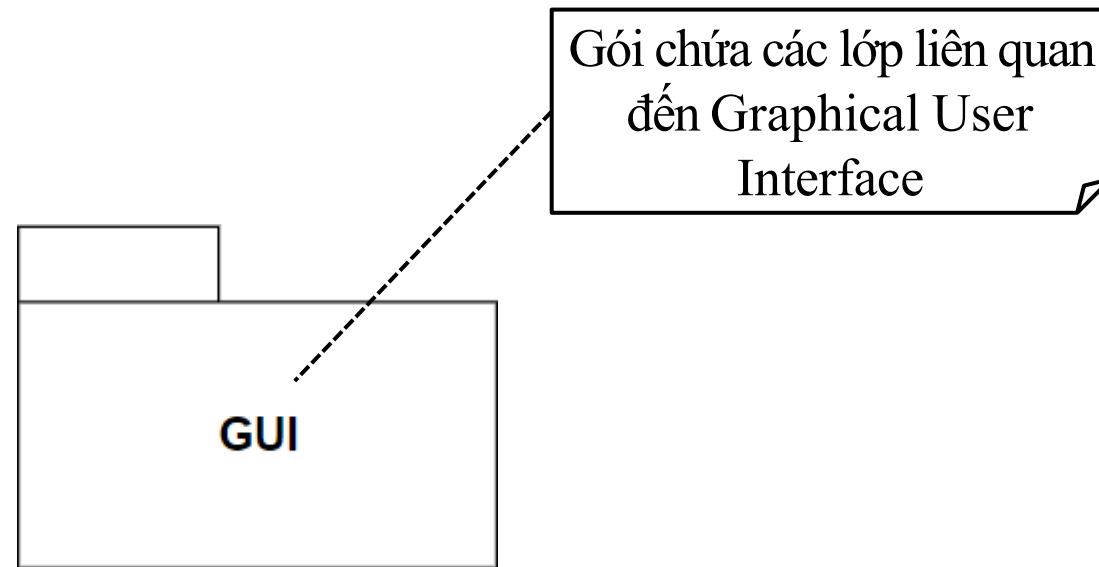
- Được sử dụng để nhóm các phần tử có liên quan lại với nhau thành các gói (packages).
- Sử dụng để tổ chức các lớp (class), mô-đun trong hệ thống phần mềm và mối quan hệ giữa chúng
- Giúp quản lý sự phức tạp của hệ thống. Hỗ trợ tái sử dụng mã nguồn, giúp việc bảo trì và phát triển phần mềm dễ dàng hơn

4.7.2. Nguyên tắc đóng gói

- Sự cố kết (cohesion) cao: tính cố kết thể hiện các lớp trong cùng một gói phải hợp tác cùng nhau để hướng tới cùng mục đích
- Sự móc nối (coupling) kém: là sự ràng buộc ảnh hưởng nhau giữa các gói. Sự móc nối giữa các gói càng lỏng lẻo càng tốt

4.7.3. Các thành phần của biểu đồ

- Gói (Package): là một nhóm chứa các phần tử chẳng hạn như: các lớp (thực thể, giao diện và điều khiển) hoặc các gói con khác

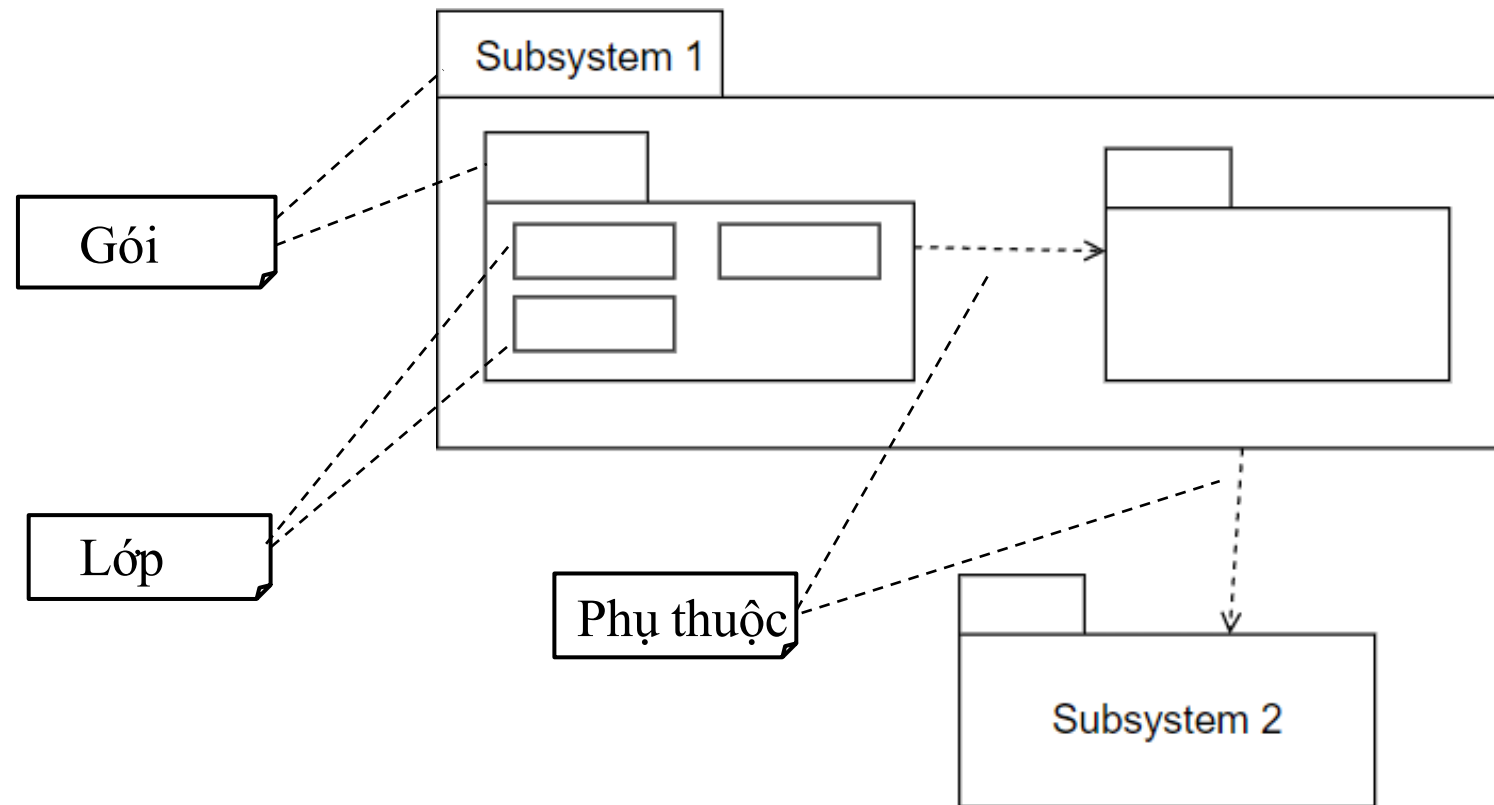


4.7.2. Các thành phần của biểu đồ

- Mỗi quan hệ giữa các gói:
 - Phụ thuộc (dependency): Một gói phụ thuộc vào một gói khác nếu nó sử dụng hoặc yêu cầu các thành phần từ gói kia.
 - Nhập (import): Một gói có thể nhập toàn bộ hoặc một phần của một gói khác.
 - Truyền tải (access): Một gói có thể truy cập một phần tử cụ thể trong một gói khác.

4.7.2. Các thành phần của biểu đồ

○Ký hiệu



4.7.3. Các bước xây dựng biểu đồ

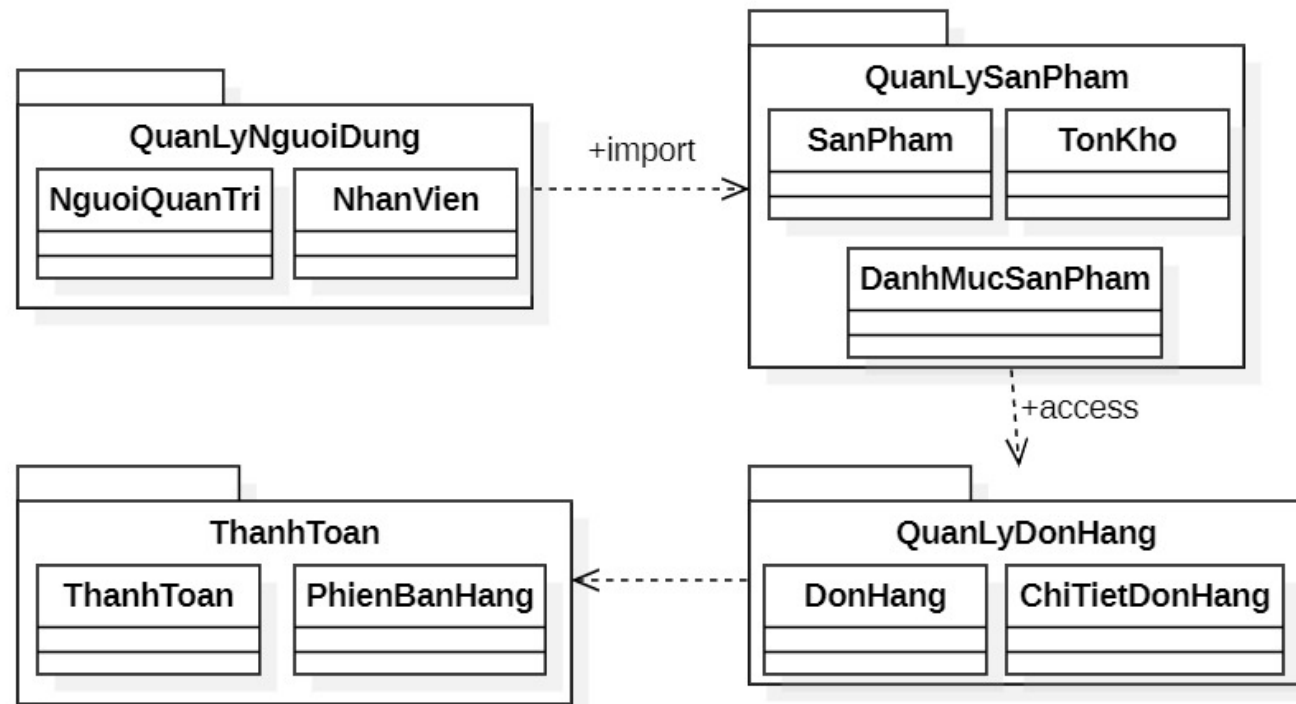
- Xác định các nhóm chức năng chính trong hệ thống có thể tổ chức thành các gói.
 - Ví dụ: Trong hệ thống bán hàng, có thể nhóm các chức năng như Quản lý người dùng, Quản lý sản phẩm, Quản lý đơn hàng,...
- Bước 2: Xác định xem gói sẽ chứa những lớp hoặc mô-đun nào.
 - Ví dụ, gói Quản lý sản phẩm có thể chứa các lớp như Sản phẩm, Danh mục sản phẩm, Tồn kho.

4.7.3. Các bước xây dựng biểu đồ

- Bước 3: Xác định mối quan hệ giữa các gói như phụ thuộc, nhập, hoặc truy cập.
 - Ví dụ, gói Thanh toán có thể phụ thuộc vào gói Quản lý đơn hàng để lấy thông tin về đơn hàng cần thanh toán.
- Bước 4: Vẽ biểu đồ gói UML
 - Dùng các ký hiệu UML để biểu diễn gói và mối quan hệ giữa chúng.

4.7.4. Ví dụ minh họa

- Biểu đồ gói của Hệ thống bán hàng



4.8. Biểu đồ triển khai

- Ý nghĩa
- Các thành phần của biểu đồ
- Các bước xây dựng biểu đồ
- Ví dụ minh họa

4.8.1. Ý nghĩa

- Biểu đồ triển khai dùng để mô tả sự phân bố các thành phần phần mềm trên các phần tử phần cứng của hệ thống.
- Giúp các nhà phát triển hiểu được hệ thống hoạt động như thế nào trên các môi trường khác nhau.

4.8.2. Các thành phần của biểu đồ

- Nút (node), đại diện cho phần tử phần cứng (server, client, database, thiết bị IoT,...)
- Thành phần (component), biểu diễn các phần tử phần mềm
- Quan hệ giữa chúng
 - Communication: Quan hệ kết nối giữa các node.
 - Dependency: Mỗi quan hệ phụ thuộc giữa các thành phần

4.8.3. Các bước xây dựng biểu đồ

- Xác định các phần tử phần cứng của hệ thống (server, client, database).
- Xác định các thành phần cần triển khai.
- Xác định mối quan hệ giữa chúng.
- Vẽ biểu đồ triển khai trên công cụ UML

4.8.3. Các bước xây dựng biểu đồ

- Xây dựng biểu đồ theo kiến trúc 3 tầng:
 - Tầng trình diễn (Presentation Layer): Giao diện người dùng.
 - Tầng nghiệp vụ (Application Layer): Xử lý logic (ứng dụng) của hệ thống.
 - Tầng dữ liệu (Database Layer): Tương tác với cơ sở dữ liệu.
 - Các dịch vụ dùng chung (Common Services) trên môi trường Internet

4.8.4. Ví dụ minh họa Thương mại điện tử

- Các nút:

- **Server, Client**

- Tầng giao diện

- **Web Browser:** Trình duyệt web của người dùng để truy cập ứng dụng
 - **Mobile App:** Ứng dụng di động cho người dùng truy cập.

4.8.4. Ví dụ minh họa Thương mại điện tử

○ Tầng ứng dụng

- **Web Server:** Máy chủ web xử lý các yêu cầu từ trình duyệt web và ứng dụng di động.
- **Application Server:** Máy chủ ứng dụng xử lý logic nghiệp vụ

○ Tầng dữ liệu

- **Database Server:** Máy chủ cơ sở dữ liệu lưu trữ thông tin người dùng, sản phẩm, đơn hàng, khuyến mãi, khách hàng và các tương tác.

4.8.4. Ví dụ minh họa Thương mại điện tử

- Các dịch vụ dùng chung (Common Services)
 - **Authentication Service:** Xác thực người dùng và phân quyền truy cập.
 - **Email Service:** Gửi email xác nhận đăng ký, thông báo đơn hàng, khuyến mãi.
 - **Payment Gateway:** Xử lý thanh toán trực tuyến.
 - **Logging and Monitoring Service:** Ghi log hoạt động và giám sát hệ thống.
 - **Backup and Recovery Service:** Sao lưu và khôi phục dữ liệu.

4.8.4. Ví dụ minh họa Thương mại điện tử

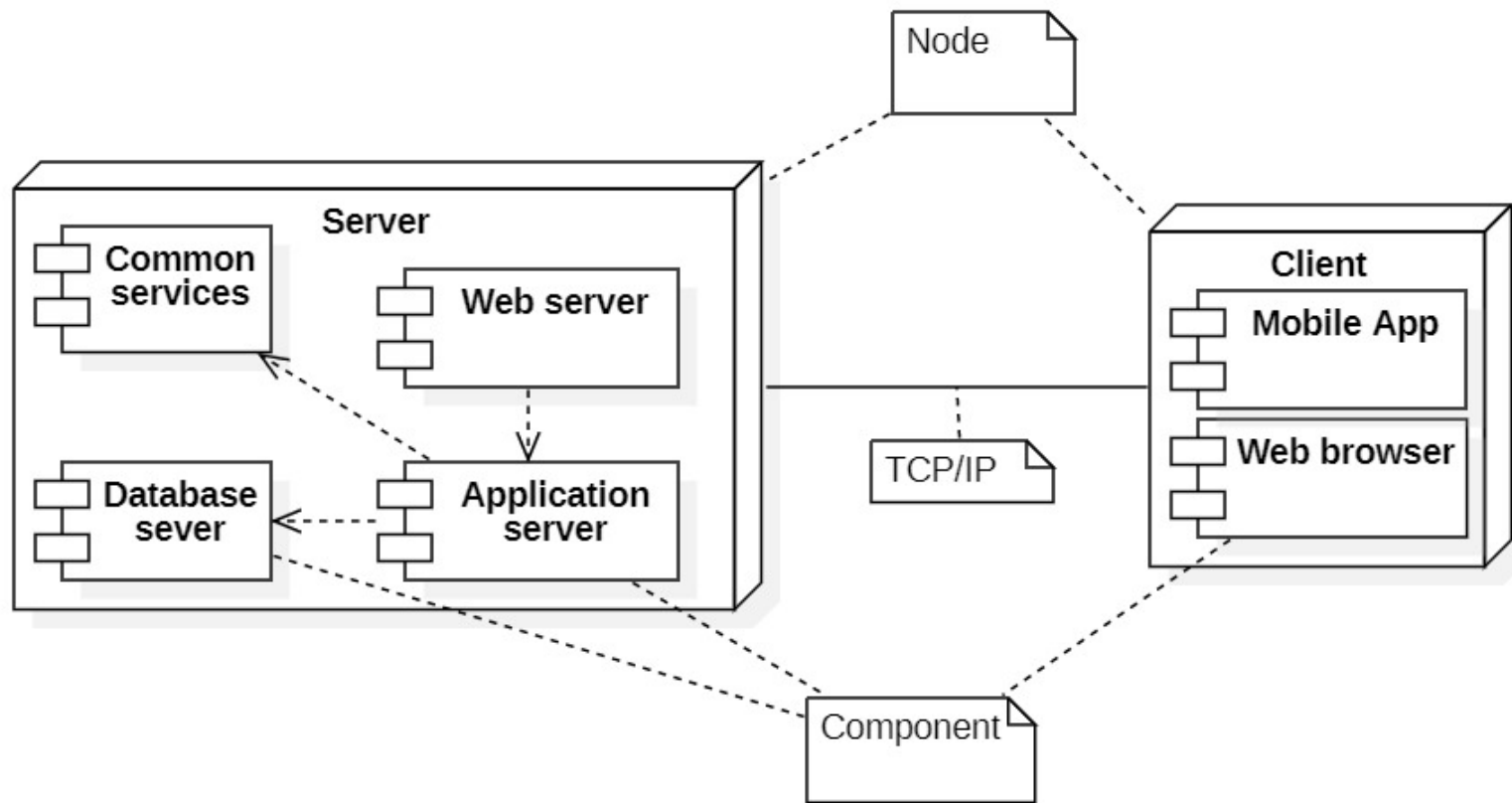
○ Mô tả triển khai

1. Người dùng truy cập ứng dụng thông qua Web Browser hoặc Mobile App.
2. Web Browser hoặc Mobile App gửi yêu cầu đến Web Server.
3. Web Server chuyển tiếp yêu cầu đến Application Server để xử lý logic nghiệp vụ.
4. Application Server tương tác với Database Server để truy xuất hoặc cập nhật dữ liệu.

4.8.4. Ví dụ minh họa Thương mại điện tử

5. Application Server sử dụng các Common Services như Authentication Service, Email Service, Payment Gateway, Logging and Monitoring Service, và Backup and Recovery Service để hỗ trợ các chức năng của hệ thống.
6. Application Server trả kết quả về Web Server.
7. Web Server gửi phản hồi về Web Browser hoặc Mobile App để hiển thị cho người dùng.

4.8.4. Ví dụ minh họa Thương mại điện tử



Cảm ơn các bạn sinh viên!